Ciencia de Datos en la Terminal Linux con Nushell

Luis Gómez Guzmán

Índice

1. Introducción **10**

2. Instalación de Software **13**
2.1. Windows 10+ . . . 13
2.1.1. Instalación Ubuntu WSL . . . 13
2.1.2. Instalación de Docker . . . 13
2.1.3. Abrir distribución . . . 13
2.2. Linux, WSL2 y el Contenedor `dsin` (Data Science in Nushell) . . . 14
2.2.1. Instalación de Docker en Linux / WSL (basado en Debian/Ubuntu) . . . 14
2.2.2. Instalación de Docker en Linux (basado en Arch / CachyOS / Manjaro) . . . 14
2.2.3. Descarga y Ejecución del Contenedor `dsin` . . . 14
2.2.4. Crear alias para llamado simple (opcional) . . . 15

3. Fundamentos y Estructuras de Datos en Nushell **16**
3.1. Ciencia de Datos . . . 16
3.1.1. Obtención de datos . . . 17
3.1.2. Depuración de datos . . . 17
3.1.3. Exploración de datos . . . 17
3.1.4. Modelación de datos . . . 17
3.1.5. Interpretación de datos . . . 18
3.2. Historia de la terminal . . . 18
3.2.1. Shell . . . 19
3.2.2. Ventajas del uso de la terminal . . . 19
3.3. La Terminal . . . 19
3.3.1. El Ambiente . . . 20
3.3.2. Ejecutar un Comando y Guardar Resultados en Archivos . . . 20
3.3.3. Tipos de Herramientas en la Línea de Comandos . . . 21
3.3.4. Otros Shells como Bash y Zsh: Qué Hacen, su Filosofía y su Uso . . . 22
3.3.4.1. ¿Qué es y qué hace un Shell Tradicional? . . . 22
3.3.4.2. La Filosofía Unix y los Flujos de Texto Plano . . . 22
3.3.4.3. Uso y Sintaxis Habitual en Bash . . . 22

3.3.4.4. ¿Por qué el Modelo de Texto Plano de Bash es Frágil para la Ciencia de Datos? . . . 23
3.4. Como entender nu . . . 24
3.4.1. Nushell no es bash . . . 24
3.4.2. Nushell como un lenguaje compilado . . . 24
3.4.3. Las variables son inmutables por defecto . . . 25
3.5. Elementos básicos de nushell . . . 25
3.5.1. Obtener Ayuda . . . 27
3.6. Moviéndose por el Sistema . . . 28
3.6.1. Viendo el Contenido de un Directorio . . . 28
3.6.2. Patrones Glob (comodines) . . . 28
3.6.2.1. Conversión de Cadenas a Globs . . . 29
3.7. Trabajando con strings . . . 30
3.7.1. Formatos de String . . . 30
3.7.2. Strings con Comillas Simples . . . 31
3.7.3. Strings con Comillas Dobles . . . 31
3.7.4. Strings Raw . . . 31
3.7.5. Strings de Palabra Simple . . . 32
3.7.6. Strings con Comillas Invertidas . . . 33
3.7.7. Strings como comandos externos . . . 33
3.7.8. Añadir y anteponer a strings . . . 33
3.7.9. Interpolación de strings . . . 34
3.7.10. Dividir Strings . . . 35
3.7.11. El comando `str` . . . 35
3.7.11.1. Recortar Strings . . . 35
3.7.11.2. Substrings . . . 36
3.7.11.3. Relleno de Strings . . . 36
3.7.11.4. Invertir Strings . . . 36
3.7.11.5. Análisis de Strings . . . 36
3.7.12. Comparación de Strings . . . 36
3.7.13. Convertir Strings . . . 37
3.7.13.1. A string . . . 37
3.7.13.2. De string . . . 37
3.7.14. Colorear Strings . . . 37
3.8. Trabajando con listas . . . 37
3.8.1. Creando listas . . . 37
3.8.2. Actualizando listas . . . 38
3.8.3. Eliminando o Agregando Elementos de la Lista . . . 38
3.8.3.1. Usando el Operador de Propagación . . . 39
3.8.4. Iterando sobre Listas . . . 39
3.8.5. Accediendo a la Lista . . . 40
3.8.6. Convirtiendo la Lista . . . 41
3.9. Trabajando con Registros (records) . . . 42

3.9.1. Creando registros . . . 42
3.9.2. Actualizando Registros . . . 43
3.9.3. Iterando sobre un Registro . . . 43
3.10. Trabajando con tablas . . . 44
3.10.1. Ordenando los Datos . . . 44
3.10.2. Seleccionando Datos . . . 44
3.10.3. Obteniendo Datos de una Tabla . . . 44
3.10.4. Cambiando Datos en una Tabla . . . 45
3.10.4.1. Concatenando Tablas . . . 45
3.10.4.2. Combinando Tablas . . . 45
3.10.5. Añadiendo una columna . . . 46
3.10.6. Actualizando una Columna . . . 47
3.10.7. Moviendo Columnas . . . 47
3.10.8. Renombrando Columnas . . . 47
3.10.9. Rechazando/Eliminando Columnas . . . 47
3.11. Navegando y Accediendo a Datos Estructurados . . . 48
3.11.1. Antecedentes . . . 48
3.11.2. Rutas de Celda . . . 48
3.11.2.1. Registros . . . 48
3.11.2.2. Listas . . . 49
3.11.2.3. Tablas . . . 49
3.11.2.3.1. Ejemplo - Acceder a una Fila de la Tabla (Registro) . . . 49
3.11.2.3.2. Ejemplo - Acceder a una Columna de la Tabla (Lista) . . . 50
3.11.2.3.3. Ejemplo - Acceder a una Celda de la Tabla (Valor) . . . 50
3.11.3. Datos Anidados . . . 50
3.11.3.1. Ejemplo - Accediendo a Datos de Tabla Anidados . . . 50
3.11.4. Usando `get` y `select` . . . 50
3.11.4.1. Ejemplo - `get` vs. `select` una fila de la tabla . . . 51
3.11.4.2. Ejemplo - `select` con múltiples filas y columnas . . . 51
3.11.5. Nombres de Clave/Columna con espacios . . . 51
3.11.6. Manejo de Datos Faltantes . . . 52
3.11.6.1. El Operador Opcional . . . 52
3.11.6.1.1. Ejemplo - El Operador Opcional . . . 52
3.11.6.2. Asignar un `default` para datos faltantes o `null` . . . 52
3.11.7. Otros comandos para acceder a datos estructurados . . . 52
3.12. Edición de Archivos en la Terminal con `ttt` . . . 53
3.12.1. Uso Básico de `ttt` . . . 53
3.13. Control de Versiones . . . 53
3.13.1. Git: La herramienta esencial . . . 54
3.13.2. GitHub/GitLab . . . 55
3.13.3. Mejores prácticas para proyectos de ciencia de datos . . . 56
3.13.4. Clonar . . . 56
3.13.5. Commit y Push . . . 57

3.13.6. Hacer aportes a un repositorio . . . 57
3.13.7. Sugerencias de mensajes commit . . . 58
3.13.8. Recursos adicionales . . . 59
3.13.9. Para mayor exploración . . . 59

4. Programación y Creación de Herramientas de Línea de Comandos 60
4.1. Programación en Nushell . . . 60
4.1.1. Operadores . . . 60
4.1.1.1. Orden de Operaciones . . . 61
4.1.1.2. Tipos . . . 62
4.1.1.3. Expresión Regular / Operadores string-contains . . . 62
4.1.1.4. Sensibilidad a Mayúsculas y Minúsculas . . . 62
4.1.1.5. Operador de propagación . . . 63
4.1.1.5.1. En literales de Lista . . . 63
4.1.1.5.2. En literales de Registro . . . 64
4.1.1.5.3. En llamadas de Comando . . . 64
4.1.2. Variables . . . 65
4.1.2.1. Tipos de Variables . . . 65
4.1.2.1.1. Variables Inmutables . . . 65
4.1.2.1.2. Variables Mutables . . . 66
4.1.2.1.3. Variables Constantes . . . 67
4.1.2.2. Escogiendo entre variables mutables e inmutables . . . 67
4.1.2.2.1. Consideraciones de Rendimiento . . . 68
4.1.2.3. Nombres de Variables . . . 69
4.1.3. Comandos personalizados . . . 70
4.1.4. Nombres de Comandos . . . 70
4.1.5. Subcomandos . . . 71
4.1.6. Tipos de Parámetros . . . 71
4.1.7. Parámetros con un Valor Predeterminado . . . 72
4.1.8. Parámetros Posicionales Opcionales . . . 73
4.1.9. Opciones (*flags*) . . . 74
4.1.10. Parámetros rest . . . 76
4.1.11. Documentando un Comando . . . 76
4.1.12. Salida de la *pipeline* . . . 78
4.1.13. Entrada de la *pipeline* . . . 78
4.1.14. Control de flujo . . . 78
4.1.14.1. Elección (Condicionales) . . . 79
4.1.14.1.1. `if` . . . 79
4.1.14.1.2. `match` . . . 80
4.1.14.2. Bucles . . . 81
4.1.14.2.1. Bucles y Otros Comandos de Iteración . . . 81
4.1.14.2.2. `for` . . . 83
4.1.14.2.3. `while` . . . 83

4.1.14.2.4. `loop` . . . 83
4.1.14.2.5. `break` . . . 84
4.1.14.2.6. `continue` . . . 84
4.1.14.2.7. Errores . . . 84
4.1.14.2.8. Otros . . . 85
4.2. Ejemplo de Desarrollo de Comando: API de Google Maps y Geocodificación . . . 86
4.2.1. Obtener API Key . . . 86
4.2.2. Exploración de la API y Anatomía de una URL . . . 86
4.2.3. Empaquetando el Módulo `maps.nu` con `ttt` . . . 88
4.3. Conversión de One-Liners en Herramientas de Línea de Comandos . . . 88
4.3.1. Paso 1: Crear el Archivo . . . 89
4.3.2. Paso 2: Dar Permisos de Ejecución . . . 90
4.3.3. Paso 3: Definir el Shebang . . . 90
4.3.4. Paso 4: Remover la Entrada Fija . . . 90
4.3.5. Paso 5: Añadir Argumentos y Opciones Tipadas (`def main`) . . . 90
4.3.6. Paso 6: Extender el `$env.PATH` . . . 91
4.4. Creando Herramientas de Línea de Comandos con Python y R . . . 91
4.4.1. Portando la Herramienta a Python (`top-words.py`) y R (`top-words.R`) . . . 91
4.4.2. Procesando Datos en Transmisión (*Streaming*) desde `stdin` (`fizzbuzz.py` y `fizzbuzz.R`) . . . 91

5. Obtención de Datos **93**
5.1. Copiando Archivos Locales en el Contenedor Docker . . . 93
5.2. Descargando desde Internet con `http get` . . . 93
5.2.1. Descargando Archivos JSON y Páginas Web . . . 93
5.2.2. Descargando desde Múltiples URLs y Consultas `http post` . . . 94
5.2.3. Siguiendo Redirecciones e Inspeccionando Cabeceras HTTP (`http head`) . . . 94
5.3. Descomprimiendo Archivos . . . 94
5.4. Conversión de Hojas de Cálculo de Microsoft Excel y Archivos CSV . . . 95
5.5. Consultando Bases de Datos Relacionales (SQLite) . . . 96
5.6. Llamando a las API Web . . . 97
5.6.1. Autenticación con Clave API . . . 97
5.6.2. API de Transmisión (*Streaming*) . . . 97
5.7. Descargando datos . . . 98
5.7.1. Descargando un archivo json . . . 98
5.7.2. Descargando desde multiples urls . . . 98

6. Depuración y Transformación de Datos **101**
6.1. Transformaciones en un Pipeline de Datos . . . 101
6.2. Texto Plano . . . 101
6.2.1. Filtrando Líneas . . . 101
6.2.1.1. Basado en la Ubicación . . . 101
6.2.1.2. Basado en un Patrón . . . 102

6.2.1.3. Basado en Aleatoriedad (Muestreo) . . . 102
6.2.2. Extrayendo Valores . . . 103
6.2.3. Reemplazando y Eliminando Valores . . . 103
6.3. Formateando salida de comandos (parsing) . . . 104
6.3.1. `detect columns` (bastante automático) . . . 104
6.3.2. Usando `from ssv` . . . 104
6.3.3. Usando `parse` . . . 104
6.4. CSV y Tablas Estructuradas . . . 105
6.4.1. Cuerpos, Encabezados y Columnas . . . 105
6.4.2. Realizando Consultas SQL en CSV . . . 105
6.4.3. Extrayendo y Reordenando Columnas . . . 105
6.4.4. Filtrando Filas . . . 105
6.4.5. Combinar Columnas (y Robustez ante Comas Internas) . . . 106
6.4.6. Combinando Múltiples Archivos CSV . . . 106
6.4.6.1. Concatenar Horizontalmente (`merge`) . . . 106
6.4.6.2. Unión Relacional (`join`) . . . 106
6.5. Trabajando con XML/HTML y JSON . . . 107
6.6. El plugin polars de Nushell . . . 107
6.6.1. Casos de uso . . . 108
6.6.2. Conceptos básicos . . . 108
6.6.2.1. Diferencia entre DataFrames de Polars y Tablas de Nushell . . . 108
6.6.3. Uso básico . . . 109
6.6.3.1. Carga de datos . . . 109
6.6.3.2. Exploración de datos . . . 109
6.6.3.3. Conversión de tipos de datos . . . 110
6.6.4. Manipulación de datos . . . 111
6.6.4.1. Selección de columnas . . . 111
6.6.4.2. Filtrado de filas . . . 112
6.6.4.3. Ordenación de datos . . . 112
6.6.4.4. Agrupación y agregación . . . 112
6.6.5. Guardado y formatos . . . 114
6.6.6. Pivoteo de columnas . . . 114
6.6.7. Buenas prácticas y recomendaciones . . . 115
6.6.8. Conclusión . . . 116

7. Exploración y Visualización de Datos **117**
7.1. Inspeccionando Datos y sus Propiedades . . . 117
7.1.1. ¿Tiene Encabezado, Cuántos Registros y Qué Columnas? . . . 117
7.1.2. Inspeccionando Tipos de Datos e Inferencia de Esquemas . . . 118
7.1.3. Identificando Valores Únicos y Eliminando Columnas Constantes . . . 118
7.2. Calculando Estadísticas Descriptivas . . . 119
7.2.1. Estadísticas con `math` y `polars summary` en Nushell . . . 119
7.2.2. Transformaciones y Resúmenes Estadísticos con Scripts en R y Python . . . 119

7.3. Creando Visualizaciones Estadísticas desde la Línea de Comandos . . . 120
7.3.1. Visualizando Imágenes Directamente en la Terminal con `timg` . . . 120
7.3.2. Alternativa Remota: Copiando Gráficos con `scp` . . . 121
7.4. Catálogo de Gráficos Estadísticos con Scripts en R y Python . . . 121
7.4.1. Gráfico de Barras (`geom_bar`) . . . 121
7.4.2. Histograma (`geom_histogram`) . . . 122
7.4.3. Gráfico de Densidad (`geom_density`) . . . 122
7.4.4. Gráfico de Líneas para Series Temporales (`geom_line`) . . . 122
7.4.5. Diagrama de Dispersión y Tendencia (`geom_point` y `geom_smooth`) . . . 123
7.4.6. Diagrama de Caja (`geom_boxplot`) y Gráfico de Violín (`geom_violin`) . . . 123
7.4.7. Visualización Estadística Equivalente en Python (`matplotlib` / `seaborn`) . . . 123
7.5. Resumen . . . 123

8. Procesamiento en Serie y Pipelines Paralelos **125**
8.1. Procesamiento en Serie: Recorriendo Datos con `each` . . . 125
8.1.1. Iterando sobre Números . . . 125
8.1.2. Iterando sobre Líneas y Filas . . . 126
8.1.3. Iterando sobre Archivos . . . 126
8.2. Procesamiento en Paralelo en Nushell con `par-each` . . . 126
8.2.1. Ejecución Concurrente y Control de Hilos (`--threads` y `--keep-order`) . . . 126
8.3. Paralelización de Procesos con GNU `parallel` desde Nushell . . . 128
8.3.1. Especificando Argumentos y Marcadores de Reemplazo (`{}`, `{/}`, `{.}` y `{#}`) . . . 128
8.3.2. Controlando el Número de Trabajos Concurrentes (`--jobs`) . . . 129
8.3.3. Registro de Salidas y Etiquetado (`--results`, `--keep-order` y `--tag`) . . . 129
8.3.4. Creando Herramientas Paralelas y Procesamiento por Bloques (`--pipe`) . . . 130
8.4. Procesamiento Distribuido en Múltiples Nodos . . . 130
8.4.1. Obteniendo una Lista de Servidores Activos . . . 130
8.4.2. Ejecutando un Comando en Todos los Nodos (`--nonall`) . . . 130
8.4.3. Distribuyendo Datos Locales entre Nodos (`--pipe --slf`) . . . 130
8.4.4. Procesando Archivos en Nodos Remotos (`--transferfile` y `--return`) . . . 130
8.5. Resumen . . . 131

9. Modelado Políglota de Datos **132**
9.1. Preparación y Exploración del Conjunto de Datos de Vinos . . . 133
9.2. Reducción de Dimensionalidad (PCA y t-SNE) . . . 133
9.2.1. Reducción de Dimensionalidad con Herramientas CLI (`tapkee-reduce`: PCA y t-SNE) . . . 134
9.2.2. Reducción de Dimensionalidad con `Rscript` (`pca_wine.R`) . . . 134
9.2.3. Reducción de Dimensionalidad con Python (`pca_wine.py`) . . . 135
9.2.4. Diagnóstico de Multicolinealidad (VIF) y Regresión de Componentes Principales (PCR) . . . 135
9.3. Agrupamiento o Clustering . . . 136
9.3.1. Clustering K-Means con Herramientas CLI (`kmeans-cluster`) . . . 136

9.3.2. Clustering K-Means con `Rscript` (`cluster_wine.R`) . . . 137
9.3.3. Clustering K-Means con Python (`cluster_wine.py`) . . . 137
9.3.4. Clustering Jerárquico Aglomerativo (hclust_fish.R y hclust_fish.py) . . . 137
9.3.5. Determinación Óptima de K con Evaluación Multimétrica (cluster_eval.py) 138
9.3.6. Clustering Basado en Densidad con DBSCAN y K-Medoides con PAM . . . 139
9.4. Regresión: Predicción de la Calidad del Vino . . . 140
9.4.1. Regresión en Línea con Vowpal Wabbit (`vw-regress`) . . . 140
9.4.2. Regresión Lineal Múltiple con `Rscript` (`regress_wine.R`) . . . 140
9.4.3. Regresión Regularizada (Ridge) con Python (`regress_wine.py`) . . . 141
9.5. Clasificación Supervisada . . . 141
9.5.1. Experimentos de Clasificación con SKLL (`run_experiment` + `skll-metrics`) . 141
9.5.2. Clasificación con Regresión Logística en `Rscript` (`classify_wine.R`) . . . 141
9.5.3. Clasificación con Random Forest en Python (`classify_wine.py`) . . . 142
9.5.4. Árboles de Decisión con R y Python (dt_banco.R y dt_banco.py) . . . 142
9.5.5. Clasificación con Vecinos Más Cercanos k-NN (knn_banco.R y knn_banco.py) 142
9.5.6. Clasificación Alternativa: Naïve Bayes y Máquinas de Vectores de Soporte (nb_svm_banco.py) . . . 143
9.5.7. Redes Neuronales Artificiales: Perceptrón Multicapa (MLP) . . . 143
9.6. Resumen del Capítulo y Conclusión del Libro . . . 144

10. Para seguir explorando **146**
10.1. La Línea de Comandos . . . 146
10.2. Programación de Shell . . . 146
10.3. Python, R y SQL . . . 146
10.4. APIs . . . 146
10.5. Aprendizaje Automático . . . 146

11. Referencias **147**
11.1. Ecosistema Nushell . . . 147
11.2. Data Science at the Command Line (DSitCL) . . . 147
11.3. Herramientas de Línea de Comandos Integradas en `dsin` . . . 147
11.4. Bibliografía y Literatura Académica . . . 148

Índice de figuras

1. Proceso de la ciencia de datos. . . . 16
2. Primeros computadores. . . . 18
3. Primeras terminales. . . . 19
4. Primeras terminales de video. . . . 20
5. Primeras terminales conectadas a un computador. . . . 21
6. Redirección de salida estándar en la terminal . . . 23
7. Gráfico de barras generado con ggplot2 . . . 121
8. Histograma del total de la cuenta . . . 122

9. Estimación de densidad por núcleos 122
10. Serie temporal de accidentes 122
11. Diagrama de dispersión con curva de tendencia suavizada 123
12. Diagrama de caja de cuentas por día 123
13. Comparación conceptual entre ejecución secuencial y ejecución en paralelo 127
14. Flujo de ejecución de GNU parallel distribuyendo argumentos en múltiples núcleos . 128
15. Entrelazado de salidas cuando varios procesos escriben simultáneamente 129
16. Proyecciones bidimensionales obtenidas mediante reducción de dimensionalidad . . 134

1. Introducción

En la práctica contemporánea de la ciencia de datos, la escala masiva, la heterogeneidad y el dinamismo de los flujos de información exigen herramientas capaces de trascender la rigidez de las interfaces gráficas y la sobrecarga computacional propia de los entornos de desarrollo monolíticos. Durante más de cinco décadas, la consola de comandos de Unix se ha consolidado como el estándar de referencia para la reproducibilidad científica, la automatización desatendida y la eficiencia algorítmica. Su filosofía fundacional postula una premisa de notable vigencia: articular sistemas a partir de programas compactos y modulares, concebidos para resolver una tarea concreta con precisión técnica y acoplarse con naturalidad mediante canales estandarizados de comunicación.

Sin embargo, las shells tradicionales (entre las que destacan Bash y Zsh) fueron proyectadas en una época en que la información en tránsito se interpretaba exclusivamente como secuencias no estructuradas de caracteres. Bajo este esquema clásico, operar sobre matrices de datos, estructuras tabulares complejas o registros jerárquicos demanda intrincadas combinaciones de expresiones regulares y utilidades de corte léxico (tales como `sed`, `awk` y `cut`). Semejante paradigma introduce una fragilidad persistente; resulta vulnerable a delimitadores imprevistos, sufre ante secuencias de escape y, de modo crítico, despoja a las variables de sus atributos semánticos y tipos fundamentales.

Nushell (`nu`) supera de raíz esta limitación histórica mediante una arquitectura concebida expresamente para las exigencias del análisis de datos contemporáneo. En lugar de canalizar flujos de bytes planos y opacos, los pipelines de Nushell transportan estructuras de datos enriquecidas: tablas bidimensionales tipadas, registros jerárquicos indexados por clave y valor, listas ordenadas y valores primitivos regulados por esquemas estrictos. Al reconocer de forma nativa enteros, valores en coma flotante, marcas temporales con huso horario, rutas del sistema de archivos y magnitudes de memoria, Nushell permite filtrar, ordenar, remodelar y agregar conjuntos de información mediante una sintaxis declarativa, concisa y elegante.

A esta claridad expresiva se suma un desempeño computacional extraordinario. Su integración profunda con el motor columnar Polars (programado en Rust y edificado sobre el estándar en memoria Apache Arrow) dota a la terminal de una capacidad analítica sin precedentes. Esta sinergia hace viable ejecutar consultas analíticas multihilo sobre millones de registros en fracciones de segundo, preservando en todo momento la inmediatez y la ligereza del flujo interactivo en la línea de comandos.

Esta obra constituye una derivación directa y una evolución conceptual del trabajo pionero de Jeroen Janssens en su célebre libro *Data Science at the Command Line*, publicado por O'Reilly Media. Aquella contribución seminal demostró con total elocuencia que la línea de comandos de Unix dista de ser una reliquia del pasado; representa, más bien, un entorno ágil, versátil y privilegiado para todas las fases de la indagación cuantitativa. No obstante, el ecosistema del cómputo interactivo ha madurado sustancialmente en la última década.

Mientras la propuesta original de Janssens cimentaba sus flujos operativos sobre utilidades de filtrado textual y el instrumental provisto por csvkit, el presente volumen refundamenta la práctica analítica en torno al paradigma tipado de Nushell. Con este cambio de fundamento, los principios

de modularidad, reproducibilidad y composición que inspiraron a Janssens alcanzan un estándar superior de seguridad estática, expresividad y eficiencia. La manipulación fragmentaria de texto plano da paso, de esta manera, a un diálogo fluido y continuo con estructuras de datos vivas, gobernadas por esquemas rigurosos.

La presente obra no fue concebida para una lectura pasiva, contemplativa o abstracta, sino como un manual de experimentación computacional interactiva, proyectado para abordarse con una terminal abierta junto al texto en todo instante. En virtud de esta decisión metodológica, la inmensa mayoría de los bloques de código presentados omiten deliberadamente la salida textual producida por las instrucciones. Salvo en circunstancias donde se despliegan gráficos generados en la consola, imágenes visualizadas mediante herramientas auxiliares o diagramas conceptuales indispensables para ilustrar la arquitectura de los datos, las respuestas del sistema se han reservado de manera sistemática.

Esta determinación didáctica responde a tres postulados cardinales del aprendizaje técnico profundo. En primer lugar, estimula el compromiso activo frente a la ilusión de comprensión: contemplar una tabla de resultados previamente impresa adormece el esfuerzo analítico, mientras que escribir, adaptar y despachar cada instrucción sitúa al estudiante en contacto directo con la reactividad del intérprete, permitiéndole asimilar de forma tangible la mecánica interna de los pipelines. En segundo lugar, propicia la exploración en tiempo real y el descubrimiento guiado, pues las virtudes más notables de Nushell (tales como la navegación multidimensional con `explore`, la paginación interactiva de registros extensos, el autocompletado contextual y la introspección dinámica de esquemas) despliegan su auténtico valor formativo únicamente en el diálogo vivo con la terminal. En tercer lugar, forja la resiliencia diagnóstica requerida en la práctica profesional de la ingeniería estadística. La labor cotidiana con datos demanda convivir con discrepancias estructurales, advertencias de coerción y excepciones sintácticas; omitir salidas triviales consolida el hábito metódico de auditar los estados intermedios del procesamiento con `describe`, verificando tipos y dimensiones de manera autónoma. Convocamos, en consecuencia, a asumir una actitud de indagación incesante: no reproduzca los ejemplos como fórmulas mecánicas, altere los parámetros, formule filtros alternativos, combine operadores imprevistos y someta a examen riguroso la respuesta que su propia terminal le devuelva.

Para garantizar que cada experiencia práctica se ejecute en condiciones rigurosamente reproducibles e inmunes a divergencias de configuración local, el texto se apoya en un entorno unificado denominado `dsin` (Data Science in Nushell), distribuido como imagen pública de Docker en `ghcr.io/kurokirasama/dsin:latest`. Dicho contenedor constituye una estación analítica completa. Encapsula la versión más reciente de Nushell con un perfil afinado para el procesamiento de datos, su ecosistema de complementos (con soporte nativo para Polars, manipulación de formatos estructurados, consultas relacionales avanzadas, control de versiones y graficación en terminal), entornos robustos de Python y R configurados con sus bibliotecas científicas indispensables, algoritmos de alto rendimiento para modelado predictivo (como Vowpal Wabbit, Tapkee y SKLL), herramientas modernas de línea de comandos (entre las que destacan el editor interactivo `ttt` y el visor gráfico de alta fidelidad `timg`), así como la totalidad de los conjuntos de datos del curso, organizados meticulosamente por capítulos en el directorio `/data`.

El itinerario formativo de la obra se estructura con fidelidad en torno a las etapas del ciclo clásico OSEMN (Obtain, Scrub, Explore, Model, iNterpret), formulado originalmente por Hilary Mason y Chris Wiggins. El recorrido inicia en el capítulo 2 con la instalación del software y la preparación del entorno de trabajo reproducible mediante Docker y el contenedor `dsin`. A continuación, el capítulo 3 establece las bases operativas de la línea de comandos y el sistema tipado de estructuras de datos en Nushell, mientras que el capítulo 4 profundiza en la programación estructurada y la construcción de herramientas modulares reutilizables. El capítulo 5 aborda las estrategias de obtención e ingesta masiva desde la web, interfaces de programación de aplicaciones, archivos estructurados y bases de datos relacionales. Posteriormente, el capítulo 6 introduce el saneamiento riguroso de datos y la analítica columnar a gran escala con Polars. La exploración estadística multivariante y la visualización gráfica de alta densidad en consola se desarrollan en el capítulo 7, dando paso en el capítulo 8 a la orquestación de flujos de trabajo, procesamiento en lotes y paralelización distribuida. La obra concluye en el capítulo 9 con el modelado predictivo políglota, integrando reducción de dimensionalidad, agrupamiento no supervisado, regresión y clasificación supervisada.

2. Instalación de Software

En esta sección se entregaran las instrucciones para instalar el software necesario para este libro, tanto en Windows 10+ como en Linux, para distribución basada en Debian como Ubuntu.

2.1. Windows 10+

2.1.1. Instalación Ubuntu WSL

1. Habilitar WSL1, ejecutando en Powershell (con permisos de administrador):

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux
```

2. Habilitar VMP, ejecutando en Powershell:

```
Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform
```

3. Listar las distribuciones disponibles y seleccionar (recomendada Ubuntu 24.04):

```
wsl --list --online
```

4. Instalamos la distribución seleccionada:

```
wsl --install -d Ubuntu-24.04
```

4. Configuramos para que la distribución Linux en WSL use v2. Ejecutamos en Powershell:

```
wsl --set-version Ubuntu-24.04 2
```

Para hacerlo por defecto en todas las distribuciones (recomendado):

```
wsl --set-default-version 2
```

5. Actualizar kernel. Descarga desde WSL e instalar.

2.1.2. Instalación de Docker

1. Descargar instalador Docker para Windows desde: docker y ejecutar. Seleccionar “Usar WSL 2...”.
2. Habilitar “Iniciar Docker al iniciar sesión”, en la configuración de Docker.
3. Reiniciar.

2.1.3. Abrir distribución

Para entrar a la distribución instalada, ejecutar en `cmd` o `powershell`:

```
wsl
```

2.2. Linux, WSL2 y el Contenedor `dsin` (Data Science in Nushell)

En este libro trabajaremos sobre una imagen Docker integrada llamada **`dsin`** (*Data Science in Nushell*), publicada en `ghcr.io/kurokirasama/dsin:latest`. A diferencia de los entornos tradicionales basados en *shells* de texto plano, el contenedor `dsin` inicia directamente en **Nushell** como *shell* predeterminado en el directorio de trabajo `/data`, e incluye preconfigurados:

- **Nushell (`nu`)** y su suite completa de complementos registrados: **`nu_plugin_polars`** (motor de DataFrames Arrow/Polars), **`nu_plugin_formats`**, **`nu_plugin_gstat`**, **`nu_plugin_inc`**, **`nu_plugin_query`**, **`nu_plugin_file`**, **`nu_plugin_plot`** (gráficos en la terminal) y **`nu_plugin_port_extension`**.
- **Directorio `/data` integrado por defecto:** Contiene todas las carpetas de datos del libro organizadas exactamente según el número del capítulo en que se utilizan (`/data/ch03`, `/data/ch04`, `/data/ch05`, `/data/ch06`, `/data/ch07`, `/data/ch08` y `/data/ch09`), sin necesidad de descargas ni configuraciones manuales adicionales.
- **Editor de texto en terminal `ttt`** en `$env.PATH` con resaltado de sintaxis y atajos preconfigurados.
- **Visor gráfico de terminal `timg`** en `$env.PATH` para inspeccionar imágenes y gráficos estadísticos directamente en la consola.
- **Ecosistema políglota:** R (`Rscript`, `tidyverse`, `ggplot2`, `rpart`, `class`, `cluster`), Python 3 (`pandas`, `scikit-learn`, `scipy`, `matplotlib`, `seaborn`, `skll`), **GNU Parallel (`parallel`)**, **Tapkee (`tapkee`)** y **Vowpal Wabbit (`vw`)**.
- **Módulo de herramientas del libro (`dsin_tools.nu`):** Comandos personalizados (`to-vw`, `vw-regress`, `tapkee-reduce`, `kmeans-cluster`) cargados automáticamente al iniciar.

2.2.1. Instalación de Docker en Linux / WSL (basado en Debian/Ubuntu)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg lsb-release git
sudo usermod -aG docker $USER
```

2.2.2. Instalación de Docker en Linux (basado en Arch / CachyOS / Manjaro)

```
sudo pacman -Syu --needed docker docker-buildx docker-compose git
sudo systemctl enable --now docker.service
sudo usermod -aG docker $USER
```

2.2.3. Descarga y Ejecución del Contenedor `dsin`

Para descargar la imagen oficial del libro desde GitHub Container Registry:

```
docker pull ghcr.io/kurokirasama/dsin:latest
```

Para iniciar una sesión interactiva en el contenedor con el entorno listo para usar, ejecute:

```
docker run --rm -it ghcr.io/kurokirasama/dsin:latest
```

2.2.4. Crear alias para llamado simple (opcional)

Si deseas iniciar el contenedor escribiendo únicamente `dsin` en tu terminal anfitriona, puedes definir un alias en tu archivo de configuración del shell anfitrión (`~/.bashrc` o `~/.config/nushell/config.nu`):

```
alias dsin='docker run --rm -it ghcr.io/kurokirasama/dsin:latest'
```

Una vez dentro del contenedor `dsin`, verificamos que nos encontramos en `/home/dsin`, inspeccionamos las carpetas de los capítulos en `/data` y comprobamos que todas las herramientas y complementos están disponibles:

```
pwd
ls /data
version
which nu ttt timg parallel vw tapkee run_experiment Rscript python3
plugin list
```

3. Fundamentos y Estructuras de Datos en Nushell

3.1. Ciencia de Datos

De acuerdo a la definición de Hilary Mason y Chris H. Wiggins (2010), la *ciencia de datos* se define de acuerdo a 4 pasos:

1. Obtención de datos.
2. Depuración de datos.
3. Exploración de datos.
4. Modelación de datos.
5. Interpretación de datos.

En este libro veremos los primeros 4 pasos.

Aunque estos 5 pasos se discuten en forma lineal e incremental, en la practica es común el moverse de uno a otro o incluso realizarlos en paralelo (Figura 1). Es decir, la ciencia de datos es un proceso iterativo no lineal.

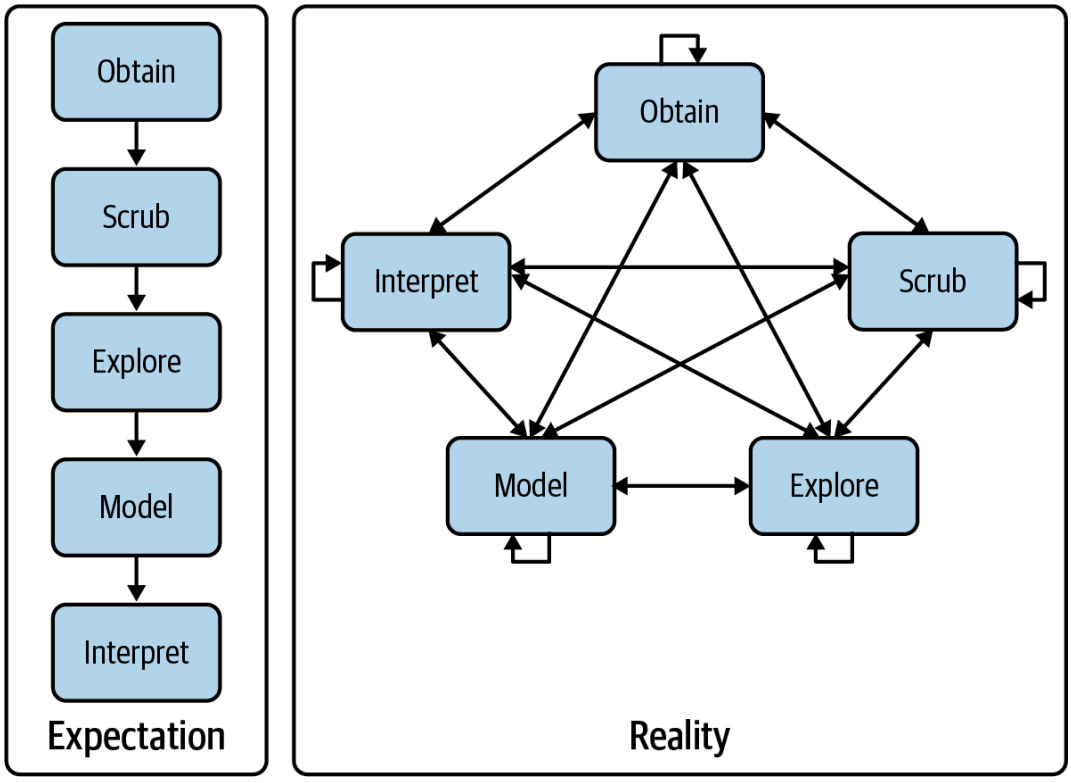


Figura 1: Proceso de la ciencia de datos.

3.1.1. Obtención de datos

Sin datos, no hay ciencia de datos, por lo que el primer paso es obtenerlos. Para ello, se puede necesitar realizar uno o mas de las siguientes tareas:

- Descargar datos desde otra ubicación, por ejemplo, pagina web o servidor.
- Obtener datos desde una base de datos o a través de una API, por ejemplo, MySQL o Twitter.
- Extraer datos de un archivo, por ejemplo, html, excel, csv.
- Generar datos, ya sea mediante sensores o haciendo encuestas.

Los formatos mas usuales para almacenar datos son, texto plano (txt), csv, json, html, xml, excel.

3.1.2. Depuración de datos

No es poco usual que los datos obtenidos tengan valores faltantes, inconsistencias, errores, caracteres erróneos o columnas no interesantes. En tales casos, los datos deben depurarse o limpiarse antes de poder hacer cualquier cosa con ellos. La operaciones mas comunes incluyen:

- Filtrar lineas.
- Extraer algunas columnas.
- Reemplazar valores.
- Extraer palabras.
- Lidiar con datos faltantes y duplicados.
- Convertir de un formato a otro.

Usualmente, un 80 % de cualquier trabajo con datos corresponde a la obtención y depuración. El otro 20 % es la exploración, modelación y visualización.

3.1.3. Exploración de datos

Una vez hecha la limpieza de datos, se procede a explorarlos. Es en esta etapa donde realmente se comprende de mejor forma los datos obtenidos. A través de la terminal se puede:

- Observar los datos.
- Derivar estadísticas.
- Crear visualizaciones.

3.1.4. Modelación de datos

Si se quiere explicar los datos o predecir lo que pasara, probablemente se necesita crear un modelo estadístico de los datos. Algunas técnicas para crear modelos incluyen clustering, clasificación, regresión y reducción de dimensionalidad.

La terminal no es apta para programar un nuevo modelo desde 0, pero es muy útil el ser capaz de crear un modelo desde ella.

3.1.5. Interpretación de datos

El ultimo y probablemente mas importante paso es interpretarlos. Este paso involucra:

- Obtener conclusiones.
- Evaluación de resultados.
- Comunicación de resultados.

Esta etapa no involucra la terminal, y queda en manos del investigador.

3.2. Historia de la terminal

La *terminal* (*terminal emulator*) en un programa o interfaz que nos permite acceder a la *linea de comandos* (*command line*) a través de un tipo particular de sintaxis llamado *shell*.

Originalmente, una terminal era una pieza de *hardware* usado para ingresar y mostrar datos a/desde un computador.

Los primeros computadores, eran maquinas que ocupaban mucho espacio (ver Figura 2) y las primeras terminales eran tele-impresoras electrónicas (teletypewriter o tty) como se muestra en la (ver Figura 3).



Figura 2: Primeros computadores.

Conforme fue avanzando la tecnología, estas terminales se reemplazaron por terminales de video (ver Figura 4).

Notar que lo mostrado en la Figura 4 no es un computador, es solo la interfaz para poder conectarse al computador. En la Figura 5 se muestra conectada a uno.

En la actualidad no son necesarias estas terminales de *hardware*, por lo que se usan emuladores de terminales para realizar la comunicación directa con el sistema operativo sin necesidad de una interfaz gráfica (gui), esto ultimo siendo lo mas conocido (ventanas, botones, etc).



Figura 3: Primeras terminales.

3.2.1. Shell

La *shell* es la sintaxis particular que se utiliza para ejecutar comandos y programas en la terminal. Hay varios tipos de *shell*, como *bash*, *csh*, *zsh*, *nushell*, entre muchos otros, siendo la mas usual *bash* como la por defecto en la mayoría de distribuciones Linux.

En este libro utilizaremos la shell Nushell, en la cual a cada comando lo precede un indicador llamado prompt (por ejemplo, `/data>`), que señala que la terminal está lista para recibir instrucciones.

3.2.2. Ventajas del uso de la terminal

El uso de la terminal para ciencia de datos tienes varias ventajas, como por ejemplo:

- Es ágil: permite explorar datos de forma interactiva, obteniendo resultados de forma casi inmediata.
- Es integradora: se integra bien con otras tecnologías y software.
- Es escalable: se pueden automatizar muchas tareas mediante *scripts*, lo que da como resultado herramientas mas complejas.
- Es extensible: permite el uso de diferentes lenguajes de programación.
- Es ubicua: se encuentra en todos los sistemas operativos.

3.3. La Terminal

Antes de profundizar en las estructuras de datos de Nushell, revisemos los conceptos esenciales del entorno de terminal y cómo operaban los *shells* tradicionales que precedieron a Nushell.



Figura 4: Primeras terminales de video.

3.3.1. El Ambiente

En la terminal interactuamos mediante un ciclo continuo conocido como **REPL** (*Read-Eval-Print Loop*: Leer, Evaluar, Imprimir y Repetir):

1. La terminal muestra un indicador o *prompt* (por ejemplo, `/data/ch03>`) indicando el directorio actual y que está lista para recibir instrucciones.
2. El usuario escribe una línea de comando y presiona **`Enter`**.
3. El *shell* analiza (*parsea*) y evalúa la instrucción, ejecutando los programas o transformaciones correspondientes.
4. El resultado se imprime en la pantalla y el *prompt* vuelve a aparecer.

3.3.2. Ejecutar un Comando y Guardar Resultados en Archivos

Para comenzar a trabajar con los ejemplos de este capítulo, ingresamos a su directorio correspondiente:

```
cd /data/ch03
```

A partir de este punto, todos los comandos se ejecutan dentro de `/data/ch03`. Inspeccionemos los archivos disponibles:

```
ls
```

Si deseamos inspeccionar las primeras 5 líneas del archivo `movies.txt`, abrimos el archivo en el directorio actual (`open movies.txt`), dividimos sus líneas con `lines` y tomamos las primeras 5 con `first 5`:



Figura 5: Primeras terminales conectadas a un computador.

```
open movies.txt | lines | first 5
```

En ciencia de datos reproducible, además de visualizar el resultado en pantalla, frecuentemente necesitamos persistir el resultado procesado en un nuevo archivo utilizando el comando `save -f`:

```
open movies.txt | lines | first 5 | to text | save -f movies_top5.txt
open movies_top5.txt
```

3.3.3. Tipos de Herramientas en la Línea de Comandos

Cuando escribimos el nombre de un comando en la terminal, este puede corresponder a uno de cinco tipos de herramientas:

1. **Comando interno (*built-in*) de Nushell:** Ejecutado directamente por el motor de Nushell en memoria y que opera nativamente sobre datos estructurados (por ejemplo, `ls`, `open`, `where`, `select`, `math avg`).
2. **Comando personalizado (*custom command* / función):** Definido por el usuario o en un módulo mediante `def` o `export def` (por ejemplo, `to-vw` o `tapkee-reduce`).
3. **Alias:** Un nombre abreviado que expande a otro comando con opciones predefinidas (`alias ll = ls -l`).
4. **Script ejecutable interpretado:** Un archivo de texto con permisos de ejecución y una línea inicial *shebang* (`#!/usr/bin/env nu`, `#!/usr/bin/env python3`, `#!/usr/bin/env Rscript`).
5. **Ejecutable binario compilado:** Un programa compilado a código máquina ubicado en el `$env.PATH` del sistema (como `ttt`, `timg`, `vw`, `tapkee` o `git`).

Podemos consultar de qué tipo es cualquier comando utilizando `which`:

```
which ls open ttt timg python3 Rscript
```

3.3.4. Otros Shells como Bash y Zsh: Qué Hacen, su Filosofía y su Uso

Para comprender cabalmente por qué **Nushell** representa un cambio de paradigma en la ciencia de datos, es fundamental conocer primero cómo funcionan los *shells* tradicionales de Unix/Linux, tales como **Bourne Shell (`sh`)**, **Bourne-Again Shell (`bash`)** y **Z Shell (`zsh`)**.

3.3.4.1. ¿Qué es y qué hace un Shell Tradicional? Un *shell* tradicional cumple dos roles simultáneos en un sistema operativo tipo Unix:

- **Intérprete interactivo de órdenes:** Actúa como intermediario entre el usuario y el núcleo (*kernel*) del sistema operativo. Cuando escribes un comando en `bash`, el *shell* busca un archivo binario en las rutas de la variable `PATH`, solicita al *kernel* crear un nuevo proceso hijo mediante las llamadas al sistema `fork()` y `execve()`, y espera a que el proceso termine.
- **Lenguaje de *scripting* y automatización:** Permite encadenar programas, definir variables de entorno, evaluar condicionales (`if/then/fi`) y ejecutar ciclos (`for`, `while`) para automatizar tareas repetitivas de administración de sistemas.

3.3.4.2. La Filosofía Unix y los Flujos de Texto Plano Diseñada en los Laboratorios Bell en la década de 1970 por Ken Thompson, Dennis Ritchie y Douglas McIlroy, la **filosofía Unix** se resume en tres principios fundamentales formulados por McIlroy (creador de las tuberías o *pipes* de Unix en 1973):

1. *Escribe programas que hagan una sola cosa y la hagan bien.*
2. *Escribe programas que trabajen juntos.*
3. *Escribe programas que manejen flujos de texto, porque esa es una interfaz universal.*

En `bash` y los demás *shells* compatibles con el estándar **POSIX**, cada proceso se comunica con el exterior a través de tres canales o flujos de bytes sin estructura:

- **Entrada estándar (`stdin`, descriptor 0):** Flujo de texto que el programa lee como entrada.
- **Salida estándar (`stdout`, descriptor 1):** Flujo de texto donde el programa escribe sus resultados normales.
- **Error estándar (`stderr`, descriptor 2):** Flujo de texto donde el programa emite mensajes de diagnóstico o error.

3.3.4.3. Uso y Sintaxis Habitual en Bash En un *shell* tradicional como `bash`, todas las herramientas (`grep`, `sed`, `awk`, `cut`, `tr`, `sort`, `uniq`, `wc`, `head`, `tail`) reciben **texto plano línea por línea** y devuelven **texto plano línea por línea**. Por ejemplo, en `bash`:

```
# Filtrar líneas que contienen "CHAPTER" y guardarlas con redirección '>'
grep "CHAPTER" alice.txt > chapters.txt
```

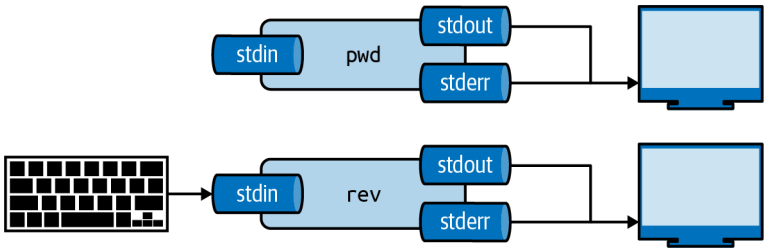


Figura 6: Redirección de salida estándar en la terminal

```
# Cortar la primera y tercera columna separadas por coma, ordenar y contar líneas
cut -d ',' -f 1,3 datos.csv | sort | uniq -c | head -n 5
```

En `bash`, el símbolo `|` conecta el flujo de bytes crudos del `stdout` del comando izquierdo con el `stdin` del comando derecho, mientras que `>` y `>>` redirigen ese flujo de bytes para sobrescribir o anexar un archivo en disco.

3.3.4.4. ¿Por qué el Modelo de Texto Plano de Bash es Frágil para la Ciencia de Datos? Aunque el flujo de texto plano fue revolucionario hace cincuenta años, cuando hacemos **ciencia de datos moderna** trabajar únicamente con flujos de caracteres sin estructura introduce cuatro problemas críticos:

1. **Ausencia de tipos de datos:** Para `bash`, el número `42.5`, la fecha `2026-10-20`, el booleano `true` y el texto `"hola"` son exactamente lo mismo: secuencias de bytes ASCII. Para sumar una columna o comparar números hay que invocar lenguajes externos como `awk` o `bc`.
2. **Fragilidad ante delimitadores y espacios:** Herramientas como `cut -d ','` o `awk` dividen el texto buscando comas o espacios literales. Si un archivo CSV contiene un campo entre comillas con una coma interna (por ejemplo, `Concepción, Chile"`), `cut` y `awk` parten erróneamente la columna por la mitad y corrompen toda la tabla.
3. **Pérdida de encabezados:** Si aplicas `sort` o `grep` a un archivo CSV en `bash`, la primera línea (que contiene los nombres de las columnas) es tratada como una fila de datos cualquiera, mezclándose con los registros o desapareciendo tras el filtro.
4. **Incompatibilidad con datos jerárquicos (JSON, XML, Excel, SQLite, Parquet):** El texto línea por línea no puede representar directamente objetos anidados ni tablas tipadas sin recurrir a múltiples conversores externos (`jq`, `xmllint`, `sqlite3`) con sintaxis inconexas entre sí.

Nushell conserva lo mejor de la filosofía Unix (la composición modular mediante pipes `|`), pero reemplaza los flujos de texto crudo por **flujos de datos estructurados y tipados (tablas, registros y listas)**. Comprender esta diferencia hace que la siguiente sección resulte completamente natural.

3.4. Como entender nu

3.4.1. Nushell no es bash

Nushell es tanto un lenguaje de programación como un shell. Debido a esto, tiene su propia forma de trabajar con archivos, directorios, sitios web y más. Se ha modelado para que funcione en estrecha colaboración con lo que se puede estar familiarizado con otros shells. Las *pipelines* funcionan uniendo dos comandos:

```
ls | length
```

Nushell, por ejemplo, también tiene soporte para otras capacidades comunes como obtener el código de salida de comandos ejecutados previamente.

Si bien tiene estas comodidades, Nushell no es bash. La forma de trabajar de bash, y el estilo POSIX en general, no es algo que Nushell soporte. Por ejemplo, en bash, se podrías usar:

```
echo "hello" > output.txt
```

En Nushell, se usa `>` como el operador "mayor que". Esto encaja mejor con el aspecto del lenguaje de Nushell. En cambio, se canalizas a un comando que tiene la tarea de guardar contenido:

```
"hello" | save -f output.txt
```

La forma en que Nushell ve los datos es que los datos fluyen a través de la *pipeline* hasta que llegan al usuario o son manejados por un comando final. Se puede simplemente escribir datos, desde cadenas hasta listas y registros de estilo JSON, y seguirlos con `|` para enviarlos a través de la *pipeline*. Nushell usa comandos para hacer el trabajo y producir más datos. Aprender estos comandos y cuándo usarlos ayuda a componer muchos tipos de *pipelines*.

3.4.2. Nushell como un lenguaje compilado

Una parte importante del diseño de Nushell y específicamente donde difiere de muchos lenguajes dinámicos es que Nushell convierte el código fuente en algo para ejecutar, y luego ejecuta el resultado. No tiene una función `eval` que permita continuar introduciendo nuevo código fuente durante el tiempo de ejecución. Esto significa que las tareas como incluir archivos para que formen parte de un proyecto deben ser rutas conocidas, al igual que las inclusiones en lenguajes compilados como C++ o Rust.

Por ejemplo, lo siguiente no tiene sentido en Nushell, y fallará al ejecutarse si se ejecuta como un script:

```
"def abc [] { 1 + 2 }" | save -f output.nu
source "output.nu"
abc
```

El comando `source` aumentará el código fuente que se compila, pero el `save` de la línea anterior no habrá tenido la oportunidad de ejecutarse. Nushell ejecuta todo el bloque como si fuera un solo

archivo, en lugar de ejecutar una línea a la vez. En el ejemplo, dado que el archivo `output.nu` no se crea hasta después del paso de 'compilación', el comando `source` no puede leer las definiciones del mismo durante el tiempo de análisis.

Otro problema común es intentar crear dinámicamente el nombre de archivo desde el que obtener el código fuente:

```
source $"($my_path)/common.nu"
```

Esto no funciona si `my_path` es una variable de tiempo de ejecución regular declarada con `let`. Esto requeriría que el evaluador se ejecutara y evaluara la cadena, pero desafortunadamente Nushell necesita esta información en tiempo de compilación.

Sin embargo, si `my_path` es una constante, entonces esto funcionaría, ya que la cadena se puede evaluar en tiempo de compilación:

```
const my_path = ([$nu.home-dir nushell] | path join)
source $"($my_path)/common.nu" # sources /home/user/nushell/common.nu
```

Nushell está diseñado para usar un solo paso de compilación para todo el código fuente, y esto es independiente de la evaluación. Esto permitirá un fuerte soporte IDE, mensajes de error precisos, un lenguaje más fácil de usar para herramientas de terceros y, en el futuro, incluso una salida más sofisticada, como la posibilidad de compilar Nushell directamente a un archivo binario.

3.4.3. Las variables son inmutables por defecto

Otra sorpresa común para las personas que vienen de otros lenguajes es que las variables de Nushell son inmutables por defecto. Al llegar a Nushell, se querrá dedicar algún tiempo a familiarizarte con el trabajo en un estilo más funcional, ya que esto tiende a ayudar a escribir código que funciona mejor con variables inmutables.

Si se está acostumbrado a usar variables mutables para diferentes tareas, llevará algún tiempo aprender a hacer cada tarea en un estilo más funcional. Nushell tiene un conjunto de capacidades integradas para ayudar con muchos de estos patrones, y aprenderlos ayudará a escribir código en un estilo más Nushell. El beneficio adicional de acelerar scripts ejecutando partes del código en paralelo es una buena ventaja.

3.5. Elementos básicos de nushell

En esta sección se revisarán los primeros elementos de nushell.

Nushell funciona bajo la base de que todo son datos estructurados, y por lo tanto se pueden realizar consultas, similares a SQL, directamente en la salida de cada comando. Comenzamos en el siguiente directorio:

```
ls
```

Observar como `ls` retorna en lugar de que vuelva un bloque de texto plano, una tabla estructurada. La tabla hace más que mostrar el directorio de una manera diferente. Al igual que las tablas en una hoja de cálculo, esta tabla permite trabajar con los datos de forma más interactiva.

Para ordenar la tabla por tamaño, tomamos la salida de `ls` y la introduciremos en un comando que puede ordenar tablas en función del contenido de una columna.

```
ls | sort-by size | reverse
ls | sort-by size -r
ls | sort-by modified
```

Se puede observar que para que esto funcione no se utilizan argumentos de línea de comandos de `ls`. En su lugar, se utiliza el comando `sort-by` que Nu proporciona para hacer la clasificación de la salida del comando `ls`. Para ver los archivos más grandes en la parte superior, también se utiliza `reverse` o la opcion `-r` de `sort-by`.

Nu proporciona muchos comandos que pueden funcionar en tablas. Por ejemplo, se puede utilizar `where` para filtrar el contenido de la tabla `ls` para que sólo muestre los archivos de más de 5 kilobytes

```
ls | where size > 100kb
```

Al igual que en la filosofía Unix, ser capaz de hacer que los comandos se comuniquen entre sí entrega formas de mezclar y combinar en muchas combinaciones diferentes. Veamos un comando diferente:

```
ps
```

Este comando entrega una lista de todos los procesos actuales que el sistema está ejecutando, cuál es su estado y cuál es su nombre. También podemos ver la carga de CPU de los procesos.

Si se quisiera mostrar los procesos que están utilizando activamente la CPU, se puede proceder como sigue:

```
ps | where cpu > 0
```

Nu no solo permite tablas estructuradas de forma simple, si no que también permite anidaciones arbitrarias. Por ejemplo, consideremos el comando `help commands`. Este comando proporciona información para todos los comandos de Nushell como una tabla.

```
help commands
```

La salida es bastante extensa, por lo que se obtendra la fila para el comando `each`.

```
help commands | where name == each
help commands | where name == each | first
```

Al ecuperar una sola fila de una tabla se obtiene un *registro* (*record*), que es un conjunto de

pares clave-valor. Observar como la columnas "signatures" contiene un registro en lugar de valores simples. Para ver sólo una de estas "claves" o columnas, se puede utilizar el comando `get`:

```
help commands | where name == each | first | get description
help commands | where name == each | first | get params
```

El comando `get` nos permite saltar al contenido de los datos estructurados (una tabla, registro o lista). Incluso se puede pasar columnas anidadas para acceder a los datos a cualquier profundidad.

```
help commands | where name == each | first | get category
help commands | where name == each | first | get command_type
help commands | where name == each | first | get description
help commands | where name == each | first
```

Estas columnas anidadas se llaman *rutas de celda* (*cellpaths*). Nu tomará la ruta de la celda e irá al bit de datos correspondiente en una tabla, registro o lista. Las rutas de celda también admiten números de fila, por lo que los comandos anterior también se puede escribir como:

```
help commands | where name == each | first | get params
help commands | where name == each | first | get params.0
help commands | where name == each | first | get params.3.description
```

Si se quiere ver la estructura anidada, se puede utilizar `table -e`:

```
help commands | where name == each | first | table -e
```

Notar que se pueden acceder a todas las herramientas estandar de la terminal utilizando el símbolo ^ si existe una versión nushell. Si no existe, no es necesario utilizarlo:

```
^ls
open tips.csv | first 5
uname
```

Todo comando nushell que retorna datos lo hará en una tabla, por ejemplo:

```
#muestra los procesos en ejecucion
ps

#ordenados por mayor utilizacion de cpu
ps | where cpu > 0 | sort-by cpu -r
```

3.5.1. Obtener Ayuda

Se puede leer texto de ayuda para cualquiera de los comandos incorporados de Nu usando el comando `help` o pasando la opción `--help` a un comando. También se puede buscar un tema utilizando `help -f <topic>`.

```
help explore
help http get
help -f open
```

3.6. Moviéndose por el Sistema

Una característica definitoria de un shell es la capacidad de navegar e interactuar con el sistema de archivos. Nushell es, por supuesto, ninguna excepción. Aquí hay algunos comandos comunes que puede usar al interactuar con el sistema de archivos:

3.6.1. Viendo el Contenido de un Directorio

```
ls
```

Como se ve indicó anteriormente, el comando `ls` devuelve el contenido de un directorio. El `ls` de Nushell devolverá el contenido como una tabla.

El comando `ls` también toma un argumento opcional para cambiar lo que se quiere ver. Por ejemplo, se puede listar los archivos que terminan en ".csv"

```
ls *.csv
```

3.6.2. Patrones Glob (comodines)

Al asterisco (*) en el argumento opcional anterior `*.csv` a veces se llama comodín o glob. Permite hacer coincidir cualquier cosa. Se puede leer este glob `*.csv` como *"coincidir con cualquier nombre de archivo, siempre que termine con '.csv'."*

El glob más general es `*`, que coincidirá con todas las rutas. Con mayor frecuencia, se verá este patrón utilizado como parte de otro patrón, por ejemplo, `*.bak` y `temp*`.

Nushell también admite un `*` doble que atravesará rutas que están anidadas dentro de otros directorios. Por ejemplo, `ls **/*` enumerará todas las rutas no ocultas anidadas en el directorio actual, recursivamente.

```
ls **/*.csv
```

Aquí, buscanmos cualquier archivo que termine con ".csv". Los dobles asteriscos especifican adicionalmente *"en cualquier directorio a partir de aquí."*

La sintaxis globbing de Nushell no solo admite `*`, sino también la coincidencia de caracteres individuales con `?` y grupos de caracteres con `[...]`.

Escapar los patrones `*`, `?` y `[]` funciona delimitándolos entre comillas simples, comillas dobles o "cadena raw". Por ejemplo, para mostrar el contenido de un directorio llamado `[slug]`, se utiliza use `ls "[slug]"` o `ls '[slug]'`.

Sin embargo, las cadenas delimitados con *comillas inversas* (o backtips) no escapan los glob. Por ejemplo, compare los siguientes escenarios:

1. Sin comillas: Patrón glob

   Una *cadena de palabra simple* sin comillas con caracteres glob se interpreta como un patrón glob, por lo que lo siguiente eliminará todos los archivos en el directorio actual que contengan `myfile` como cualquier parte del nombre del archivo:

   ```
   rm *myfile*
   ```

2. Entre comillas: Literal de cadena con asteriscos

   Cuando se cita con comillas simples o dobles, o se usa una *cadena raw*, se pasa al comando una *cadena* con los asteriscos literales escapados (u otros caracteres glob). El resultado no es un glob. El siguiente comando solo eliminará un archivo llamado literalmente `*myfile*` (incluidos los asteriscos). Otros archivos con `myfile` en el nombre no se ven afectados:

   ```
   rm "*myfile*"
   ```

3. Entre comillas invertidas: Patrón glob

   Los asteriscos (y otros patrones glob) dentro de una *cadena entre comillas invertidas* se interpretan como un patrón glob. Observe que este es el mismo comportamiento que el del ejemplo de cadena de palabra simple en el n.° 1 anterior.

   Lo siguiente, como en el primer ejemplo, elimina todos los archivos en el directorio actual que contienen `myfile` como parte del nombre del archivo

   ```
   rm `*myfile*`
   ```

   Nushell también incluye un comando `glob` dedicado con soporte para escenarios globbing más complejos.

3.6.2.1. Conversión de Cadenas a Globs Las técnicas de comillas mencionadas anteriormente son útiles al construir globos literales, pero es posible que se deban construir globs mediante programación. Hay varias técnicas disponibles para este propósito:

1. `into glob`

   El comando `into glob` se puede usar para convertir una cadena (y otros tipos) en un glob. Por ejemplo:

   ```
   # Encuentra archivos cuyo nombre incluya el mes actual en el formato YYYY-mm
   let current_month = (date now | format date '%Y-%m')
   let glob_pattern = ($"*($current_month)*" | into glob)
   ls $glob_pattern
   ```

2. El operador de propagación (*spread operator*) combinado con el comando `glob`:

El comando `glob` (nota: no es lo mismo que `into glob`) produce una `list` de nombres de archivo que coinciden con el patrón de globo. Esta lista se puede expandir y pasar a los comandos del sistema de archivos utilizando el operador de propagación `...`:

```
# Encuentra archivos cuyo nombre incluya el mes actual en el formato YYYY-mm
let current_month = (date now | format date '%Y-%m')
ls ...(glob $"*($current_month)*")
```

3. Forzar el tipo `glob` mediante anotación:

```
# Encuentra archivos cuyo nombre incluya el mes actual en el formato YYYY-mm
let current_month = (date now | format date '%Y-%m')
let glob_pattern: glob = ($"*($current_month)*")
ls $glob_pattern
```

3.7. Trabajando con strings

Como con la mayoría de los lenguajes, los strings son una colección de 0 o más caracteres que representan texto. Esto puede incluir nombres de archivos, rutas de archivos, nombres de columnas, y mucho más. Las strings son tan comunes que Nushell ofrece múltiples formatos de string para que coincidan con cada caso de uso.

3.7.1. Formatos de String

Formato de string	Ejemplo	Escapes	Notas
String con comillas simples	`'[^\n]+'`	Ninguno	No puede contener comillas simples dentro del string
String con comillas dobles	`"The\nEnd"`	Escapes de barra invertida estilo C	Todas las barras invertidas literales deben tener escape
Strings raw	`r#'Raw string'#`	Ninguno	Puede incluir comillas simples
String de palabra simple	`ozymandias`	Ninguno	Solo puede contener caracteres de "palabra"; No se puede usar en la posición de comando
String con comillas invertidas	`` `[^\n]+` ``	Ninguno	String simple que puede incluir espacios en blanco. No puede contener comillas invertidas
Interpolación con comillas simples	`$'Captain ($name)'`	Ninguno	No puede contener ningún `'` o `()` no coincidentes
Interpolación con comillas dobles	`$"Captain ($name)"`	Escapes de barra invertida estilo C	Todas las barras invertidas literales y `()` deben tener escape

3.7.2. Strings con Comillas Simples

El string más simple en Nushell es el string con comillas simples. Este string usa el carácter `'` para rodear algún texto. Por ejemplo:

```
'hello world'

'The
end'
```

Los strings con comillas simples no le hacen nada al texto que se les da, lo que las hace ideales para contener una amplia gama de datos de texto.

3.7.3. Strings con Comillas Dobles

Para strings más complejas, Nushell también ofrece strings con comillas dobles. Estas strings usan el carácter `"` para rodear el texto. También admiten la capacidad de escapar caracteres dentro del texto usando el carácter `\`.

Por ejemplo, se podría escribir un texto seguido de una nueva línea y luego otro texto, usando caracteres de escape y un string con comillas dobles:

```
"hello\nworld"
```

Los caracteres de escape permiten agregar rápidamente un carácter que de otro modo sería difícil de escribir.

Nushell actualmente admite los siguientes caracteres de escape:

- `\"` - carácter de comillas dobles
- `\'` - carácter de comillas simples
- `\\` - barra invertida
- `\/` - barra diagonal
- `\b` - retroceso
- `\f` - avance de formulario
- `\r` - retorno de carro
- `\n` - nueva línea (avance de línea)
- `\t` - tabulación
- `\u{X...}` - un solo carácter unicode, donde X… son 1-6 dígitos hexadecimales (0-9, A-F)

3.7.4. Strings Raw

Los strings raw se comportan igual que las strings con comillas simples, excepto que las strings raw también pueden contener comillas simples. Esto es posible porque las strings raw están encerradas por un `r#'` inicial y un `'#` final.

```
r#'Raw strings can contain 'quoted' text.'#
```

Se pueden agregar símbolos `#` adicionales al principio y al final del string raw para encerrar uno menos que la misma cantidad de símbolos `#` junto a un símbolo `'` en el string. Esto se puede usar para anidar strings raw:

```
r###'r##'This is an example of a raw string.'##'###
```

3.7.5. Strings de Palabra Simple

Al igual que otros lenguajes de shell (pero a diferencia de la mayoría de los otros lenguajes de programación), las strings que constan de una sola 'palabra' también se pueden escribir sin comillas:

```
print hello
print "hello"

[hello] | describe
```

Pero se debe tener cuidado: si usa una palabra simple directamente en la línea de comandos (es decir, no dentro de una estructura de datos o como un parámetro de comando) o dentro de paréntesis redondos `( )`, se interpretará como un comando externo:

```
hello
Error: nu::shell::external_command

  × External command failed
   ╭─[entry #5:1:1]
 1 │ hello
   · ──┬──
   ·   └── executable was not found
   ╰────
  help: program not found
```

Además, muchas palabras simples tienen un significado especial en nu, por lo que no se interpretarán como una string:

```
true | describe

[true] | describe

[trueX] | describe

"trueX" | describe
```

Entonces, si bien los strings simples son útiles para el uso informal de la línea de comandos, al programar más formalmente en nu, se recomienda usar comillas.

3.7.6. Strings con Comillas Invertidas

Las strings de palabras simples, por su naturaleza, no pueden incluir espacios ni comillas. Como alternativa, Nushell también incluye strings con comillas invertidas usando el carácter ‘. En la mayoría de los casos, estos deberían funcionar igual que una string de palabra simple.

Por ejemplo, al igual que con una palabra simple, una string entre comillas invertidas en la primera posición de una expresión se interpretará como un *comando* o *ruta*. Por ejemplo:

```
# Ejecutar el binario ls externo o cambiar de directorio interactivamente
^ls
```

Las strings con comillas invertidas pueden ser útiles para combinar globs con archivos o directorios que incluyen espacios:

```
ls `./my dir/*`
```

3.7.7. Strings como comandos externos

Puede colocar el signo ^ delante de cualquier string (incluida una variable) para que Nushell ejecute la string como si fuera un comando externo:

```
^ls

^'ls'

let foo = 'ls'
^$foo
```

También se puede usar el comando `run-external` para este propósito, que proporciona indicadores y opciones adicionales.

3.7.8. Añadir y anteponer a strings

Hay varias formas de anteponer o añadir strings. Si se desea agregar algo al principio de cada string, se puede iterar:

```
['foo', 'bar'] | each {|s| '~/' ++ $s} # ~/foo, ~/bar
['foo', 'bar'] | each {|s| '~/' + $s} # ~/foo, ~/bar
```

También puede usar una expresión regular para reemplazar el principio o el final de una string:

```
['foo', 'bar'] | str replace -r '^' '~/' # ~/foo, ~/bar
['foo', 'bar'] | str replace -r '$' '~/' # foo~/, bar~/
```

Si se desea obtener un string a partir de una lista:

```
"hello" | append "world!" | str join " "

ls | get name | str join "/"
```

También se puede usar reduce:

```
1..10 | reduce -f "" {|elt, acc| $acc + ($elt | into string) + " + "}
```

Aunque en el caso de los strings, especialmente si no se tiene que operar en las strings, suele ser más fácil y correcto (observe el + adicional al final del ejemplo anterior) usar `str join`.

Finalmente, también se puede usar la interpolación de strings.

3.7.9. Interpolación de strings

Los casos de uso de strings más complejos también necesitan una nueva forma de string: interpolación de strings. Esta es una forma de construir texto a partir de texto sin formato y el resultado de ejecutar expresiones. La interpolación de strings combina los resultados, dándo una nueva string.

La interpolación de strings usa `$" "` y `$' '` como formas de encapsular texto interpolado.

Por ejemplo, si se tiene una variable llamada `$name` y queremos saludar al nombre de la persona contenida en esta variable:

```
let name = "Alice"

$"greetings, ($name)"

$"greetings, (ls | get name.2)"
```

Al encapsular expresiones en `()`, podemos ejecutarlas hasta su finalización y usar los resultados para ayudar a construir el string.

La interpolación de strings tiene una forma con comillas simples, `$' '`, y una con comillas dobles, `$" "`. Éstas corresponden a las strings con comillas simples y comillas dobles: la interpolación de strings con comillas simples no admite caracteres de escape, mientras que la interpolación de strings con comillas dobles sí.

A partir de la versión 0.61, las strings interpoladas admiten el escape de paréntesis, de modo que los caracteres `(` y `)` se pueden usar en una string sin que Nushell intente evaluar lo que aparece entre ellos:

```
$"2 + 2 is (2 + 2) \(adivinaste!)"
```

Los strings interpolados se pueden evaluar en tiempo de análisis, pero si incluyen valores cuyo formato depende de su configuración y su `config.nu` aún no se ha cargado, utilizarán la configuración predeterminada. Entonces, si se tiene algo como esto en su `config.nu`, `x` será `"2.0 KB"` incluso si

su configuración dice que use `MB` para todos los tamaños de archivo (las fechas y horas utilizarán de manera similar la configuración predeterminada).

```
const x = $"(2kb)"
```

3.7.10. Dividir Strings

El comando `split row` crea una lista a partir de una string basándose en un delimitador.

```
"red,green,blue" | split row ","
```

El comando `split column` creará una tabla a partir de una string basándose en un delimitador. Esto aplica nombres de columna genéricos a la tabla.

```
"red,green,blue" | split column ","
```

Finalmente, el comando `split chars` dividirá una string en una lista de caracteres.

```
'aeiou' | split chars
```

3.7.11. El comando `str`

Muchas funciones de string son subcomandos del comando `str`. Puede obtener una lista completa usando `help str`.

Por ejemplo, puede buscar si una string contiene una subcadena (*substring*) en particular usando `str contains`:

```
"hello world" | str contains "o wo"
```

También se puede utilizar el operador `=~`.

3.7.11.1. Recortar Strings Se puede recortar los lados de una string con el comando `str trim`. De forma predeterminada, el comando `str trim` recorta los espacios en blanco de ambos lados de la string. Por ejemplo:

```
'       My   string    ' | str trim
```

Se puede especificar en qué lado se produce el recorte con las opciones `--right` y `--left`. (`-r` y `-l` son las opciones abreviadas respectivamente)

Para recortar un carácter específico, se utiliza `--char <Carácter>` o `-c <Carácter>` para especificar el carácter a recortar.

Ejemplo:

```
'=== Nu shell ===' | str trim -r -c '='
```

3.7.11.2. Substrings

Los substrings son porciones de una string. Tienen un punto de inicio y un punto final

```
'Hello World!' | str index-of 'o'

'Hello World!' | str index-of 'r'

'Hello World!' | str substring 4..8
```

3.7.11.3. Relleno de Strings

Con el comando `fill` puede agregar relleno a una string. El relleno agrega caracteres a la string hasta que tenga una cierta longitud. Por ejemplo:

```
'1234' | fill -a right -c '0' -w 10

'1234' | fill -a left -c '0' -w 10
```

3.7.11.4. Invertir Strings

Esto se puede hacer fácilmente con el comando `str reverse`.

```
'Nushell' | str reverse

['Nushell' 'is' 'cool'] | str reverse
```

3.7.11.5. Análisis de Strings

Con el comando `parse` se puede analizar una string en columnas. Por ejemplo:

```
'Nushell 0.80' | parse '{shell} {version}'

'where all data is structured!' | parse --regex '(?P<subject>\w*\s?\w+) is (?P<adjective>\w+)'
```

Si se sabe que un string contiene datos separados por comas, separados por tabulaciones o separados por varios espacios, puede usar `from csv`, `from tsv` o `from ssv`, respectivamente:

```
"acronym,long\nAPL,A Programming Language" | from csv

"name  duration\nonestop.mid  4:06" | from ssv

"rank\tsuit\nJack\tSpades\nAce\tClubs" | from tsv
```

3.7.12. Comparación de Strings

Además de los operadores estándar `==` y `!=`, existen algunos operadores para comparar específicamente strings entre sí.

Para expresiones regulares:

```
'APL' =~ '^\w{0,3}$'
'FORTRAN' !~ '^\w{0,3}$'
```

Existen otros dos operadores para comparaciones más sencillas:

```
'JavaScript' starts-with 'Java'

'OCaml' ends-with 'Caml'
```

3.7.13. Convertir Strings

Hay varias formas de convertir strings a otros tipos y viceversa.

3.7.13.1. A string

1. Usando `into string`. Ej: `123 | into string`
2. Usando interpolación de strings. Ej: `$'(123)'`

3.7.13.2. De string

1. Usando `into <tipo>`. Ej: `'123' | into int`

3.7.14. Colorear Strings

Se pueden colorear strings con el comando `ansi`. Por ejemplo:

```
$'(ansi purple_bold)This text is a bold purple!(ansi reset)'
```

`ansi purple_bold` hace que el texto sea de color púrpura intenso `ansi reset` restablece el color al predeterminado. (Consejo: Siempre debe terminar las strings coloreadas con `ansi reset`)

3.8. Trabajando con listas

Las listas son equivalentes a las columnas individuales de las tablas. Una lista es esencialmente una "tabla de una columna" (sin nombre de columna). Por lo tanto, cualquier comando que opere en una columna *también* opera en una lista. Por ejemplo, `where` se puede usar con listas:

```
[bell book candle] | where ($it =~ 'b')
```

3.8.1. Creando listas

Una lista es una colección ordenada de valores. Una lista se crea usando corchetes que rodean los valores separados por espacios, saltos de línea y/o comas. Por ejemplo, `[foo bar baz]` o `[foo, bar, baz]`.

Las listas de Nushell son similares a las matrices JSON. El mismo `[ Ïtem1", Ïtem2", Ïtem3" ]` que representa una matriz JSON también se puede usar para crear una lista de Nushell.

3.8.2. Actualizando listas

Se usa `insert` para insertar valores en las listas a medida que fluyen a través de la *pipeline*, por ejemplo, insertamos el valor `10` en el medio de una lista:

```
[1, 2, 3, 4] | insert 2 10
```

También se puede usar `update` para reemplazar el segundo elemento con el valor `10`.

```
[1, 2, 3, 4] | update 1 10
```

3.8.3. Eliminando o Agregando Elementos de la Lista

Además de `insert` y `update`, también se tiene `prepend` y `append`. Estos permiten insertar al principio de una lista o al final de la lista, respectivamente.

Por ejemplo:

```
let colors = [yellow green]
let colors = ($colors | prepend red)
let colors = ($colors | append purple)
let colors = ($colors ++ ["blue"])
let colors = (["black"] ++ $colors)
$colors # [black red yellow green purple blue]
```

En caso de que se requiera eliminar elementos de la lista, hay muchas maneras. `skip` permite omitir las primeras filas de la entrada, mientras que `drop` permite omitir filas numeradas específicas del final de la lista.

```
let colors = [red yellow green purple]
let colors = ($colors | skip 1)
let colors = ($colors | drop 2)
$colors # [yellow]
```

También está `last` y `first` que permiten tomar valores desde el final o el principio de la lista, respectivamente.

```
let colors = [red yellow green purple black magenta]
let colors = ($colors | last 3)
$colors # [purple black magenta]
```

Y desde el principio de una lista,

```
let colors = [yellow green purple]
let colors = ($colors | first 2)
$colors # [yellow green]
```

3.8.3.1. Usando el Operador de Propagación

Para agregar una o más listas, opcionalmente con valores intercalados, se puede usar el operador de propagación (`...`)

```
let x = [1 2]

[
   ...$x
   3
   ...(4..7 | take 2)
]
```

3.8.4. Iterando sobre Listas

Para iterar sobre los elementos de una lista, se utiliza el comando `each` con un bloque de código Nu que especifica qué hacer con cada elemento. El parámetro de bloque (por ejemplo, `|elt|` en `{ |elt| print $elt }`) es el elemento de lista actual, pero el filtro `enumerate` se puede usar para proporcionar valores de `index` y `item` si es necesario. Por ejemplo:

```
let names = [Mark Tami Amanda Jeremy]

$names | each { |elt| $"Hello, ($elt)!" }

$names | enumerate | each { |elt| $"($elt.index + 1) - ($elt.item)" }
```

El comando `where` se puede usar para crear un subconjunto de una lista, filtrando eficazmente la lista según una condición.

El siguiente ejemplo obtiene todos los colores cuyos nombres terminan en “e”.

```
let colors = [red orange yellow green blue purple]
$colors | where ($it | str ends-with 'e')
# El bloque pasado a `where` debe evaluarse como un booleano.
# Esto imprime la lista [orange blue purple].
```

En este ejemplo, mantenemos solo los valores superiores a `7`.

```
let scores = [7 10 8 6 7]
$scores | where $it > 7 # [10 8]
```

El comando `reduce` calcula un solo valor a partir de una lista. Utiliza un bloque que toma 2 parámetros: el elemento actual (convencionalmente llamado `elt`) y un acumulador (convencionalmente

llamado `acc`). Para especificar un valor inicial para el acumulador, usa la opción `--fold` (`-f`). Para cambiar `elt` para que tenga valores de `index` y `item`, se usa el filtro `enumerate`. Por ejemplo:

```
let scores = [3 8 4]
$"total = ($scores | reduce { |elt, acc| $acc + $elt })" # total = 15

$"total = ($scores | math sum)" # enfoque más fácil, mismo resultado

$"product = ($scores | reduce --fold 1 { |elt, acc| $acc * $elt })" # product = 96

$scores | enumerate | reduce --fold 0 { |elt, acc| $acc + $elt.index * $elt.item } # 0*3 + 1*8 +
 ↪  2*4 = 16
```

3.8.5. Accediendo a la Lista

Para acceder a un elemento de la lista en un índice dado, se usa la forma `$name.index` donde `$name` es una variable que contiene una lista.

Por ejemplo, se puede acceder al segundo elemento de la lista a continuación con `$names.1`.

```
let names = [Mark Tami Amanda Jeremy]
$names.1 # da Tami
```

Si el índice está en alguna variable `$index`, podemos usar el comando `get` para extraer el elemento de la lista.

```
let names = [Mark Tami Amanda Jeremy]
let index = 1
$names | get $index # da Tami
```

El comando `length` devuelve el número de elementos de una lista. Por ejemplo, `[red green blue] | length` imprime `3`.

El comando `is-empty` determina si una cadena, lista o tabla está vacía. Se puede usar con listas de la siguiente manera:

```
let colors = [red green blue]
$colors | is-empty # false

let colors = []
$colors | is-empty # true
```

Los operadores `in` y `not-in` se utilizan para probar si un valor está en una lista. Por ejemplo:

```
let colors = [red green blue]
'blue' in $colors # true
'yellow' in $colors # false
'gold' not-in $colors # true
```

El comando `any` determina si algún elemento de una lista coincide con una condición dada. Por ejemplo:

```
let colors = [red green blue]
# ¿Algún nombre de color termina con "e"?
$colors | any {|elt| $elt | str ends-with "e" } # true

# ¿La longitud de algún nombre de color es inferior a 3?
$colors | any {|elt| ($elt | str length) < 3 } # false

let scores = [3 8 4]
# ¿Alguna puntuación es superior a 7?
$scores | any {|elt| $elt > 7 } # true

# ¿Alguna puntuación es impar?
$scores | any {|elt| $elt mod 2 == 1 } # true
```

El comando `all` determina si todos los elementos de una lista coinciden con una condición dada. Por ejemplo:

```
let colors = [red green blue]
# ¿Todos los nombres de color terminan con "e"?
$colors | all {|elt| $elt | str ends-with "e" } # false

# ¿La longitud de todos los nombres de color es mayor o igual a 3?
$colors | all {|elt| ($elt | str length) >= 3 } # true

let scores = [3 8 4]
# ¿Todas las puntuaciones son superiores a 7?
$scores | all {|elt| $elt > 7 } # false

# ¿Todas las puntuaciones son pares?
$scores | all {|elt| $elt mod 2 == 0 } # false
```

3.8.6. Convirtiendo la Lista

El comando `flatten` crea una nueva lista a partir de una lista existente agregando elementos de listas anidadas a la lista de nivel superior. Esto se puede llamar varias veces para aplanar listas anidadas a cualquier profundidad. Por ejemplo:

```
[1 [2 3] 4 [5 6]]
[1 [2 3] 4 [5 6]] | table -e
[1 [2 3] 4 [5 6]] | flatten # [1 2 3 4 5 6]

[[1 2] [3 [4 5 [6 7 8]]]]
```

```
[[1 2] [3 [4 5 [6 7 8]]]] | table -e
[[1 2] [3 [4 5 [6 7 8]]]] | flatten

[[1 2] [3 [4 5 [6 7 8]]]] | flatten | flatten | flatten # [1 2 3 4 5 6 7 8]
```

El comando `wrap` convierte una lista en una tabla. Cada valor de la lista se convertirá en una fila separada con una sola columna:

```
let zones = [UTC CET Europe/Moscow Asia/Yekaterinburg]

# Mostrar el reloj mundial para las zonas horarias seleccionadas
$zones |
  wrap 'Zone' |
  upsert Time {|row|
    (date now | date to-timezone $row.Zone | format date '%Y.%m.%d %H:%M')
  }
```

3.9. Trabajando con Registros (records)

Los registros son aproximadamente equivalentes a las filas individuales de una tabla. Esencialmente son una "tabla de una fila". Por lo tanto, la mayoría de los comandos que operan en una fila de tabla *también* operan en un registro. Por ejemplo, `update` se puede usar con registros:

```
let my_record = {
   name: "Sam"
   age: 30
   }
$my_record | update age { $in + 1 }
```

3.9.1. Creando registros

Un registro es una colección de cero o más asignaciones de pares clave-valor. Es similar a un objeto JSON y se puede crear utilizando la misma sintaxis:

```
# Nushell
{ "apples": 543, "bananas": 411, "oranges": 0 }
╭─────────┬─────╮
│ apples  │ 543 │
│ bananas │ 411 │
│ oranges │ 0   │
╰─────────┴─────╯
# JSON
'{ "apples": 543, "bananas": 411, "oranges": 0 }' | from json
```

En Nushell, los pares clave-valor de un registro también se pueden separar utilizando espacios o

saltos de línea.

Como los registros pueden tener muchos campos, de forma predeterminada, se muestran verticalmente en lugar de de izquierda a derecha. Para mostrar un registro de izquierda a derecha, de debe convertir a un "nuon". Por ejemplo:

```
{
   name: "Sam"
   rank: 10
} | to nuon
```

3.9.2. Actualizando Registros

Al igual que con las listas, se pueden insertar valores en los registros. Por ejemplo:

```
{ "apples": 543, "bananas": 411, "oranges": 0 } | insert pears { 21 }
```

También se pueden actualizar valores:

```
{ "apples": 543, "bananas": 411, "oranges": 0 } | update oranges { 100 }
```

Para hacer una copia de un registro con nuevos campos, se puede:

- Usar el comando `merge`:

```
let first_record = { name: "Sam", rank: 10 }
$first_record | merge { title: "Mayor" }
```

- Usar el operador de propagación (`...`) para expandir el primer registro dentro de un nuevo registro:

```
let first_record = { name: "Sam", rank: 10 }
{
   ...$first_record
   title: "Mayor"
 }
```

3.9.3. Iterando sobre un Registro

Se puede iterar sobre los pares clave-valor de un registro transponiéndolo primero a una tabla:

```
{ "apples": 543, "bananas": 411, "oranges": 0 } |
  transpose fruit count |
  each {|f| $"We have ($f.count) ($f.fruit)" }
```

3.10. Trabajando con tablas

Una de las maneras comunes de ver datos en Nu es a través de una tabla. Nu viene con una serie de comandos para trabajar con tablas para que sea conveniente encontrar lo que se está buscando, y para reducir los datos a solo lo que se necesita.

Para comenzar, obtenemos una tabla que podamos usar:

```
ls
```

3.10.1. Ordenando los Datos

Se puede ordenar una tabla llamando al comando `sort-by` y estableciendo qué columnas utilizar para ordenar. Si se quisiese ordenar la tabla anterior por el tamaño del archivo:

```
ls | sort-by size
```

Se puede ordenar una tabla por cualquier columna que se pueda comparar. Por ejemplo, se podría también haber ordenado la anterior usando las columnas "name", "accessed", o "modified".

3.10.2. Seleccionando Datos

Se puede seleccionar datos de una tabla eligiendo seleccionar columnas específicas o filas específicas, con el comando `select`:

```
ls | select name size
```

Esto ayuda a crear una tabla que está más enfocada en lo que se necesitan. Si solo quisiéramos ver los 5 archivos más pequeños en este directorio, ejecutamos:

```
ls | sort-by size | first 5
```

Notar que primero se ordenam la tabla por tamaño para llegar al archivo más pequeño, y luego se utiliza `first 5` para devolver las primeras 5 filas de la tabla.

También se pueden saltar filas que no se necesitan:

```
ls | sort-by size | first 5 | skip 2
```

También se pueden seleccionar filas directamente:

```
ls | sort-by name | select 5
```

3.10.3. Obteniendo Datos de una Tabla

Hasta ahora, se ha trabajado con tablas recortando la tabla a solo lo que se necesita. A veces, se puede necesitar ir un paso más allá y solo mirar los valores en las celdas en sí mismas en lugar de

tomar una columna completa. Por ejemplo, supongamos que queremos obtener solo una lista de los nombres de los archivos:

```
ls | get name
```

Esto podría verse como el comando `select` que se vió anteriormente:

```
ls | select name
```

Se ven similares, pero hay diferencias:

- `select`: crea una nueva tabla que incluye solo las columnas especificadas.
- `get`: devuelve los valores dentro de la columna especificada como una lista.

3.10.4. Cambiando Datos en una Tabla

En adición a seleccionar datos de una tabla, también se pueden actualizar lo que la tabla tiene. Se puede necesitar combinar tablas, añadir nuevas columnas, o editar el contenido de una celda. En Nu, en lugar de editar en el lugar, cada uno de los comandos en la sección devolverá una nueva tabla.

3.10.4.1. Concatenando Tablas

Se pueden concatenar tablas usando `append`:

```
let first = [[a b]; [1 2]]
let second = [[a b]; [3 4]]

$first
$second

$first | append $second
```

Si los nombres de columna no son idénticos, entonces, adicionalmente, se crearán columnas y valores según sea necesario:

```
let first = [[a b]; [1 2]]
let second = [[a b]; [3 4]]
let third = [[a c]; [3 4]]

$first | append $second | append $third
```

También puede usar el operador `++` como un reemplazo en línea para `append`:

```
$first ++ $second ++ $third
```

3.10.4.2. Combinando Tablas

Se puede utilizar el comando `merge` para combinar dos (o más) tablas:

```
let first = [[a b]; [1 2]]
let second = [[c d]; [3 4]]

$first | merge $second
```

Con una tercera tabla:

```
let third = [[e f]; [5 6]]
```

Se pueden unir las tres tablas de esta manera:

```
$first | merge $second  | merge $third
```

O se podría usar el comando `reduce` para combinar dinámicamente todas las tablas:

```
[$first $second $third] | reduce {|elt, acc| $acc | merge $elt }
```

3.10.5. Añadiendo una columna

Se puede utilizar el comando `insert` para agregar una nueva columna a la tabla. Por ejemplo:

```
{edition: 2024} | save -f edition.json
open edition.json
open edition.json | save -f edition_copia.json
```

Agregamos una columna llamada "next_edition" con el valor 2025:

```
open edition.json | insert next_edition 2025
```

Notar que esta visualización puede ser ligeramente confusa, porque parece que lo que se acaba de hacer es añadir una fila. En este caso, se debe recordar que las filas tienen números, las columnas tienen nombres. Si todavía es confuso, se puede añadir una fila mas, lo quehará que la tabla se renderice como se espera:

```
open edition.json | insert next_edition 2025 | append {edition: 2021 next_edition: 2023}
```

Notar que si se abre el archivo original, el contenido se ha mantenido igual:

```
open edition.json
open edition.json | save -f edition_copia.json
```

Los cambios en Nu son cambios funcionales, lo que significa que trabajan en los valores en sí mismos en lugar de intentar causar un cambio permanente. Esto permite hacer muchos tipos diferentes de trabajo en la *pipeline* hasta que se está listo para escribir el resultado con los cambios elegidos. Se puede escribir el resultado usando el comando `save`:

```
open edition.json |
  insert next_edition 2025 |
  append {edition: 2021 next_edition: 2023} |
  save -f edition2.json

open edition.json
open edition2.json
```

3.10.6. Actualizando una Columna

De forma similar al comando `insert`, el comando [`update` sirve para cambiar el contenido de una columna a un nuevo valor.

```
open edition.json
open edition.json | save -f edition_copia.json
```

Y ahora, actualizamos la edición:

```
open edition.json | update edition 2025
```

También se puede utilizar el comando `upsert`para insertar o actualizar dependiendo de si la columna ya existe.

3.10.7. Moviendo Columnas

Se puede usar `move` para mover columnas en una tabla. Por ejemplo, si se quisiera mover la columna "name" de `ls` después de la columna "size", se puede realizar:

```
ls

ls | move name --after size
```

3.10.8. Renombrando Columnas

También se puede columnas en una tabla pasándola por el comando `rename`:

```
ls | rename filename filetype filesize filedate
```

3.10.9. Rechazando/Eliminando Columnas

También pueden rechazar/eliminar columnas en una tabla pasándola por el comando de `reject`:

```
ls -l

ls -l | reject readonly num_links inode created accessed modified
```

3.11. Navegando y Accediendo a Datos Estructurados

Dado el fuerte soporte de Nushell para datos estructurados, algunas de las tareas más comunes involucran navegar y acceder a esos datos.

3.11.1. Antecedentes

Para los ejemplos y descripciones a continuación, se debe tener en cuenta varias definiciones con respecto a los datos estructurados:

- ***Lista:*** Las listas contienen una serie de cero o más valores de cualquier tipo. Una lista con cero valores se conoce como una "lista vacía".
- ***Registro:*** Los registros contienen cero o más pares de claves con nombre y su valor correspondiente. Los datos en el valor de un registro también pueden ser de cualquier tipo. Un registro con cero pares clave-valor se conoce como un "registro vacío".
- ***Datos Anidados:*** Los valores contenidos en una lista, registro o tabla pueden ser de un tipo básico o datos estructurados en sí mismos. Esto significa que los datos se pueden anidar en múltiples niveles y en múltiples formas:
  - Los valores de la lista pueden contener tablas, registros e incluso otras listas
    - ***Tabla:*** Las tablas son una lista de registros
  - Los valores de registro pueden contener tablas, listas y otros registros
    - Esto significa que los registros de una tabla también pueden contener tablas anidadas, listas y otros registros

  Debido a que una tabla es una lista de registros, cualquier comando o sintaxis que funcione en una lista también funcionará en una tabla. Lo contrario no es necesariamente el caso; hay algunos comandos y sintaxis que funcionan en tablas pero no en listas.

3.11.2. Rutas de Celda

Una ruta de celda (*cellpath*) es la forma principal de acceder a valores dentro de datos estructurados. Esta ruta se basa en un concepto similar al de una hoja de cálculo, donde las columnas tienen nombres y las filas tienen números. Los nombres e índices de las rutas de celda están separados por puntos.

3.11.2.1. Registros Para un registro, la ruta de celda especifica el nombre de una clave, que es una `string`.

Por ejemplo, para acceder a un Valor de Registro:

```
let my_record = {
    a: 5
    b: 42
  }

$my_record.b + 5
```

3.11.2.2. Listas Para una lista, la ruta de celda especifica la posición (índice) del valor en la lista. Este es un `int`:

Por ejemplo, para acceder a un Valor de Lista (recordar que los índices de lista comienzan en 0):

```
let scoobies_list = [ Velma Fred Daphne Shaggy Scooby ]

$scoobies_list.3
```

3.11.2.3. Tablas

- Para acceder a una columna, una ruta de celda usa el nombre de la columna, que es una `string`
- Para acceder a una fila, usa el número de índice de la fila, que es un `int`
- Para acceder a una sola celda, usa una combinación del nombre de la columna con el índice de la fila.

Los siguientes ejemplos usarán la siguiente tabla:

```
let data = [[date                          temps                                    condition ];
            [2022-02-01T14:30:00+05:00,  [ 38.24, 38.50, 37.99, 37.98, 39.10 ],  'summy'   ],
            [2022-02-02T14:30:00+05:00,  [ 35.24, 35.94, 34.91, 35.24, 36.65 ],  'sunny'   ],
            [2022-02-03T14:30:00+05:00,  [ 35.17, 36.67, 34.42, 35.76, 36.52 ],  'cloudy'  ],
            [2022-02-04T14:30:00+05:00,  [ 39.24, 40.94, 39.21, 38.99, 38.80 ],  'rain'    ],
           ]

$data

$data | table -e
```

Esto representa datos meteorológicos en forma de una tabla con tres columnas:

1. ***date***: Una `date` de Nushell para cada día
2. ***temps***: Una `list` de Nushell de 5 valores `float` que representan lecturas de temperatura en diferentes estaciones meteorológicas en el área
3. ***condition***: Una `string` de Nushell para la condición meteorológica de cada día para el área

3.11.2.3.1. Ejemplo - Acceder a una Fila de la Tabla (Registro)

Acceder a los datos del segundo día como un registro:

```
> $data.1
╭──────────┬──────────────────╮
│ date     │ 2 years ago      │
│          │ ╭──┬─────────╮   │
│ temps    │ │ 0 │ 35.24 │   │
│          │ │ 1 │ 35.94 │   │
```

```
|           | | 2 | 34.91 | |
|           | | 3 | 35.24 | |
|           | | 4 | 36.65 | |
|           | ╰───┴───────╯ |
| condition | sunny         |
╰───────────┴───────────────╯
```

3.11.2.3.2. Ejemplo - Acceder a una Columna de la Tabla (Lista)

```
$data.condition
```

3.11.2.3.3. Ejemplo - Acceder a una Celda de la Tabla (Valor)

La condición para el cuarto día:

```
$data.condition.3

$data | get condition | get 3

$data | get condition.3
```

3.11.3. Datos Anidados

Dado que los datos se pueden anidar, una ruta de celda puede contener referencias a múltiples nombres o índices.

3.11.3.1. Ejemplo - Accediendo a Datos de Tabla Anidados

Para obtener la temperatura en la segunda estación meteorológica el tercer día:

```
$data.temps.2.1
```

El primer índice `2` accede al tercer día, luego el siguiente índice `1` accede a la lectura de temperatura de la segunda estación meteorológica.

3.11.4. Usando `get` y `select`

Además de la sintaxis literal de la ruta de celda utilizada anteriormente, Nushell también proporciona varios comandos que utilizan rutas de celda. Los más importantes de estos son:

- `get` es equivalente a usar una ruta de celda literal pero con soporte para nombres de variables y expresiones. `get`, como los ejemplos de ruta de celda anteriores, devuelve el **valor** indicado por la ruta de celda.
- `select` es sutilmente, pero críticamente, diferente. Devuelve la **estructura de datos** especificada en sí misma, en lugar de solo su valor.

- Usar `select` en una tabla devolverá una tabla de igual o menor tamaño
- Usar `select` en una lista devolverá una lista de igual o menor tamaño
- usar `select` en un registro devolverá un registro de igual o menor tamaño

Continuando con la tabla de muestra anterior:

```
$data | get 1

$data | select 1
```

3.11.4.1. Ejemplo - `get` vs. `select` una fila de la tabla

Observe que:

- `get` devuelve el mismo registro que el ejemplo `$data.1` anterior
- `select` devuelve una nueva tabla de una sola fila, incluidos los nombres de las columnas y los índices de las filas

Los índices de fila de la tabla resultante de `select` no son los mismos que los del original. La nueva tabla tiene su propio índice que comienza en 0.

Para obtener el índice original, puede usar el comando `enumerate`. Por ejemplo:

```
$data | enumerate | select 1
```

3.11.4.2. Ejemplo - `select` con múltiples filas y columnas

Debido a que `select` da como resultado una nueva tabla, es posible especificar varios nombres de columna, índices de fila o incluso ambos. Este ejemplo crea una nueva tabla que contiene las columnas de fecha y condición de la primera y segunda filas:

```
$data | first 2 | select date condition
```

3.11.5. Nombres de Clave/Columna con espacios

Si un nombre de clave o un nombre de columna contiene espacios u otros caracteres que impiden que sea accesible como una cadena de palabra simple, entonces el nombre de la clave se puede especificar entre comillas.

Ejemplo:

```
let record_example = {
    "key x":12
    "key y":4
  }
$record_example."key x"

# o
$record_example | get "key x"
```

También se requieren comillas cuando un nombre de clave puede confundirse con un valor numérico.

Ejemplo:

```
let record_example = {
    "1": foo
    "2": baz
    "3": far
  }

$record_example."1"
```

No se debe confundir el nombre de la clave con un índice de fila en este caso. Aquí, al primer elemento se le *asigna* el nombre de clave `1` (una cadena). Si se convierte a una tabla usando el comando `transpose`, la clave `1` (`string`) estaría en el índice de fila `0` (un entero).

3.11.6. Manejo de Datos Faltantes

3.11.6.1. El Operador Opcional De forma predeterminada, el acceso a la ruta de celda fallará si no puede acceder a la fila o columna solicitada. Para suprimir estos errores, puede agregar un `?` a un miembro de la ruta de celda para marcarlo como opcional:

3.11.6.1.1. Ejemplo - El Operador Opcional

Usando los datos de temperatura de arriba:

```
let cp: cell-path = $.temps?.1

$data | reject temps | get $cp
```

De forma predeterminada, las celdas faltantes se reemplazarán por `null` cuando se acceda a ellas a través del operador opcional.

3.11.6.2. Asignar un `default` para datos faltantes o `null` El comando `default` se puede usar para aplicar un valor predeterminado al resultado de una columna faltante o nula.

```
let missing_value = [{a:1 b:2} {b:1}]
$missing_value

let with_default_value = ($missing_value | default 'n/a' a)
$with_default_value

$with_default_value.1.a
```

3.11.7. Otros comandos para acceder a datos estructurados

- `reject` es lo opuesto a `select`, eliminando las filas y columnas especificadas

- `range` especifica las filas de una lista o tabla para seleccionar usando un tipo `range`.

3.12. Edición de Archivos en la Terminal con `ttt`

Cuando trabajamos en ciencia de datos desde la línea de comandos (especialmente dentro de contenedores Docker o servidores remotos vía SSH), necesitamos un editor de texto rápido, ergonómico y que funcione íntegramente dentro de la terminal. En el contenedor `dsin` disponemos de **`ttt`** (*Terminal Text Editor*), ubicado directamente en `$env.PATH` y preconfigurado con resaltado de sintaxis y atajos de teclado intuitivos.

Podemos verificar la versión instalada de `ttt` llamándolo directamente desde Nushell:

```
ttt -v
```

3.12.1. Uso Básico de `ttt`

Para crear o editar un script de Nushell, Python, R o un archivo de configuración dentro de nuestro directorio del capítulo (`/data/ch03`), ejecutamos en una sesión interactiva:

```
ttt mi_script.nu
```

Atajos esenciales preconfigurados en `ttt` dentro de `dsin`:

- **`Ctrl + S`:** Guardar el archivo actual.
- **`Ctrl + Q`:** Salir del editor.
- **`Ctrl + F`:** Buscar texto dentro del archivo.
- **`Ctrl + Z`** / **`Ctrl + Y`:** Deshacer / Rehacer cambios.

También podemos generar archivos estructurados directamente desde pipelines de Nushell en el directorio actual usando `save -f` y luego ejecutarlos o inspeccionarlos:

```
"# Script generado en /data/ch03\ndef saludo [nombre: string] {\n  $\"Hola,
 ↪  ($nombre)!\"\n}\nsaludo 'Ciencia de Datos en Nushell'\n" | save -f ejemplo_ttt.nu
nu ejemplo_ttt.nu
nu ejemplo_ttt.nu | save -f saludo_salida.txt
open saludo_salida.txt
```

3.13. Control de Versiones

Esta sección se introduce en el mundo del control de versiones con Git y cómo plataformas como GitHub y GitLab pueden potenciar el flujo de trabajo en ciencia de datos.

En el desarrollo de cualquier proyecto, especialmente en ciencia de datos, el código fuente evoluciona constantemente. Llevar un control de estos cambios resulta crucial para evitar errores, facilitar la colaboración y garantizar la reproducibilidad de los análisis. Aquí es donde entra en juego el **control de versiones**.

El *control de versiones* es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo. Permite regresar a versiones anteriores, comparar diferentes versiones, identificar quién realizó un cambio específico y mucho más.

Este sistema es esencial para la ciencia de datos debido a:

- **Seguimiento de cambios:** Permite entender la evolución del código, facilitando la identificación y corrección de errores.
- **Colaboración eficiente:** En proyectos colaborativos, varios miembros pueden trabajar simultáneamente sin sobrescribir el trabajo del otro.
- **Restauración de versiones anteriores:** Permite volver a puntos específicos en el desarrollo, lo cual es útil si se introducen errores o se desea revertir un cambio.
- **Reproducibilidad:** Garantiza la consistencia del código y los resultados a lo largo del tiempo, lo que es fundamental en la investigación científica.

  Notar que en esta sección, trabajaremos fuera del contenedor docker.

3.13.1. Git: La herramienta esencial

Git es un sistema de control de versiones distribuido, ampliamente utilizado en la actualidad. A diferencia de sistemas centralizados, cada usuario posee una copia completa del historial del proyecto, lo que permite trabajar sin conexión y facilita la colaboración.

Conceptos básicos de Git:

- **Repositorio:** Un directorio que contiene todos los archivos del proyecto y su historial de cambios.
- **Rama (branch):** Una línea de desarrollo independiente dentro del repositorio. Permite trabajar en nuevas características o corregir errores sin afectar la rama principal.
- **Commit:** Una instantánea del estado del proyecto en un momento determinado. Cada commit contiene un mensaje que describe los cambios realizados.
- **Merge:** La acción de combinar los cambios de una rama a otra.

Estados de un archivo:

- **Modificado (modified):** Se han realizado cambios en el archivo, pero aún no se han guardado en el repositorio.
- **Preparado (staged):** Se han marcado los archivos modificados para incluirlos en el siguiente commit.
- **Confirmado (committed):** Los cambios se han guardado en el repositorio.

Comandos esenciales de Git:

- **`git init`**: Crea un nuevo repositorio Git en el directorio actual.
- **`git clone <URL>`**: Crea una copia local de un repositorio remoto existente.
- **`git add <archivo>`**: Agrega un archivo al área de preparación (staging area).
- **`git commit -m "Mensaje del commit"`**: Crea un nuevo commit con los archivos preparados.
- **`git push`**: Envía los commits locales al repositorio remoto.

- **`git pull`**: Descarga los cambios del repositorio remoto al repositorio local.
- **`git branch`**: Muestra las ramas existentes.
- **`git checkout <nombre_de_la_rama>`**: Cambia a la rama especificada.
- **`git merge <nombre_de_la_rama>`**: Combina la rama especificada con la rama actual.

Flujos de trabajo comunes con Git:

- **Flujo de trabajo centralizado:** Todos los desarrolladores trabajan en una sola rama principal. Es un flujo de trabajo simple, adecuado para proyectos pequeños.
- **Flujo de trabajo con ramas (feature branching):** Se crea una rama separada para cada nueva característica o corrección de errores. Permite un desarrollo más organizado y facilita la colaboración en proyectos grandes.

3.13.2. GitHub/GitLab

GitHub y **GitLab** son plataformas de alojamiento de código basadas en Git que ofrecen herramientas adicionales para la colaboración y gestión de proyectos.

Creación de un repositorio remoto:

1. Crear una cuenta gratuita en GitHub o GitLab.
2. Crear un nuevo repositorio e indicar un nombre y una descripción.

Conexión del repositorio local con el remoto:

1. En la configuración del repositorio remoto, copiar la URL de clonación.
2. En la terminal, navegar al directorio del repositorio local.
3. Ejecutar el comando `git remote add origin <URL_del_repositorio_remoto>`.

Colaboración en proyectos:

- **Clonar un repositorio existente:** Utilizar el comando `git clone <URL_del_repositorio>` para crear una copia local.
- **Bifurcación (fork) de repositorios:** Crear una copia personal de un repositorio para trabajar en él de forma independiente.
- **Pull requests:** Proponer cambios al repositorio original. Se realiza una solicitud al propietario del repositorio para que revise y combine los cambios.

Herramientas adicionales en GitHub/GitLab:

- **Gestión de issues:** Permite registrar errores, tareas y solicitudes de características.
- **Wiki:** Un espacio para documentar el proyecto de forma colaborativa.
- **Integración continua/despliegue continuo (CI/CD):** Permite automatizar pruebas, compilación y despliegue del código.

3.13.3. Mejores prácticas para proyectos de ciencia de datos

- **Organización de proyectos:** Utilizar una estructura de directorios clara para organizar el código, los datos y la documentación.
- **Archivos `.gitignore`:** Excluir del control de versiones archivos innecesarios, como datos sin procesar o archivos temporales.
- **Mensajes de commit informativos:** Describir claramente los cambios realizados en cada commit.
- **Gestión de dependencias:** Utilizar archivos de requisitos para especificar las librerías y versiones necesarias para ejecutar el código.

3.13.4. Clonar

Clonaremos un repositorio para mostrar el procedimiento, utilizando el repositorio `ddgr`, que permite búsquedas web en la terminal.

Fuera del contenedor docker (salir de él mediante `exit` o `Ctrl+d`), realizar lo siguiente:

Creamos directorios necesarios:

```
cd
mkdir -p bin
mkdir -p software
```

Clonamos el repositorio e instalamos:

```
cd software
git clone https://github.com/jarun/ddgr.git
cd ddgr
PREFIX=~/bin make install
```

Añadimos el directorio de instalación al `PATH` de sistema:

```
export PATH="${PATH}:~/bin"

cd ~/bin/bin
chmod +x ddgr
cd
```

Ejecutamos la herramienta:

```
ddgr como clonar un repositorio github
```

> Para salir presione q.
>
> Para poder visualizar los resultados de búsqueda, se debe tener configurado un navegador por defecto. Esto solo es posible fuera de la imagen docker.

3.13.5. Commit y Push

Para crear proyectos en Gitlab o Github, primero se debe crear una cuenta de usuario.

Github permite la existencia de solo un repositorio personal privado (en su versión gratis), mientras que Gitlab no tiene esa limitante.

Una vez creada la cuenta de usuario, se debe autentificar la maquina local mediante una clave `ssh`.

Creamos la clave (seleccione Si a todo):

```
ssh-keygen -t rsa
cat ~/.ssh/id_rsa.pub
```

Copie el contenido de ese archivo y cree una nueva clave ssh en la web de github/gitlab.

Ahora en la web de github/gitlab, creamos un nuevo repositorio vacío y seleccionamos un nombre.

En la máquina local, realizamos los siguiente para conectarla al repositorio.

```
git config --global init.defaultBranch main
cd ~/software
mkdir test #o algun nombre seleccionado
cd test
echo "# test" >> README.md
git init
git add README.md
git commit -m "first commit" #mensaje a eleccion
git branch -M main
git remote add origin git@github.com:your_user/your_repo_name.git
git push -u origin main
```

Esto sincroniza el directorio local con el repositorio.

Supongamos que hacemos algún cambio al contenido de este directorio. Por ejemplo, editamos el archivo `README.md`:

```
nano README.md
```

Para subir los cambios realizamos:

```
git add -A
git status
git commit -am "mensaje apropiado"
git push origin main
```

3.13.6. Hacer aportes a un repositorio

Para realizar aportes a un repositorio, o una *pull request* (PR), se debe realizar un siguiente:

1. **Crear una copia del repositorio o un *fork*.**

- En el navegador web, ir al repositorio de GitHubitlab al que desea contribuir.
- En la esquina superior derecha de la página del repositorio, hacer click en el bóton "Fork". Esto crea una copia del repositorio en su cuenta de GitHub/Gitlab.

2. **Clonar su Repositorio Bifurcado.**

- En la página del *fork* en su repositorio copie la URL de clonación `ssh`.
- En la terminal, ejecutar:

```
git clone <copied_https_url>
```

Reemplace `<copied_https_url>` con la URL que copió en el paso anterior.

3. **Crear una Nueva Rama (*branch*).**

```
cd <repository_directory>
git checkout -b <your-branch-name>
```

Por ejemplo: `git checkout -b fix-typo-in-readme`

4. **Hacer sus Cambios y hacer el *commit*.**

- Realice los cambios deseados en el código o la documentación.
- Realice `git add`, `git commit` y `git push` según lo descrito en la sección anterior.

5. **Crear la Solicitud de Aporte (*Pull Request*).**

- **Navegar a GitHub:** Vuelva al repositorio original que bifurcó en GitHub.
- **Crear PR:** Por lo general, verá un aviso sobre su rama recientemente enviada. Haga clic en el botón "Comparar y solicitar PR". Si no es así, vaya a la pestaña "Solicitudes de extracción" y haga clic en "Nueva solicitud de extracción", luego seleccione su rama.
- **Revisar y Enviar:** Revise cuidadosamente sus cambios, agregue un título y una descripción a su solicitud de extracción explicando sus cambios y haga clic en "Crear solicitud de extracción".

3.13.7. Sugerencias de mensajes commit

Algunas directrices al momento de elegir el mensaje del `commit`.

1. Elija una etiqueta. Algunas sugerencias:

- *build*: Cambios que afectan al sistema de compilación o a las dependencias externas (ejemplos de ámbitos: `gulp`, `broccoli`, `npm`).
- *chore*: Actualización de bibliotecas, derechos de autor u otra configuración del repositorio, incluye la actualización de dependencias.
- *ci*: Cambios en nuestros archivos de configuración y scripts de CI (ejemplos de ámbitos: Travis, Circle, GitHub Actions).

- *docs*: Cambios que no son de código, como corregir errores tipográficos o añadir nueva documentación.
- *feat*: Una confirmación de tipo *feat* introduce una nueva característica en la base de código.
- *fix*: Una confirmación de tipo *fix* corrige un error en su base de código.
- *perf*: Un cambio de código que mejora el rendimiento.
- *refactor*: Un cambio de código que no corrige un error ni añade una característica.
- *style*: Cambios que no afectan al significado del código (espacios en blanco, formato, punto y coma faltantes, etc.).
- *test*: Añadir pruebas que faltan o corregir las existentes.

2. Elija un título para la confirmación.
3. Resuma el *commit*, escribiendo una lista mínima de los cambio realizados.
4. El mensaje *commit* final debe tener el siguiente formato:

```
<etiqueta>: <título>

- Item 1
- Item 2
  etc
```

3.13.8. Recursos adicionales

- **Documentación oficial:**
  - Git: https://git-scm.com/docs
  - GitHub: https://docs.github.com/
  - GitLab: https://docs.gitlab.com/
- **Ejemplos de proyectos de ciencia de datos que utilizan Git y GitHub:**
  - TensorFlow
  - scikit-learn
  - Pandas

3.13.9. Para mayor exploración

Una lista completa de los comandos o herramientas utilizadas en este libro se encuentran en el siguiente link: https://jeroenjanssens.com/dsatcl/list-of-command-line-tools.

4. Programación y Creación de Herramientas de Línea de Comandos

4.1. Programación en Nushell

Esta sección entra en más detalle de Nushell como un lenguaje de programación.

Al igual que la mayoría de los lenguajes de programación permiten definir funciones, Nushell usa comandos personalizados (*custom commands*) para este propósito.

Los *alias* de Nushell funcionan de manera similar a los alias de otras shell y son parte del lenguaje de programación, no solo una característica del shell.

Las operaciones comunes pueden, como la suma o la búsqueda regex, hacerse con *operadores*. No todas las operaciones son compatibles con todos los tipos de datos y Nushell se asegurará de hacerlo saber.

Se pueden almacenar resultados intermedios en *variables*. Las variables pueden ser inmutables, mutables o una constante de tiempo de análisis.

Los *scripts* son la forma más simple de organización del código: simplemente se escribe el código en un archivo y se ejecuta. Sin embargo, también se pueden ejecutar scripts como programas independientes con firmas de línea de comandos utilizando el comando `main`.

4.1.1. Operadores

Nushell soporta los siguientes operadores para operaciones comunes de matemáticas, lógica y cadenas:

Operador	Descripción
`+`	sumar
`-`	restar
`*`	multiplicar
`/`	dividir
`//`	división de piso
`mod`	módulo
`**`	exponenciación (potencia)
`==`	igual
`!=`	no igual
`<`	menor que
`<=`	menor o igual que
`>`	mayor que
`>=`	mayor o igual que
`=~`	coincidencia de expresiones regulares / cadena contiene otra
`!~`	coincidencia inversa de expresiones regulares / cadena *no* contiene otra

Operador	Descripción
`in`	valor en lista
`not-in`	valor no en lista
`not`	no lógico
`and`	y dos expresiones booleanas (cortocircuitos)
`or`	o dos expresiones booleanas (cortocircuitos)
`xor`	o exclusivo dos expresiones booleanas
`bit-or`	o bit a bit
`bit-xor`	xor bit a bit
`bit-and`	y bit a bit
`bit-shl`	desplazamiento bit a bit a la izquierda
`bit-shr`	desplazamiento bit a bit a la derecha
`starts-with`	cadena comienza con
`ends-with`	cadena termina con
`++`	agregar listas

Los paréntesis se pueden usar para agrupar para especificar el orden de evaluación o para llamar a comandos y usar los resultados en una expresión

4.1.1.1. Orden de Operaciones Para entender la precedencia de las operaciones, se puede ejecutar el comando: `help operators | sort-by precedence -r`.

Presentado en orden descendente de precedencia, el artículo detalla las operaciones de la siguiente manera:

- Paréntesis (`()`)
- Exponenciación/Potencia (`**`)
- Multiplicar (`*`), Dividir (`/`), División Entera/Suelo (`//`), y Módulo (`mod`)
- Sumar (`+`) y Restar (`-`)
- Desplazamiento de bits (`bit-shl`, `bit-shr`)
- Operaciones de comparación (`==`, `!=`, `<`, `>`, `<=`, `>=`), pruebas de pertenencia (`in`, `not-in`, `starts-with`, `ends-with`), coincidencia de expresiones regulares (`=~`, `!~`), y agregar a la lista (`++`)
- Bit a bit and (`bit-and`)
- Bit a bit xor (`bit-xor`)
- Bit a bit or (`bit-or`)
- Lógico and (`and`)
- Lógico xor (`xor`)
- Lógico or (`or`)
- Operaciones de asignación
- Lógico not (`not`)

Por ejemplo:

```
3 * (1 + 2)
```

4.1.1.2. Tipos No todos las operaciones tienen sentido para todos los tipos de datos. Si se intenta realizar una operación en tipos de datos no compatibles, se encontrará con un mensaje de error que debería explicar qué salió mal:

```
> "spam" - 1
Error: nu::parser::unsupported_operation (link)

  × Tipos no coincidentes para la operación.
   ╭─[entry #49:1:1]
 1 │ "spam" - 1
   · ──┬── ┬ ┬
   ·   │   │ └── int
   ·   │   └── no admite estos valores.
   ·   └── string
   ╰────
  help: Cambie string o int para que sean los tipos correctos e intente nuevamente.
```

Las reglas a veces pueden parecer un poco estrictas, pero por otro lado, debería haber menos efectos secundarios inesperados

4.1.1.3. Expresión Regular / Operadores string-contains Los operadores `=~` y `!~` proporcionan una forma conveniente de evaluar expresiones regulares. No se necesita saber de expresiones regulares para usarlos; también son una forma fácil de comprobar si una cadena contiene otra.

- `string =~ pattern` devuelve **true** si `string` contiene una coincidencia para `pattern`, y **false** en caso contrario.
- `string !~ pattern` devuelve **false** si `string` contiene una coincidencia para `pattern`, y **true** en caso contrario.

Por ejemplo:

```
"foobarbaz" =~ "bar" # devuelve true
"foobarbaz" !~ "bar" # devuelve false
ls | where name =~ ^nu # devuelve todos los archivos cuyos nombres comienzan con "nu"
```

4.1.1.4. Sensibilidad a Mayúsculas y Minúsculas Los operadores suelen ser sensibles a mayúsculas y minúsculas cuando operan con cadenas. Hay algunas maneras de trabajar sin distinción entre mayúsculas y minúsculas:

1. En los operadores de expresiones regulares, especifique el modificador de modo insensible a mayúsculas y minúsculas `(?i)`:

```
"FOO" =~ "foo" # devuelve falso
"FOO" =~ "(?i)foo" # devuelve verdadero
```

2. Utilice la opción `--ignore-case` del comando `str contains`:

```
"FOO" | str contains --ignore-case "foo"
```

3. Convierta las cadenas a minúsculas con `str downcase` antes de comparar:

```
("FOO" | str downcase) == ("Foo" | str downcase)
```

4.1.1.5. Operador de propagación Nushell tiene un operador de propagación (`...`) para desempaquetar listas y registros. Se puede expandir listas o registros en lugares donde se esperan múltiples valores o pares clave-valor.

Hay tres lugares donde puede usar el operador de propagación:

- En literales de lista.
- En literales de registro.
- En llamadas de comando.

4.1.1.5.1. En literales de Lista

Supongamos que se tienen varias listas que desea concatenar, pero también quiere intercalar algunos valores individuales. Esto se puede hacer con `append` y `prepend`, pero la propagación el operador puede permitir hacerlo más fácilmente.

```
let dogs = [Spot, Teddy, Tommy]
let cats = ["Mr. Humphrey Montgomery", Kitten]
[
    ...$dogs
    Polly
    ...($cats | each { |elt| $"($elt) \(cat\)" })
    ...[Porky Bessie]
    ...Nemo
]
```

El siguiente código es una versión equivalente usando `append`:

```
$dogs |
  append Polly |
  append ($cats | each { |elt| $"($elt) \(cat\)" }) |
  append [Porky Bessie] |
  append ...Nemo
```

Considerar que cada llamada a `append` da como resultado la creación de una nueva lista, lo que

significa que en este segundo ejemplo, se crean 3 listas intermedias innecesarias. Este no es el caso con el operador de propagación, por lo que puede haber beneficios de rendimiento (muy menores) al usar `...` si se está uniendo muchas listas grandes juntas, una y otra vez.

Puede haber notado que el último elemento de la lista resultante anterior es `"...Nemo"`. Esto se debe a que dentro de los literales de lista, solo se puede usar para difundir listas, no cadenas. Como tal, dentro de los literales de lista, solo se puede usar antes de variables (`...$foo`), subexpresiones (`...(foo)`), y literales de lista (`...[foo]`).

El `...` tampoco se reconocerá como el operador de propagación si hay algún espacio en blanco entre él y la siguiente expresión:

```
[ ... [] ]
```

Esto es principalmente para que `...` no se confunda con el operador de propagación en comandos como `mv ... $dir`.

4.1.1.5.2. En literales de Registro

Digamos que se tiene un registro con alguna información de configuración y desea agregar más campos a este registro:

```
let my_config = { path: /tmp, limit: 5 }
```

Se puede crear un nuevo registro con todos los campos de `$config` y algunas nuevas adiciones usando la propagación operador. Se puede usar la propagación de múltiples registros dentro de un solo literal de registro.

```
{
    ...$my_config,
    users: [alice bob],
    ...{ url: example.com },
    ...(sys mem)
}
```

De manera similar a las listas, dentro de los literales de registro, el operador de propagación solo se puede usar antes de las variables (`...$foo`), subexpresiones (`...(foo)`), y literales de registro (`...{foo:bar}`). Aquí también, no es necesario que haya espacio en blanco entre el `...` y la siguiente expresión para que se reconozca como el operador de propagación.

4.1.1.5.3. En llamadas de Comando

También se pueden propagar argumentos a un comando, siempre que tenga un parámetro `rest` o sea un comando externo.

Aquí hay un ejemplo de comando personalizado que tiene un parámetro `rest`:

```
def foo [ --flag req opt? ...args ] { [$flag, $req, $opt, $args] | to nuon }
```

Tiene una opción (`--flag`), un parámetro posicional requerido (`req`), un parámetro posicional opcional (`opt?`), y el parámetro rest (`args`).

Si se tiene una lista de argumentos para pasar a `args`, se puede propagar de la misma manera que propagaría una lista dentro de un literal de lista. Se aplican las mismas reglas: el operador de propagación solo se reconoce antes de variables, subexpresiones y literales de lista, y no se permiten espacios en blanco en el medio.

```
foo "bar" "baz" ...[1 2 3] # Con ..., los números se tratan como argumentos separados
foo "bar" "baz" [1 2 3] # Sin ..., [1 2 3] se trata como un solo argumento
```

Una forma más útil de usar el operador de propagación es si tiene otro comando con un parámetro `rest` y desea que reenvíe sus argumentos a `foo`:

```
def bar [ ...args ] { foo --flag "bar" "baz" ...$args }
bar 1 2 3
```

Puede propagar varias listas en una sola llamada y también intercalar argumentos individuales:

```
foo "bar" "baz" 1 ...[2 3] 4 5 ...(6..9 | take 2) last
```

Las opciones/argumentos con nombre pueden ir después de un argumento de propagación, al igual que pueden ir después de argumentos `rest` regulares:

```
foo "bar" "baz" 1 ...[2 3] --flag 4
```

Si un argumento de propagación viene antes de un parámetro posicional opcional, ese parámetro opcional se trata como omitido:

```
foo "bar" ...[1 2] "not opt"
```

4.1.2. Variables

Los valores de Nushell se pueden asignar a variables con nombre usando las palabras clave `let`, `const` o `mut`. Después de crear una variable, se puede referir a ella usando `$` seguido de su nombre.

4.1.2.1. Tipos de Variables

4.1.2.1.1. Variables Inmutables

Una variable inmutable no puede cambiar su valor después de la declaración. Se declaran usando la palabra clave `let`,

```
let val = 42
$val
$val = 100
```

```
Error: nu::shell::assignment_requires_mutable_variable

  × Assignment to an immutable variable.
   ╭─[entry #10:1:1]
 1 │ $val = 100
   · ──┬─
   ·   └── needs to be a mutable variable
   ╰────
```

Sin embargo, las variables inmutables pueden ser 'sombreadas'. Sombrear significa que se vuelven a declarar y su valor inicial ya no se puede usar dentro del mismo alcance.

```
let val = 42                    # declare a variable
do { let val = 101;  $val }     # in an inner scope, shadow the variable: 101
$val                            # in the outer scope the variable remains unchanged: 42
let val = $val + 1              # now, in the outer scope, shadow the original variable
$val                            # in the outer scope, the variable is now shadowed,
                                # and its original value is no longer available: 43
```

4.1.2.1.2. Variables Mutables

Una variable mutable puede cambiar su valor por asignación. Estas se declaran usando la palabra clave `mut`.

```
mut val = 42
$val += 27
$val
```

Hay un par de operadores de asignación usados con variables mutables

Operador	Descripción
`=`	Asigna un nuevo valor a la variable
`+=`	Suma un valor a la variable y hace que la suma sea su nuevo valor
`-=`	Resta un valor de la variable y hace que la diferencia sea su nuevo valor
`*=`	Multiplica la variable por un valor y hace que el producto sea su nuevo valor
`/=`	Divide la variable por un valor y hace que el cociente sea su nuevo valor
`++=`	Agrega una lista o un valor a una variable

Nota:

1. `+=`, `-=`, `*=` y `/=` solo son válidos en los contextos donde se espera que sus operaciones raíz funcionen. Por ejemplo, `+=` usa la suma, por lo que no se puede usar para contextos donde la suma normalmente fallaría
2. `++=` requiere que la variable **o** el argumento sea una lista.

Más sobre Mutabilidad

Los Cierres y las `def` anidadas no pueden capturar variables mutables de su entorno. Por ejemplo

```
# método ingenuo para contar el número de elementos en una lista
mut x = 0

[1 2 3] | each { $x += 1 }   # error: $x es capturado en un cierre
```

Para usar variables mutables para tal comportamiento, se recomienda usar los bucles.

4.1.2.1.3. Variables Constantes

Una variable constante es una variable inmutable que puede ser completamente evaluada en tiempo de análisis. Estas son útiles con comandos que necesitan saber el valor de un argumento en tiempo de análisis, como `source`, `use` y `plugin use`. Se declaran usando la palabra clave `const`:

```
const script_file = 'path/to/script.nu'
source $script_file
```

4.1.2.2. Escogiendo entre variables mutables e inmutables Se debe tratar de usar variables inmutables para la mayoría de los casos de uso.

Muchos, si no la mayoría, de los casos de uso de variables mutables en Nushell tienen una solución funcional que:

- Solo usa variables inmutables, y como resultado …
- Tiene mejor rendimiento
- Soporta streaming
- Puede soportar características adicionales como `par-each`

Por ejemplo, los contadores de bucles son un patrón común para las variables mutables y están integrados en la mayoría de los comandos de iteración. Por ejemplo, se puede obtener cada elemento y el índice de cada elemento usando `each` con `enumerate`:

```
ls |
  enumerate |
  each { |elt|
    $"Item #($elt.index) is size ($elt.item.size)"
  }
```

También se puede usar el comando `reduce` para trabajar de la misma manera que podrías mutar

una variable en un bucle. Por ejemplo, si se quisiera encontrar la cadena más grande en una lista de cadenas, se podría hacer:

```
[one, two, three, four, five, six] | reduce {|current_item, max|
    if ($current_item | str length) > ($max | str length) {
        $current_item
    } else {
        $max
    }
  }
```

Mientras que `reduce` procesa listas, el comando `generate` se puede usar con fuentes arbitrarias como API REST externas, también sin requerir variables mutables. A continuación, un ejemplo que recupera datos meteorológicos locales cada hora y genera una lista continua a partir de esos datos. El comando `each` se puede usar para consumir cada nuevo elemento de la lista a medida que esté disponible.

```
generate khot {|weather_station|
  let res = try {
    http get -ef $'https://api.weather.gov/stations/($weather_station)/observations/latest'
  } catch {
    null
  }
  sleep 1hr
  match $res {
    null => {
      next: $weather_station
    }
    _ => {
      out: ($res.body? | default '' | from json)
      next: $weather_station
    }
  }
}
| each {|weather_report|
    {
      time: ($weather_report.properties.timestamp | into datetime)
      temp: $weather_report.properties.temperature.value
    }
}
```

4.1.2.2.1. Consideraciones de Rendimiento

Usar comandos de filtro con variables inmutables es a menudo mucho más eficiente que las variables mutables con sentencias tradicionales de control de flujo como `for` y `while`. Por ejemplo:

- Usando una sentencia `for` para crear una lista de 50,000 números aleatorios:

```
timeit {
  mut randoms = []
  for _ in 1..500 {
    $randoms = ($randoms | append (random int))
  }
}
```

Resultado: 1min 4sec 191ms 135µs 90ns

- Usando `each` para hacer lo mismo:

```
timeit {
  let randoms = (1..5_000 | each {random int})
}
```

Resultado: 19ms 314µs 205ns

- Usando `each` con 10,000,000 iteraciones:

```
timeit {
  let randoms = (1..50_000 | each {random int})
}
```

Resultado: 4sec 233ms 865µs 238ns

Como con muchos filtros, la sentencia `each` también transmite sus resultados, lo que significa que la siguiente etapa del pipeline puede continuar procesando sin esperar a que los resultados se recopilen en una variable.

Para tareas que pueden ser optimizadas por paralelización, como se mencionó anteriormente, `par-each` puede tener ganancias de rendimiento aún más drásticas.

4.1.2.3. Nombres de Variables Los nombres de variables en Nushell vienen con algunas restricciones en cuanto a los caracteres que pueden contener. En particular, no pueden contener estos caracteres:

```
.  [  (  {  +  -  *  ^  /  =  !  <  >  &  |
```

Es común que algunos scripts declaren variables que comienzan con `$`. Esto está permitido, y es equivalente a que el `$` no esté allí en absoluto.

```
let $var = 42
# identical to `let var = 42`
```

4.1.3. Comandos personalizados

La capacidad de Nu para componer *pipelines* largos permite mucho control sobre los datos y sistema, pero tiene el precio de escribir mucho. Idealmente, se podrían guardar las *pipelines* bien elaborados para usarlos una y otra vez.

Aquí es donde entran los comandos personalizados.

Un ejemplo de definición de un comando personalizado:

```
def greet [name:string] {
  ['hello' $name]
}
```

En esta definición, se define el comando `greet`, que toma un solo parámetro `name`. Siguiendo a este parámetro está el bloque que representa lo que sucederá cuando se ejecute el comando personalizado. Cuando se llama, el comando personalizado establecerá el valor pasado para `name` como la variable `$name`, que estará disponible para el bloque.

Para ejecutar lo anterior, podemos llamarlo como llamaríamos a los comandos integrados:

```
greet "world"
```

> El valor producido por la última línea de un comando se convierte en el valor devuelto del comando. En este caso, se devuelve una lista que contiene la cadena `'hello'` y `$name`. Para evitar esto, se puede colocar `null` (o el comando `ignore` al final de la *pipeline*, así: `['hello' $name] | null`. También se debe tener en cuenta que la mayoría de los comandos del sistema de archivos, como `save` o `cd`, siempre generan `null`.
>
> Si se quiere generar una sola cadena de texto, se puede usa la sintaxis de interpolación de cadenas para incrustar `$name` en ella:

```
def greet [name:string] {
  $"hello ($name)"
}

greet nushell
```

4.1.4. Nombres de Comandos

En Nushell, un nombre de comando es una cadena de caracteres. Aquí hay algunos ejemplos de nombres de comandos válidos: `greet`, `get-size`, `mycommand123`, `my command`, y `emoji_cmd` (o cualquier símbolo unicode admitido).

> Es una práctica común en Nushell separar las palabras del comando con `-` para una mejor legibilidad._ Por ejemplo `get-size` en lugar de `getsize` o `get_size`.

4.1.5. Subcomandos

También se pueden definir subcomandos a comandos usando un espacio. Por ejemplo, si se quisiera agregar un nuevo subcomando a `str`, se puede crear nombrando el subcomando de forma tal que comience con "str". Por ejemplo:

```
def "str mycommand" [] {
  "hello"
}
```

Se puede llamar al comando personalizado como si fuera un subcomando incorporado de `str`:

```
def "str mycommand" [] { "hello" }
str mycommand
```

Por supuesto, los comandos con espacios en sus nombres se definen de la misma manera:

```
def "custom command" [] {
  "this is a custom command with a space in the name!"
}
```

4.1.6. Tipos de Parámetros

Cuando se definen comandos personalizados, se puede nombrar y, opcionalmente, establecer el tipo para cada parámetro. Por ejemplo, se puede escribir lo anterior como:

```
def greet [name: string] {
  $"hello ($name)"
}
```

Los tipos de parámetros son opcionales. Nushell permite omitirlos y tratar el parámetro como `any` si es así. Si se selecciono un tipo en un parámetro, Nushell comprobará este tipo cuando llame a la función.

Por ejemplo, supongamos que necesitamos un dato tipo `int` en su lugar:

```
def greet [name: int] {
  $"hello ($name)"
}

greet world
```

Si intentamos ejecutar lo anterior, Nushell nos indica que los tipos no coinciden:

```
error: Type Error
  ┌─ shell:6:7
  │
5 │ greet world
```

```
  |       ^^^^^ Expected int
```

Esto puede ayudar a guiar a los usuarios de sus definiciones para que las llamen sólo con los tipos admitidos.

Los tipos actualmente aceptados son:

- any
- binary
- bool
- cell-path
- closure
- datetime
- directory
- duration
- error
- filesize
- float
- glob
- int
- list
- nothing
- number
- path
- range
- record
- string
- table

4.1.7. Parámetros con un Valor Predeterminado

Para hacer un parámetro opcional y directamente proporcionar un valor predeterminado para él, se puede proporcionar un valor predeterminado en la definición del comando.

```
def greet [age, name = "nushell"] {
  $"hello ($name) + ($age)"
}
```

Se puede llamar a este comando sin el parámetro o con un valor para anular el valor predeterminado:

```
def greet [name = "nushell"] { $"hello ($name)" }
greet
greet world
```

También puede combinar un valor predeterminado con un requisito de tipo:

```
def congratulate [age: int = 18] {
  $"Happy birthday! You are ($age) years old now!"
}
```

Si se desea comprobar si un parámetro opcional está presente o no y no solo confiar en un valor predeterminado, se pueden utilizar parámetros posicionales opcionales en su lugar.

4.1.8. Parámetros Posicionales Opcionales

Por defecto, los parámetros posicionales son requeridos. Si un parámetro posicional no se entrega, se encontrará un error:

```
  × Missing required positional argument.
   ╭─[entry #23:1:1]
 1 | greet
   ·       ▲
   ·       ╰── missing name
   ╰────
  help: Usage: greet <name>
```

Podemos en cambio marcar un parámetro posicional como opcional poniendo un signo de interrogación (`?`) después de su nombre. Por ejemplo:

```
def greet [name?: string] {
  $"hello ($name)"
}

greet
greet world
```

Hacer un parámetro posicional opcional no cambia su nombre cuando se accede en el cuerpo. Como el ejemplo de arriba muestra, todavía se accede con `$name`, a pesar del sufijo `?` en la lista de parámetros.

Cuando un parámetro opcional no se entrega, su valor en el cuerpo del comando es igual a `null`. Podemos usar esto para actuar en el caso donde un parámetro no fue entregado:

```
def greet [name?: string] {
  if ($name == null) {
    "hello, I don't know your name!"
  } else {
    $"hello ($name)"
  }
}

greet
```

Si solo se quiere establecer un valor por defecto cuando el parámetro falta, es más simple usar un valor por defecto en cambio.

Si se usan parámetros posicionales requeridos y opcionales juntos, entonces los parámetros requeridos deben aparecer en la definición primero.

4.1.9. Opciones (*flags*)

Además de pasar parámetros posicionales, también puede pasar parámetros con nombre definiendo opciones para los comandos personalizados.

Por ejemplo:

```
def greet [
  name: string
  --age: int
] {
  [$name $age]
}
```

En la definición de `greet` anterior, se define el parámetro posicional `name`, así como una opción llamada `age`. Esto permite a quien llama a `greet` pasar opcionalmente también el parámetro `age`.

Se puede llamar a lo anterior usando:

```
greet world --age 10
```

O:

```
greet --age 10 world
```

O incluso omitir la opción por completo:

```
greet world
```

Las opciones también se pueden definir para tener una versión abreviada. Esto permite pasar una opción de forma más simple, así como una opción larga-y-más-fácil-de-leer.

Extendemos el ejemplo anterior para usar una opción abreviada para el valor de `age`:

```
def greet [
  name: string
  --age (-a): int
] {
  [$name $age]
}
```

> *Nota:* Las opciones se nombran por su nombre largo, por lo que el ejemplo anterior necesitaría usar `$age` y no `$a`.

Ahora, se puede llamar a esta definición actualizada usando la la opción abreviada:

```
greet -a 10 hello
```

Las opciones también se pueden usar como interruptores (*switches*) básicos. Esto significa que su presencia o ausencia se toma como un argumento para la definición. Extendiendo el ejemplo anterior:

```
def greet [
  name: string
  --age (-a): int
  --twice
] {
  if $twice {
    [$name $name $age $age]
  } else {
    [$name $age]
  }
}
```

Y la definición se puede llamar como:

```
greet -a 10 --twice hello
```

O simplemente sin la opción del switch:

```
greet -a 10 hello
```

También se puede asignar un valor de verdadero/falso para habilitar/deshabilitar la opción:

```
greet -a 10 --twice=false hello
greet -a 10 --twice=true hello
```

Las opciones pueden contener guiones. Se puede acceder a ellas reemplazando el guion con un guion bajo:

```
def greet [
  name: string
  --age (-a): int
  --two-times
] {
  if $two_times {
    [$name $name $age $age]
  } else {
    [$name $age]
  }
}
```

4.1.10. Parámetros rest

Puede haber casos en los que se desee definir un comando que tome cualquier número de argumentos posicionales. Podemos hacer esto con un parámetro te tipo `rest`, usando la siguiente sintaxis `...`:

```
def greet [...name: string] {
  print "hello all:"
  for $n in $name {
    print $n
  }
}

greet earth mars jupiter venus
```

Se puede llamar a la definición anterior del comando `greet` con cualquier número de argumentos, incluyendo ninguno en absoluto. Todos los argumentos se recopilan en `$name` como una lista.

Los parámetros rest se pueden usar junto con los parámetros posicionales:

```
def greet [vip: string, ...name: string] {
  print $"hello to our VIP ($vip)"
  print "and hello to everybody else:"
  for $n in $name {
    print $n
  }
}

greet moon earth mars jupiter venus
```

Para pasar una lista a un parámetro `rest`, puede usar el operador de propagación `...`:

```
let planets = [earth mars jupiter venus]

greet moon ...$planets
```

4.1.11. Documentando un Comando

Para ayudar mejor a los usuarios acerca del uso de los comandos personalizados, también se pueden documentar con descripciones adicionales para los comandos y parámetros.

Tomando el ejemplo anterior:

```
def greet [
  name: string
  --age (-a): int
] {
  [$name $age]
}
```

Una vez definido, se puede ejecutar `help greet` para obtener la información de ayuda para el comando:

```
Usage:
  > greet <name> {flags}

Parameters:
  <name>

Flags:
  -h, --help: Display this help message
  -a, --age <integer>
```

Se puede ver el parámetro y la opción que se definió, así como la bandera de ayuda `-h` que tienen todos los comandos.

Para mejorar esta ayuda, se pueden agregar descripciones a la definición que aparecerán en la ayuda:

```
# A greeting command that can greet the caller
def greet [
  name: string      # The name of the person to greet
  --age (-a): int   # The age of the person
] {
  [$name $age]
}
```

Los comentarios que se incluyen en la definición y sus parámetros aparecen como descripciones dentro del `help` del comando.

Ahora, si se ejecuta `help greet`, se entrega un texto de ayuda más útil:

```
A greeting command that can greet the caller

Usage:
  > greet <name> {flags}

Parameters:
  <name> The name of the person to greet

Flags:
  -h, --help: Display this help message
  -a, --age <integer>: The age of the person
```

4.1.12. Salida de la *pipeline*

Los comandos personalizados transmiten su salida al igual que los comandos incorporados. Por ejemplo:

```
ls | get name
```

Hacemos una copia de `ls` a un comando personalizado:

```
def my-ls [] { ls }
```

Se puede usar la salida de este comando tal como se haría con `ls`.

```
my-ls | get name
```

Esto nos permite construir fácilmente comandos personalizados y procesar su salida. Tenga en cuenta que no usamos sentencias de retorno como otros lenguajes. En cambio, construimos pipelines que generan flujos de datos que se pueden conectar a otros pipelines.

4.1.13. Entrada de la *pipeline*

Los comandos personalizados también pueden tomar entrada desde la *pipeline*, al igual que otros comandos. Esta entrada se pasa automáticamente al bloque que utiliza el comando personalizado.

Consideremos el siguiente comando que calcula el doble para cada elemento de una lista:

```
def double [] {
  each { |elt| 2 * $elt }
}
```

Y puede utilizarse de la siguiente forma:

```
[1 2 3] | double
```

También se puede almacenar la entrada para su uso posterior utilizando la variable `$in`:

```
def nullify [...cols] {
  let start = $in
  $cols | reduce --fold $start { |col, df|
    $df | upsert $col null
  }
}
```

4.1.14. Control de flujo

Nushell proporciona varios comandos que ayudan a determinar cómo se ejecutan diferentes grupos de código. En los lenguajes de programación esta funcionalidad a menudo se conoce como *control de flujo*.

4.1.14.1. Elección (Condicionales)

Los siguientes comandos ejecutan código basado en una condición dada.

Los comandos de elección/condicionales son expresiones, por lo que devuelven valores, a diferencia de los otros comandos en esta página. Esto significa que lo siguiente funciona.

```
'foo' | if $in == 'foo' { 1 } else { 0 } | $in + 2
```

4.1.14.1.1. `if`

`if` evalúa de código ramificados basados en los resultados de una o más condiciones similares a la funcionalidad "if" en otros lenguajes de programación. Por ejemplo:

```
let x = 5
if $x > 0 { 'positivo' }
```

Devuelve `'positivo'` cuando la condición es `true` (`$x` es mayor que cero) y `null` cuando la condición es `false` (`$x` es menor o igual a cero).

Se puede agregar una rama `else` al `if` después del primer bloque que se ejecuta y devuelve el valor resultante del bloque `else` cuando la condición es `false`. Por ejemplo:

```
if $x > 0 { 'positivo' } else { 'no-positivo' }
```

Esta vez devuelve `'positivo'` cuando la condición es `true` (`$x` es mayor que cero) y `'no-positivo'` cuando la condición es `false` (`$x` es menor o igual a cero).

También podemos se pueden anidar múltiples `if`s de la siguiente manera:

```
if $x > 0 { 'positive' } else if $x == 0 { 'zero' } else { "negative" }
```

o bien:

```
if $x > 0 {
   'positive'
} else if $x == 0 {
   'zero'
} else {
   "negativo"
}
```

Cuando la primera condición es `true` (`$x` es mayor que cero) devolverá `'positivo'`, cuando la primera condición es `false` y la siguiente condición es `true` (`$x` es igual a cero) devolverá `'zero'`, de lo contrario devolverá `'negativo'` (cuando `$x` es menor que cero).

4.1.14.1.2. `match`

`match` ejecuta una de varias ramas condicionales basadas en el valor dado para coincidir. También puede hacer alguna coincidencia de patrones para desempaquetar valores en tipos compuestos como listas y registros.

El uso básico de `match` puede ejecutar condicionalmente diferentes códigos como una sentencia "switch" común en otros lenguajes. `match` comprueba si el valor después de la palabra `match` es igual al valor al comienzo de cada rama antes del `=>` y si lo es, ejecuta el código después del `=>` de esa rama.

```
match 2 {
    1 => 'one',
    2 => {
        let w = 'w'
        't' + $w + 'o'
    },
    3 => 'three',
    4 => 'four'
}
```

Las ramas pueden devolver un solo valor o, como se muestra en la segunda rama, pueden devolver los resultados de un bloque.

Rama comodín

También puede tener una condición comodín para cuando el valor dado no coincide con ninguna de las otras condiciones al tener una rama cuyo valor coincidente es `_`.

```
let foo = match 7 {
    1 => 'one',
    2 => 'two',
    3 => 'three',
    _ => 'other number'
}

$foo
```

> Notar que , `match` es una expresión, por lo que podemos asignar el resultado a una variable.

Coincidencia de patrones

Se pueden "desempaquetar" valores de tipos como listas y registros con coincidencia de patrones. Luego se puede asignar variables a las partes que se desea desempaquetar y usarlas en las expresiones coincidentes.

```
let foo = { name: 'bar', count: 7 }

match $foo {
    { name: 'bar', count: $it } => ($it + 3),
    { name: _, count: $it } => ($it + 7),
    _ => 1
}
```

El _ en la segunda rama significa que coincide con cualquier registro con el campo `name` y `count`, no solo con aquellos donde `name` es `'bar'`.

Guardias

También puede agregar una condición adicional a cada rama llamada "guardia" (*guard*) para determinar si la rama debe coincidir. Para hacerlo, después del patrón coincidente, se utiliza un `if` y luego la condición antes del `=>`.

```
let foo = { name: 'bar', count: 7 }

match $foo {
    { name: 'bar', count: $it } if $it < 5 => ($it + 3),
    { name: 'bar', count: $it } if $it >= 5 => ($it + 7),
    _ => 1
}
```

4.1.14.2. Bucles

Los comandos de bucle permiten repetir un bloque de código varias veces.

4.1.14.2.1. Bucles y Otros Comandos de Iteración

La funcionalidad de los comandos de bucle es similar a los comandos que aplican un cierre sobre elementos en una lista o tabla como `each` o `where` y muchas veces se puede lograr lo mismo con cualquiera. Por ejemplo:

```
mut result = []
for $it in [1 2 3] {
   $result = ($result | append ($it + 1))
}

$result


[1 2 3] | each { $in + 1 }
```

Si bien puede ser tentador usar bucles si se está familiarizado con ellos en otros lenguajes, se considera más en el estilo Nushell (idiomático) usar comandos que aplican cierres cuando se puede

resolver un problema de cualquier manera. La razón de esto es debido a una desventaja bastante grande con el uso de bucles.

Desventajas del Bucle

La mayor desventaja de los bucles es que son sentencias, a diferencia de `each` que es una expresión. Las expresiones, como `each` siempre resultan en algún valor de salida, sin embargo, las sentencias no.

Esto significa que no funcionan bien con variables inmutables y el uso de variables inmutables se considera más estilo Nushell. Sin una variable mutable declarada de antemano en el ejemplo de la sección anterior, sería imposible usar `for` para obtener la lista de números con números incrementados, o cualquier valor en absoluto.

Las sentencias tampoco funcionan en las *pipelines* de Nushell, que requieren alguna salida. De hecho, Nushell dará un error si se intentas:

```
[1 2 3] | for x in $in { $x + 1 } | $in ++ [5 6 7]
Error: nu::parser::unexpected_keyword

  × Statement used in pipeline.
   ╭─[entry #5:1:1]
 1 │ [1 2 3] | for x in $in { $x + 1 } | $in ++ [5 6 7]
   ·           ─┬─
   ·            ╰── not allowed in pipeline
   ╰────
  help: 'for' keyword is not allowed in pipeline. Use 'for' by itself, outside of a pipeline.
```

Debido a que Nushell está muy orientado a la *pipeline*, esto significa que usar comandos de expresión como `each` es típicamente más natural que las sentencias de bucle.

Ventajas del Bucle

Si los bucles tienen una desventaja tan grande, ¿por qué existen? Bueno, una razón es que los cierres (*clousures*), como los que usa `each`, no pueden modificar variables mutables en el entorno circundante. Si se intentas modificar una variable mutable en un cierre, se obtendrá un error:

```
mut foo = []

[1 2 3] | each { $foo = ($foo | append ($in + 1)) }
Error: nu::parser::expected_keyword

  × Capture of mutable variable.
   ╭─[entry #8:1:1]
 1 │ [1 2 3] | each { $foo = ($foo | append ($in + 1)) }
   ·                   ──┬─
   ·                     ╰── capture of mutable variable
   ╰────
```

Si se modifica una variable de entorno en un cierre, se puede, pero solo se modificará dentro del alcance del cierre, dejándola sin cambios en cualquier otro lugar. Los bucles, sin embargo, usan bloques lo que significa que pueden modificar una variable mutable regular o una variable de entorno dentro del alcance mayor.

```
mut result = []

for $it in [1 2 3] { $result = ($result | append ($it + 1)) }

$result
```

4.1.14.2.2. `for`

`for` recorre un rango o colección como una lista o una tabla.

```
for x in [1 2 3] { $x * $x | print }
```

Alternativas de Comandos de Expresión

- `each`
- `par-each`
- `where`
- `reduce`

4.1.14.2.3. `while`

`while` repite el mismo bloque de código hasta que la condición dada es `false`.

```
mut x = 0;

while $x < 10 { $x = $x + 1 }

$x
```

Alternativas de Comandos de Expresión

Los comandos “until” y otros comandos “while”

- `take until`
- `take while`
- `skip until`
- `skip while`

4.1.14.2.4. `loop`

`loop` repite un bloque infinitamente. Se puede usar `break` (como se describe en la siguiente sección) para limitar cuántas veces se repite. También puede ser útil para ejecutar scripts continuamente,

como un indicador interactivo.

```
mut x = 0; loop { if $x > 10 { break }; $x = $x + 1 }; $x
```

4.1.14.2.5. `break`

`break` detendrá la ejecución del código en un bucle y reanudará la ejecución después del bucle. Efectivamente "rompiendo" el bucle.

```
for x in 1..10 { if $x > 3 { break }; print $x }
```

4.1.14.2.6. `continue`

`continue` detendrá la ejecución del bucle actual, omitiendo el resto del código en el bucle, e irá al siguiente bucle. Si el bucle normalmente terminaría, como si `for` ha iterado a través de todos los elementos dados, o si la condición de `while` ahora es falsa, no se repetirá de nuevo y la ejecución continuará después del bloque de bucle.

```
mut x = -1
while $x <= 6 {
    $x = $x + 1;
    if $x mod 3 == 0 {
        continue
    }
    print $x
}
```

4.1.14.2.7. Errores

`error make`

`error make` crea un error que detiene la ejecución del código y cualquier código que lo haya llamado, hasta que sea manejado por un bloque `try`, o termine el script y muestre el mensaje de error. Esta funcionalidad es la misma que "excepciones" en otros lenguajes.

```
error make { msg: 'Some error info' }; print 'unprinted'


Error:   × Some error info
   ╭─[entry #9:1:1]
 1 │ print 'printed'; error make { msg: 'Some error info' }; print 'unprinted'
   ·                  ─────┬─────
   ·                       └── originates from here
   ╰────
```

El registro que se le pasa proporciona información al código que lo captura o al mensaje de error

resultante.

`try`

`try` capturará los errores creados en cualquier lugar del bloque de código de `try` y reanudará la ejecución del código después del bloque.

```
try { error make { msg: 'Some error info' }}; print 'Resuming'
```

Esto incluye la captura de errores incorporados.

```
try { 1 / 0 }; print 'Resuming'
```

El valor resultante será `nothing` si ocurre un error y el valor devuelto del bloque si no ocurre un error.

Si se incluye un bloque `catch` después del bloque `try`, ejecutará el código en el bloque `catch` si ocurrió un error en el bloque `try`.

```
try { 1 / 0 } catch { 'An error happened!' } | $in ++ ' And now I am resuming.'
```

No se ejecutará el bloque `catch` si no se produce un error.

```
try { 1 / 0 } catch { |e| $e.msg }
```

4.1.14.2.8. Otros

`return`

`return` finaliza un cierre o comando anticipadamente donde se llama, sin ejecutar el resto del comando/cierre, y devuelve el valor dado. No es a menudo necesario ya que el último valor en un cierre o comando también se devuelve, pero a veces puede ser conveniente.

```
def 'positive-check' [it] {
    if $it > 0 {
        return 'positive'
    };

    'non-positive'
}
```

```
positive-check 3

positive-check (-3)

let positive_check = {|elt| if $elt > 0 { return 'positive' }; 'non-positive' }

do $positive_check 3
```

```
do $positive_check (-3)
```

4.2. Ejemplo de Desarrollo de Comando: API de Google Maps y Geocodificación

En esta sección desarrollaremos un módulo reutilizable de Nushell (`maps.nu`) que interactúa con la **API de Google Maps** (y su equivalente abierto en **OpenStreetMap Nominatim**) para realizar geocodificación directa (obtener latitud y longitud a partir de una dirección postal) y geocodificación inversa (obtener la dirección formateada a partir de coordenadas geográficas).

4.2.1. Obtener API Key

Para utilizar los servicios oficiales de Google Maps Platform se debe crear una clave de acceso (*API Key*) en el sitio web de Google Cloud Console:

1. Ingrese al sitio web de Google Cloud Console.
2. Cree un nuevo proyecto.
3. Diríjase al *Dashboard* y seleccione el proyecto recién creado.
4. Diríjase a *APIs & Services* → *Library*.
5. Busque y active **Geocoding API** y **Directions API**.
6. Diríjase a *APIs & Services* → *Credentials*.
7. Haga clic en *Create Credentials* → *API Key* y guarde la clave obtenida en un lugar seguro.

4.2.2. Exploración de la API y Anatomía de una URL

De acuerdo con la documentación oficial de Geocoding API, podemos obtener las coordenadas geográficas asociadas a una dirección o viceversa mediante peticiones HTTP GET.

Nos ubicamos primero en el directorio de trabajo del capítulo 4 (`/data/ch04`):

```
cd /data/ch04
```

A partir de este punto, todos los ejemplos y herramientas de este capítulo se ejecutan dentro de `/data/ch04`. Definimos la clave de API junto con la dirección de prueba correspondiente al campus Concepción de la Universidad del Bío-Bío:

```
let mapsAPIkey = "the_api_key"
let address = "Avenida Collao 1202, Concepcion, Chile"
```

Para consultar la API, debemos construir una URL con la siguiente estructura:

```
https://maps.googleapis.com/maps/api/geocode/json?address=Avenida+Collao+1202%2C+Concepcion%2C+Ch
api_key
```

Toda URL (*Uniform Resource Locator*) se compone de las siguientes partes estructurales:

- **Scheme (Esquema):** Indica el *protocolo* utilizado para acceder al recurso:
  - `http://` (*Hypertext Transfer Protocol*): Protocolo estándar sin cifrar.
  - `https://` (*HTTP Secure*): Versión cifrada mediante TLS/SSL para comunicación segura.
  - `ftp://` (*File Transfer Protocol*): Transferencia de archivos.
  - `file://`: Acceso a archivos en el sistema local.
- **Host (Servidor):** Especifica el nombre de dominio (por ejemplo, `maps.googleapis.com`) o dirección IP donde reside el servicio, opcionalmente seguido de un puerto (`:8080`).
- **Path (Ruta):** Identifica el recurso o *endpoint* específico dentro del servidor, comenzando con `/` (por ejemplo, `/maps/api/geocode/json`).
- **Parámetros (*Query Parameters*):** Pares `clave=valor` opcionales que siguen al signo de interrogación `?` y se separan mediante `&` (por ejemplo, `address=...&key=...`).

En lugar de concatenar cadenas manualmente (lo que es propenso a errores con espacios y caracteres especiales), el comando **`url join`** de Nushell construye y codifica automáticamente una URL válida a partir de un registro estructurado, y podemos guardar esta estructura en un archivo de configuración:

```
let mapsAPIkey = "the_api_key"
let address = "Avenida Collao 1202, Concepcion, Chile"
let url_record = {
    scheme: "https"
    host: "maps.googleapis.com"
    path: "/maps/api/geocode/json"
    params: {
        address: $address
        key: $mapsAPIkey
    }
}
$url_record | url join
$url_record | save -f consulta_maps.json
```

De igual manera, el comando inverso **`url parse`** descompone cualquier URL en sus componentes estructurales como un registro de Nushell:

```
open consulta_maps.json | url join | url parse
open consulta_maps.json | url join | url parse | save -f url_descompuesta.json
```

Cuando se realiza la petición con una clave activa mediante `let response = (http get $url)`, la respuesta JSON de Google Maps contiene la tabla `results`, dentro de la cual se ubica el registro `geometry.location` con los campos numéricos `lat` y `lng`:

```
# Estructura de respuesta devuelta por Google Geocoding API
let sample_response = {
    status: "OK"
    results: [
        {
```

```
            formatted_address: "Collao 1202, Concepción, Bío Bío, Chile"
            geometry: {
                location: { lat: -36.8221967, lng: -73.0120093 }
            }
        }
    ]
}
$sample_response.results | select formatted_address
$sample_response.results.geometry.location
$sample_response.results.geometry.location | save -f coordenadas_ubb.json
```

4.2.3. Empaquetando el Módulo `maps.nu` con `ttt`

Podemos compilar estas dos operaciones (geocodificación directa e inversa) en un módulo reutilizable `maps.nu` dentro de `/data/ch04` editándolo con `ttt`:

```
ttt maps.nu
```

El archivo `maps.nu` (ya disponible en `/data/ch04/maps.nu`) contiene los dos comandos exportados `loc-from-address` y `address-from-loc`:

```
open --raw maps.nu
```

Cuando se cuenta con una clave válida de Google Maps, se importa el módulo con `use maps.nu *` y se invocan ambos comandos:

```
use maps.nu *
loc-from-address "Avenida Collao 1202, Concepcion, Chile"
address-from-loc -36.8221967 -73.0120093
```

4.3. Conversión de One-Liners en Herramientas de Línea de Comandos

A lo largo de este libro construiremos numerosos pipelines compactos de una sola línea (*one-liners*). Cuando necesitamos reutilizar un *one-liner* frecuentemente sobre distintos archivos o con diferentes parámetros, conviene transformarlo en una **herramienta de línea de comandos ejecutable**.

En `/data/ch04` contamos con el texto completo de *Las aventuras de Alicia en el país de las maravillas* (`alice.txt`) y una lista de palabras vacías del idioma inglés (`stopwords`):

```
ls
```

Primero, veamos cómo obtener las 10 palabras más frecuentes de `alice.txt` en Nushell (sin filtrar y filtrando *stopwords*), mostrando el resultado en pantalla y guardándolo también en un archivo CSV:

```
open --raw alice.txt |
  str lowercase |
  parse --regex '(?P<word>[a-z]{2,})' |
  get word |
  uniq -c |
  rename word count |
  sort-by count --reverse |
  first 10
```

Ahora filtramos las palabras vacías contenidas en el archivo `stopwords` y guardamos el resultado tanto en pantalla como en el archivo `top10_alice.csv`:

```
let sw = (open --raw stopwords | lines | each { str trim | str lowercase })
let top10 = (
    open --raw alice.txt
    | str lowercase
    | parse --regex '(?P<word>[a-z]{2,})'
    | get word
    | where { |w| not ($w in $sw) }
    | uniq -c
    | rename word count
    | sort-by count --reverse
    | first 10
)
$top10
$top10 | save -f top10_alice.csv
```

Para convertir este *one-liner* en una herramienta de línea de comandos reutilizable y parametrizada en Nushell, seguimos **6 pasos sistemáticos**:

1. Crear el archivo de script (`top-words-1.nu`).
2. Otorgar permisos de ejecución (`chmod u+x`).
3. Definir la línea *shebang* (`#!/usr/bin/env nu`).
4. Remover la entrada fija para leer desde el pipe (`$in`).
5. Añadir argumentos y opciones tipadas con `def main [...]`.
6. Extender la variable de entorno `$env.PATH`.

4.3.1. Paso 1: Crear el Archivo

En `/data/ch04` guardamos nuestro pipeline inicial en `top-words-1.nu` (también podemos crearla o editarla interactivamente con `ttt top-words-1.nu`):

```
open --raw top-words-1.nu
nu top-words-1.nu
nu top-words-1.nu | save -f salida_paso1.csv
```

4.3.2. Paso 2: Dar Permisos de Ejecución

Copiamos `top-words-1.nu` a `top-words-2.nu` y le otorgamos permisos de ejecución para el usuario propietario mediante `chmod u+x`:

```
cp top-words-1.nu top-words-2.nu
chmod u+x top-words-2.nu
ls -l top-words-1.nu top-words-2.nu | select name mode size
```

En la columna `mode` (por ejemplo, `-rwxr-xr-x`), el primer carácter `-` indica un archivo regular (o `d` si es un directorio), los tres siguientes (`rwx`) corresponden a los permisos de lectura (`r`), escritura (`w`) y ejecución (`x`) del propietario (`u`), seguidos por los permisos del grupo (`g`) y de otros usuarios (`o`).

4.3.3. Paso 3: Definir el Shebang

El *shebang* (`#!/usr/bin/env nu`) en la primera línea del archivo le indica al sistema operativo que debe utilizar el ejecutable `nu` ubicado en el entorno para interpretar el script cuando se invoca directamente como `./top-words-3.nu`:

```
open --raw top-words-3.nu
./top-words-3.nu
./top-words-3.nu | save -f salida_paso3.json
```

4.3.4. Paso 4: Remover la Entrada Fija

Si reemplazamos `open --raw alice.txt` por la variable especial de entrada de pipe `$in` en `top-words-4.nu`, nuestra herramienta deja de estar atada a un único archivo y puede analizar cualquier texto que le enviemos mediante `|`:

```
open --raw top-words-4.nu
open --raw alice.txt | ./top-words-4.nu
open --raw alice.txt | ./top-words-4.nu | save -f salida_paso4.json
```

4.3.5. Paso 5: Añadir Argumentos y Opciones Tipadas (`def main`)

En Nushell, al definir `def main [num_words: int = 10, --stopwords-file (-s): path = "stop-words"]` dentro de un script ejecutable (`top-words-5.nu`), Nushell valida automáticamente los tipos de los argumentos, asigna valores por defecto y genera la página de ayuda `--help` sin necesidad de librerías externas:

```
open --raw top-words-5.nu
open --raw alice.txt | ./top-words-5.nu 8
open --raw alice.txt | ./top-words-5.nu 8 | save -f top8_alice.json
```

4.3.6. Paso 6: Extender el `$env.PATH`

Para poder invocar nuestra herramienta desde cualquier directorio sin escribir `./`, agregamos el directorio `/data/ch04` (o `/home/dsin/bin`) a `$env.PATH`:

```
cp top-words-5.nu /home/dsin/bin/top-words-nu
chmod +x /home/dsin/bin/top-words-nu
open --raw alice.txt | top-words-nu 5
open --raw alice.txt | top-words-nu 5 | save -f top5_alice.json
```

4.4. Creando Herramientas de Línea de Comandos con Python y R

Dado que la línea de comandos es agnóstica al lenguaje, podemos combinar scripts escritos en **Python** (`#!/usr/bin/env python3`) y **R** (`#!/usr/bin/env Rscript`) dentro de nuestros pipelines de Nushell. Siguiendo las buenas prácticas de este libro, siempre creamos o inspeccionamos primero el archivo `.py` o `.R` en el directorio del capítulo y luego lo invocamos desde Nushell, convirtiendo su salida en una tabla estructurada con `from csv` y guardando los resultados.

4.4.1. Portando la Herramienta a Python (`top-words.py`) y R (`top-words.R`)

En `/data/ch04` disponemos de las implementaciones completas en Python (`top-words.py`) y en R (`top-words.R`). Inspeccionemos primero el archivo `top-words.py` y ejecutémoslo desde Nushell:

```
open --raw top-words.py
open --raw alice.txt | python3 top-words.py 5 | from csv
open --raw alice.txt | python3 top-words.py 5 | from csv | save -f top_words_py.csv
```

Ahora inspeccionemos el script en R (`top-words.R`), ejecutémoslo sobre `alice.txt`, transformemos su salida a una tabla de Nushell y guardémosla en `top_words_r.csv`:

```
open --raw top-words.R
open --raw alice.txt | Rscript top-words.R 5 | from csv
open --raw alice.txt | Rscript top-words.R 5 | from csv | save -f top_words_r.csv
```

4.4.2. Procesando Datos en Transmisión (*Streaming*) desde `stdin` (`fizzbuzz.py` y `fizzbuzz.R`)

Cuando la entrada es un flujo línea por línea, tanto Python como R pueden procesar cada línea conforme llega desde `stdin`. En `/data/ch04` contamos con `fizzbuzz.py` y `fizzbuzz.R`, que implementan el problema clásico de *FizzBuzz* (imprimir `"fizz"` si el número es divisible por 3, `"buzz"` si es divisible por 5, y `"fizzbuzz"` si es divisible por 15):

```
open --raw fizzbuzz.py
1..15 | to text | python3 fizzbuzz.py | lines | wrap resultado_py
1..15 | to text | python3 fizzbuzz.py | lines | wrap resultado_py | save -f fizzbuzz_py.csv
```

Y de manera idéntica con el script `fizzbuzz.R`:

```
open --raw fizzbuzz.R
1..15 | to text | Rscript fizzbuzz.R | lines | wrap resultado_r
1..15 | to text | Rscript fizzbuzz.R | lines | wrap resultado_r | save -f fizzbuzz_r.csv
```

5. Obtención de Datos

Este capítulo aborda el primer paso del modelo OSEMN: la **obtención de datos** (*Obtain*). Los datos pueden provenir de archivos locales, servidores web HTTP/HTTPS, archivos comprimidos (`.tar.gz`, `.zip`), hojas de cálculo de Microsoft Excel (`.xlsx`), bases de datos relacionales SQLite (`.db`) o APIs web REST y de transmisión continua (*streaming*).

Comenzamos ingresando al directorio `/data/ch05`, que ya viene incluido por defecto en el contenedor `dsin`:

```
cd /data/ch05
```

A partir de este punto, todos los comandos del capítulo se ejecutan dentro de `/data/ch05`. Inspeccionemos sus archivos disponibles:

```
ls | sort-by name
```

5.1. Copiando Archivos Locales en el Contenedor Docker

Aunque la imagen `dsin` ya incluye todos los conjuntos de datos del libro en `/data/ch03` a `/data/ch09`, en proyectos propios es frecuente necesitar transferir archivos adicionales entre el computador anfitrión y el contenedor Docker:

- **Montando una carpeta local al iniciar el contenedor:** Ejecutando `docker run --rm -it -v "$(pwd):/data/mis_datos" ghcr.io/kurokirasama/dsin:latest`.
- **Copiando archivos hacia o desde un contenedor en ejecución:** Utilizando `docker cp archivo_local.csv <container_id>:/data/ch05/` desde el sistema anfitrión.

5.2. Descargando desde Internet con `http get`

Nushell incorpora su propio cliente HTTP nativo (`http get`, `http post`, `http head`, `http put`, `http delete`) que reemplaza a herramientas clásicas como `curl` o `wget`. La gran ventaja de `http get` es que, cuando el servidor devuelve contenido estructurado (como JSON), **Nushell lo convierte automáticamente en una tabla o registro navegable**.

5.2.1. Descargando Archivos JSON y Páginas Web

En `/data/ch05` contamos con el archivo `urls.json` y muestras de publicaciones JSON. Cuando consultamos un *endpoint* REST o abrimos `posts_sample.json`, podemos explorar la tabla devuelta y guardarla inmediatamente en disco con `save -f`:

```
let response = (open posts_sample.json)
$response | first 3 | select userId id title
$response | save -f posts.json
open posts.json | first 3 | select id title
```

Si descargamos o leemos un documento HTML (como `MolinosFriesland.html` en `/data/ch05`), podemos inspeccionar sus primeras líneas con `open --raw` y guardar un extracto procesado:

```
open --raw MolinosFriesland.html | lines | first 10
open --raw MolinosFriesland.html | lines | first 10 | to text | save -f molinos_encabezado.txt
```

5.2.2. Descargando desde Múltiples URLs y Consultas `http post`

En `/data/ch05/urls.json` tenemos una lista de *endpoints* bajo la clave `urls`:

```
open urls.json
open urls.json | get urls
open urls.json | get urls | to text | save -f lista_urls.txt
```

Cuando trabajamos con conexión a Internet, podemos recorrer todas las URLs con `each { |u| http get $u }` o enviar cargas útiles (*payloads*) JSON a un servidor mediante `http post`:

```
open urls.json | get urls | each { |u| http get $u } | save -f respuestas_api.json

{
  my_payload: {
    title: "foo"
    body: "bar"
    userId: 1
  }
} | get my_payload | to json | http post --content-type "application/json"
 ↪  https://jsonplaceholder.typicode.com/posts $in
```

5.2.3. Siguiendo Redirecciones e Inspeccionando Cabeceras HTTP (`http head`)

Cuando se accede a una URL acortada (como `bit.ly`) o se depura un servidor web, es útil inspeccionar únicamente las cabeceras HTTP y el código de estado (por ejemplo, `200 OK`, `301 Moved Permanently`, `303 See Other`, `403 Forbidden`, `404 Not Found`) utilizando `http head` o la bandera `--full` de `http get`:

```
http head https://www.wikipedia.org | first 8
```

5.3. Descomprimiendo Archivos

Cuando un conjunto de datos es muy grande o agrupa decenas de archivos pequeños, suele distribuirse comprimido en formatos como `.tar.gz`, `.zip` o `.7z`.

En `/data/ch05` disponemos del archivo comprimido `logs.tar.gz`. Antes de extraerlo a ciegas, inspeccionamos su tabla de contenidos con `tar -tzf`:

```
tar -tzf logs.tar.gz | lines | first 10
tar -tzf logs.tar.gz | lines | save -f indice_logs.txt
```

Como los archivos dentro de `logs.tar.gz` no están contenidos en una subcarpeta única, creamos primero el directorio `logs_extraidos` con `mkdir` y descomprimimos allí usando la opción `-C`, consolidando luego el conteo de líneas de cada archivo en una tabla que también guardamos en `resumen_logs.csv`:

```
rm -rf logs_extraidos
mkdir logs_extraidos
tar -xzf logs.tar.gz -C logs_extraidos
let resumen_logs = (
    ls logs_extraidos
    | each { |f|
        {
            archivo: ($f.name | path basename)
            bytes: $f.size
            lineas: (open --raw $f.name | lines | length)
        }
    }
    | sort-by archivo
)
$resumen_logs | first 5
$resumen_logs | save -f resumen_logs.csv
```

5.4. Conversión de Hojas de Cálculo de Microsoft Excel y Archivos CSV

El formato CSV (*Comma-Separated Values*) almacena registros tabulares separados por saltos de línea y campos delimitados por comas. En `/data/ch05` tenemos tres variantes del conjunto de datos de las Tortugas Ninja (`tmnt-with-header.csv`, `tmnt-basic.csv` y `tmnt-missing-newline.csv`).

En Nushell, `open` detecta automáticamente el encabezado de `tmnt-with-header.csv`:

```
open tmnt-with-header.csv
open tmnt-with-header.csv | save -f tmnt_con_encabezado.json
```

Si un archivo CSV **no posee fila de encabezado** (como `tmnt-missing-newline.csv`), utilizamos `open --raw` combinado con `from csv --noheaders` y asignamos nombres descriptivos con `rename`, guardando la tabla normalizada en `tmnt_normalizado.csv`:

```
open --raw tmnt-missing-newline.csv | from csv --noheaders | rename nombre apodo color arma
open --raw tmnt-missing-newline.csv | from csv --noheaders | rename nombre apodo color arma |
 ↪  save -f tmnt_normalizado.csv
```

Por su parte, para trabajar con hojas de cálculo de **Microsoft Excel (`.xlsx` y `.xls`)**, Nushell no necesita herramientas externas: el comando `open` lee directamente el libro Excel y devuelve un registro donde cada clave es el nombre de una hoja de trabajo. Inspeccionemos `cars.xlsx` en `/data/ch05`:

```
open cars.xlsx | columns
```

Vemos que `cars.xlsx` contiene la hoja `autos`. Extraemos dicha hoja con `get autos`, mostramos sus primeras filas y la exportamos a un archivo CSV (`cars_exportado.csv`):

```
open cars.xlsx | get autos | first 5 | select make "body.style" horsepower "city.mpg" price
open cars.xlsx | get autos | save -f cars_exportado.csv
```

Si deseamos filtrar únicamente los vehículos de la marca `toyota` usando una expresión regular (`^toyota$`), mostramos la tabla resultante y la guardamos en `cars_toyota.csv`:

```
open cars.csv | where make =~ '^toyota$' | select make "body.style" "engine.size" horsepower
 ↪  price | first 8
open cars.csv | where make =~ '^toyota$' | save -f cars_toyota.csv
```

5.5. Consultando Bases de Datos Relacionales (SQLite)

En `/data/ch05` disponemos de la base de datos relacional SQLite `r-datasets.db`, que almacena conjuntos de datos clásicos de R. En Nushell podemos inspeccionar las tablas contenidas en la base de datos directamente con `open r-datasets.db | columns`:

```
open r-datasets.db | columns | first 10
```

Para ejecutar consultas SQL `SELECT` directamente sobre `r-datasets.db`, utilizamos el comando nativo **`query db`**, mostrando el resultado como tabla Nushell y guardándolo en `mtcars_ordenado.csv`:

```
open r-datasets.db | query db "SELECT row_names, mpg, cyl, hp, wt FROM mtcars ORDER BY mpg DESC"
 ↪  | first 8
open r-datasets.db | query db "SELECT row_names, mpg, cyl, hp, wt FROM mtcars ORDER BY mpg DESC"
 ↪  | save -f mtcars_ordenado.csv
```

También podemos realizar agrupaciones SQL directamente con `query db` y guardar el resumen en `mtcars_por_cilindros.csv`:

```
let resumen_cyl = (
    open r-datasets.db
    | query db "SELECT cyl, COUNT(*) AS n_autos, ROUND(AVG(mpg), 2) AS mpg_promedio,
     ↪  ROUND(AVG(hp), 1) AS hp_promedio FROM mtcars GROUP BY cyl ORDER BY cyl"
)
$resumen_cyl
$resumen_cyl | save -f mtcars_por_cilindros.csv
```

5.6. Llamando a las API Web

Las APIs web (*Application Programming Interfaces*) devuelven datos estructurados (generalmente en formato JSON) listos para ser procesados en un pipeline de Nushell.

5.6.1. Autenticación con Clave API

Muchas APIs web (como *News API* o *Google Maps*) requieren una clave de autenticación almacenada de forma segura en un archivo local (por ejemplo, `.secret/newsapi.org_apikey` en `/data/ch05`). En Nushell leemos la clave con `open --raw` y construimos la URL autenticada con `url join`:

```
let api_key = (open --raw .secret/newsapi.org_apikey | str trim)
let news_url = ({
    scheme: "https"
    host: "newsapi.org"
    path: "/v2/everything"
    params: {
        q: "linux"
        apiKey: $api_key
    }
} | url join)
$news_url
$news_url | save -f news_endpoint.txt
```

5.6.2. API de Transmisión (*Streaming*)

Algunas APIs web transmiten eventos en tiempo real de forma continua, como el flujo *EventStreams* de Wikimedia (`https://stream.wikimedia.org/v2/stream/recentchange`). En `/data/ch05` disponemos de una muestra capturada en el archivo `wikimedia-stream-sample`.

En Nushell podemos filtrar las líneas que comienzan con `data:`, quitar el prefijo con `str replace`, convertir cada línea en un registro con `from json` y extraer los títulos editados en la Wikipedia en inglés (`en.wikipedia.org`), mostrando la tabla en pantalla y guardándola en `wikimedia_edits_en.csv`:

```
let edits_en = (
    open --raw wikimedia-stream-sample
    | lines
    | where ($it | str starts-with "data: ")
    | each { |line| $line | str replace "^data: " "" | from json }
    | where ($it.type? == "edit") and ($it.server_name? == "en.wikipedia.org")
    | select -o title user bot timestamp
)
$edits_en | first 10
$edits_en | save -f wikimedia_edits_en.csv
```

5.7. Descargando datos

Nushell tiene su propia herramienta de descarga, similar a `curl`, llamada `http get`.

5.7.1. Descargando un archivo json

Por ejemplo, para descargar un archivo desde internet, utilizamos:

```
http get https://jsonplaceholder.typicode.com/posts | first 5
```

Podemos almacenar el resultado en una variable:

```
let response = http get https://jsonplaceholder.typicode.com/posts
$response
```

Y explorar el resultado con `explore`:

```
$response | first 5 # En sesión interactiva puedes usar: $response | explore
```

O podemos ver directamente los primeros elementos:

```
$response | first 10
```

Tambien podemos guardar el resultado en un archivo:

```
$response | save -f posts.json

ls
open posts.json
```

5.7.2. Descargando desde multiples urls

Supongamos que se están consultando varios *endpoints*, tal vez con diferentes parámetros de consulta y se desea ver todas las respuestas como un único conjunto de datos. Se puede usar `each` para ejecutar comandos nu en cada fila de datos.

Por ejemplo:

```
{
  "urls": [
    "https://jsonplaceholder.typicode.com/posts/1",
    "https://jsonplaceholder.typicode.com/posts/2",
    "https://jsonplaceholder.typicode.com/posts/3"
  ]
} | save -f urls.json

open urls.json
open urls.json | get urls
```

Luego, ejecutamos:

```
open urls.json | get urls | each { |u| http get $u }
```

Si se especifica la opcion `--raw`, el archivo json resultante no se transformará en una tabla:

```
open urls.json | get urls | each { |u| http get $u -r }
```

Para combinar estas respuestas en un formato JSON válido, se puede convertir la tabla a json:

```
open urls.json | get urls | each { |u| http get $u } | to json
```

Notar que el archivo `urls.json` es opcional, pues se puede realizar directamente:

```
{
  "urls": [
    "https://jsonplaceholder.typicode.com/posts/1",
    "https://jsonplaceholder.typicode.com/posts/2",
    "https://jsonplaceholder.typicode.com/posts/3"
  ]
} | get urls | each { |u| http get $u }
```

Tambien se puede realizar consultas `post` con el comando `http post`:

```
{
  "my_payload": {
    "title": "foo",
    "body": "bar",
    "userId": 1
  }
} | get my_payload | to json | http post https://jsonplaceholder.typicode.com/posts $in
```

`$in` indica la salida del comando anterior.

Se puede colocar todo esto en una *pipeline* donde se leen datos, se manipulan y luego se envian de vuelta a la API. En el siguiente ejemplo se descarga una publicacióm, se incrementa el `id`, y se sube de vuelta al *endpoint*. En este ejemplo en particular, el *endpoint* de prueba devuelve una respuesta arbitraria:

```
open urls.json | get urls | first | http get $in | upsert id {|item| $item.id + 1} |
  to json | http post https://jsonplaceholder.typicode.com/posts $in
```

En nushell, también se pueden escribir comandos en varias lineas. El ejemplo anterior queda:

```
open urls.json |
  get urls |
  first |
  http get $in |
```

```
upsert id {|item|
  $item.id + 1
} |
to json |
http post https://jsonplaceholder.typicode.com/posts $in
```

6. Depuración y Transformación de Datos

Este capítulo aborda el segundo paso del modelo OSEMN: **depurar y transformar los datos** (*Scrub*). Una vez obtenidos los datos, rara vez se encuentran listos para la exploración o el modelado: pueden estar en texto plano sin estructura, requerir filtrado por ubicación o patrón, contener inconsistencias o estar repartidos en múltiples archivos CSV, JSON o HTML.

Comenzamos ingresando al directorio `/data/ch06`:

```
cd /data/ch06
```

A partir de este punto, todos los ejemplos de este capítulo se desarrollan dentro de `/data/ch06`. Inspeccionemos sus archivos:

```
ls | sort-by name
```

6.1. Transformaciones en un Pipeline de Datos

Con frecuencia, la salida de una herramienta requiere una transformación intermedia antes de poder alimentar la siguiente etapa del análisis. Tomemos los primeros 100 elementos de la secuencia *FizzBuzz* generados con `fizzbuzz.py` en `/data/ch06`, guardémoslos en `fb.seq`, y luego contemos en Nushell cuántas veces aparecen `fizz`, `buzz` y `fizzbuzz`, mostrando la tabla resultante y guardándola en `fb.csv`:

```
1..100 | to text | python3 fizzbuzz.py | save -f fb.seq
let conteo_fb = (
    open --raw fb.seq
    | lines
    | where ($it =~ "fizz|buzz")
    | uniq -c
    | rename value count
    | sort-by count --reverse
)
$conteo_fb
$conteo_fb | save -f fb.csv
```

6.2. Texto Plano

El texto plano (*plain text*) consiste en secuencias de caracteres legibles por humanos divididas por saltos de línea. En Nushell convertimos cualquier archivo de texto plano en una lista de cadenas mediante `open --raw <archivo> | lines`.

6.2.1. Filtrando Líneas

6.2.1.1. Basado en la Ubicación En `/data/ch06` disponemos del archivo `lines` con 10 líneas numeradas (`Line 1` a `Line 10`). Veamos cómo filtrar líneas según su posición con `first`, `last`, `skip`,

`drop` y `slice`, mostrando los resultados y guardando los subconjuntos en archivos:

```
open --raw lines | lines
```

- **Primeras 3 líneas (`first 3`) y últimas 3 líneas (`last 3`):**

```
open --raw lines | lines | first 3
open --raw lines | lines | last 3
open --raw lines | lines | first 3 | to text | save -f lines_primeras3.txt
```

- **Omitir las primeras 3 líneas (`skip 3`), omitir las últimas 3 (`drop 3`) y extraer las líneas 4, 5 y 6 (`slice 3..5`):**

```
open --raw lines | lines | skip 3
open --raw lines | lines | drop 3
open --raw lines | lines | slice 3..5
open --raw lines | lines | slice 3..5 | to text | save -f lines_4_a_6.txt
```

- **Extraer únicamente las líneas en posiciones impares usando `enumerate` y el operador módulo (`mod`):**

```
open --raw lines | lines | enumerate | where ($it.index mod 2) == 0 | get item
open --raw lines | lines | enumerate | where ($it.index mod 2) == 0 | get item | to text | save
 ↪  -f lines_impares.txt
```

6.2.1.2. Basado en un Patrón Para conservar o descartar líneas según una expresión regular, utilizamos `where ($it =~ "patrón")` (o `!~` para invertir la coincidencia). En `/data/ch06/alice.txt`, extraigamos los encabezados de capítulos que comienzan con `CHAPTER` seguidos de `The`, y contemos el total de líneas no vacías del libro:

```
open --raw alice.txt | lines | where ($it =~ '(?i)^chapter')
open --raw alice.txt | lines | where ($it =~ '^CHAPTER (.*)\. The')
open --raw alice.txt | lines | where ($it =~ '^CHAPTER (.*)\. The') | to text | save -f
 ↪  capitulos_the.txt
open --raw alice.txt | lines | where ($it | str trim | is-not-empty) | length
```

6.2.1.3. Basado en Aleatoriedad (Muestreo) Cuando formulamos un pipeline sobre un archivo muy grande, resulta útil tomar una muestra aleatoria con el comando nativo **`shuffle`** combinado con **`first`**:

```
1..1000 | each { |i| $"Line ($i)" } | shuffle | first 10
1..1000 | each { |i| $"Line ($i)" } | shuffle | first 10 | to text | save -f muestra_10_lineas.txt
```

6.2.2. Extrayendo Valores

Para extraer únicamente el número romano y el título de cada capítulo de `alice.txt` en una tabla estructurada de dos columnas, usamos `parse --regex` con grupos con nombre y guardamos el resultado en `capitulos_alice.csv`:

```
let caps = (
    open --raw alice.txt
    | lines
    | where ($it =~ '^CHAPTER ')
    | parse --regex '^CHAPTER (?P<numero>[IVXLCDM]+)\.\s+(?P<titulo>.*)$'
)
$caps
$caps | save -f capitulos_alice.csv
```

Y si deseamos construir un conjunto de datos con todas las palabras únicas de `alice.txt` que comienzan con la letra `a` y terminan con la letra `e`, mostramos las primeras 15 y guardamos el listado completo en `palabras_a_e.csv`:

```
let palabras_ae = (
    open --raw alice.txt
    | str lowercase
    | parse --regex '(?P<palabra>[a-z]{2,})'
    | get palabra
    | where ($it =~ '^a.*e$')
    | uniq
    | sort
    | wrap palabra
)
$palabras_ae | first 15
$palabras_ae | save -f palabras_a_e.csv
```

6.2.3. Reemplazando y Eliminando Valores

En Nushell, la familia de comandos `str` (`str replace`, `str uppercase`, `str lowercase`, `str trim`) opera nativamente sobre texto **UTF-8**, por lo que convierte correctamente caracteres acentuados y no ASCII (como `'hallo wêreld!'` o `'helló világ'`) donde la herramienta clásica `tr` fallaba por operar sobre bytes individuales:

```
"hello world!" | str replace -a " " "_"
"hello world!" | str replace -a --regex '[ !]' ''
"hallo wêreld!" | str uppercase
"helló világ" | str uppercase
"  hello     world!  " | str trim | str replace -a --regex '\s+' ' ' | str replace "hello" "bye"
"  hello     world!  " | str trim | str replace -a --regex '\s+' ' ' | str replace "hello" "bye"
 ↪  | save -f saludo_limpio.txt
```

6.3. Formateando salida de comandos (parsing)

Nushell ofrece la posibilidad de transformar la salida de comandos nushell, con diferentes formas de lograr el mismo objetivo.

Las funciones integradas que se pueden utilizar incluyen:

- `lines`
- `detect columns`
- `parse`
- `str ...`
- `from ssv`

A continuación, se muestran algunos ejemplos ilustrativos de salida tabular.

6.3.1. `detect columns` (bastante automático)

Esta herramienta intenta dividir automáticamente el texto en varias columnas.

```
df -h

df -h | str replace "Mounted on" Mounted_On | detect columns
```

Para una salida como la de `df` esta es probablemente la forma más compacta de lograr una salida tabular. `str replace` es necesario porque uno de los encabezados de columna tiene un espacio en él.

6.3.2. Usando `from ssv`

También se puede usar el analizador de datos `from` incorporado para `ssv` (*s*pace *s*eparated *v*alues):

```
df -h | str replace "Mounted on" Mounted_On | from ssv --aligned-columns --minimum-spaces 1
```

La salida es idéntica al ejemplo anterior. `from ssv` admite varias opciones para ajustar su comportamiento. Notar que de todas formas se debe corregir los encabezados de las columnas si contienen espacios inesperados.

6.3.3. Usando `parse`

Permite analizar un patrón arbitrario de una cadena de texto en una tabla de varias columnas.

```
'nu = "0.116.1" # A new type of shell
ttt = "1.8.0" # Terminal text editor' | lines | parse '{crate_name} = "{version}" #{description}'
 ↪  | update description { str trim }
```

6.4. CSV y Tablas Estructuradas

6.4.1. Cuerpos, Encabezados y Columnas

En las herramientas clásicas de texto (`grep`, `sed`, `tr`, `sort`), filtrar u ordenar un archivo CSV requería utilidades auxiliares (`body`, `header`, `cols`) para evitar mezclar el encabezado con los datos o modificar columnas equivocadas.

En **Nushell**, al abrir un archivo CSV con `open`, el encabezado se convierte automáticamente en el esquema de nombres de columnas de la tabla, y cualquier operación (`sort-by`, `where`, `update`) actúa exclusivamente sobre las filas o la columna indicada sin corromper jamás el encabezado.

Por ejemplo, en `/data/ch06`, convirtamos a mayúsculas únicamente la columna `day` de `tips.csv`, mostrando las primeras 5 filas y guardando la tabla resultante en `tips_day_upper.csv`:

```
open tips.csv | update day { str uppercase } | first 5
open tips.csv | update day { str uppercase } | save -f tips_day_upper.csv
```

6.4.2. Realizando Consultas SQL en CSV

Cuando deseamos expresar transformaciones complejas utilizando sintaxis **SQL** sobre archivos CSV, Nushell permite volcar cualquier tabla directamente a una base de datos SQLite con `into sqlite` y consultarla con `query db`. Veamos cómo consultar `tips.csv` con SQL y guardar el resultado en `tips_sql_resultado.csv`:

```
rm -f tips_temp.db
open tips.csv | into sqlite tips_temp.db
open tips_temp.db | query db "SELECT bill, tip, day, time, size FROM main WHERE bill > 40.0 AND
↪  day LIKE 'S%'"
open tips_temp.db | query db "SELECT bill, tip, day, time, size FROM main WHERE bill > 40.0 AND
↪  day LIKE 'S%'" | save -f tips_sql_resultado.csv
```

6.4.3. Extrayendo y Reordenando Columnas

Para extraer un subconjunto de columnas en un orden específico utilizamos `select`, y para excluir una o más columnas conservando el resto utilizamos `reject`:

```
open iris.csv | select sepal_length petal_length sepal_width petal_width | first 5
open iris.csv | reject species | first 5
open iris.csv | select sepal_length petal_length sepal_width petal_width | save -f
↪  iris_columnas_reordenadas.csv
```

6.4.4. Filtrando Filas

En Nushell filtramos filas de tablas CSV combinando condiciones numéricas y expresiones regulares legibles mediante `where`:

```
open tips.csv | where size >= 5 | first 6
open tips.csv | where bill > 40.0 and day =~ '^S'
open tips.csv | where bill > 40.0 and day =~ '^S' | save -f tips_altos_finde.csv
```

6.4.5. Combinar Columnas (y Robustez ante Comas Internas)

Consideremos los archivos `names.csv` y `names-comma.csv` en `/data/ch06`, los cuales contienen compositores cuyos nombres (`first_name`) y apellidos (`last_name`) deseamos combinar en una columna `full_name`:

```
open names.csv
open names-comma.csv
```

En `names-comma.csv`, el último registro contiene una coma dentro del apellido (`"Stravinsky, Jr."`). Mientras que `sed`, `awk` y `cut` fallaban al partir por esa coma interna, el analizador CSV de Nushell respeta las comillas del estándar RFC 4180 y construye `full_name` de manera impecable:

```
let nombres_limpios = (
    open names-comma.csv
    | insert full_name { |row| $"($row.first_name) ($row.last_name)" }
    | select id full_name born
)
$nombres_limpios
$nombres_limpios | save -f names_completos.csv
```

6.4.6. Combinando Múltiples Archivos CSV

6.4.6.1. Concatenar Horizontalmente (`merge`) En `/data/ch06` podemos dividir `tips.csv` en tres archivos CSV con distintas columnas (`bills.csv`, `datetime.csv` y `customers.csv`) y luego volver a unirlos horizontalmente lado a lado mediante el comando **`merge`**, guardando el resultado en `tips_horizontal.csv`:

```
open tips.csv | select bill tip | save -f bills.csv
open tips.csv | select day time | save -f datetime.csv
open tips.csv | select sex smoker size | save -f customers.csv

let unido_horizontal = (open bills.csv | merge (open customers.csv) | merge (open datetime.csv))
$unido_horizontal | first 5
$unido_horizontal | save -f tips_horizontal.csv
```

6.4.6.2. Unión Relacional (`join`) Cuando dos tablas comparten una columna clave (como `species` en `iris.csv` e `irismeta.csv`), realizamos un cruce relacional (*inner join*) con el comando **`join`**, seleccionamos las columnas de interés y guardamos el resultado en `iris_usda.csv`:

```
open irismeta.csv
let iris_enriquecido = (
    open iris.csv
    | join (open irismeta.csv) species
    | select sepal_length sepal_width species usda_id
)
$iris_enriquecido | slice 48..53
$iris_enriquecido | save -f iris_usda.csv
```

6.5. Trabajando con XML/HTML y JSON

En `/data/ch06` contamos con el archivo JSON `users.json` y los archivos extraídos de Wikipedia `wiki.html`, `table.html` y `table.json`.

En lugar de modificar `users.json` con expresiones regulares de texto como `sed` (lo cual podría alterar valores además de nombres de claves), en Nushell abrimos `users.json` como estructura de datos, renombramos el campo `gender` a `sex` en la tabla `results` y guardamos el resultado tanto en JSON como en CSV:

```
let usuarios = (
    open users.json
    | get results
    | select gender email nat phone
    | rename sex email nat phone
)
$usuarios | first 5
$usuarios | save -f users_sex.csv
```

De igual manera, a partir de la tabla HTML convertida a JSON en `table.json` (lista de países y territorios por relación frontera/superficie de Wikipedia), podemos extraer las filas en Nushell y guardar el conjunto de datos tabular en `countries_from_json.csv`:

```
open countries.csv | first 10
open countries.csv | first 10 | save -f countries_top10.csv
```

6.6. El plugin polars de Nushell

Polars se define como una biblioteca de `DataFrame` para Rust que se caracteriza por su extrema rapidez y eficiencia en el uso de la memoria. El plugin `polars` para Nushell permite utilizar las potentes capacidades de Polars directamente desde la línea de comandos. Esto abre un mundo de posibilidades para el análisis y la manipulación de datos a alta velocidad, directamente en la terminal.

La filosofía de Polars se centra en aprovechar al máximo el hardware moderno, utilizando un motor de consulta paralelizado y optimizado para la ejecución en múltiples núcleos. Esto significa que se

pueden procesar conjuntos de datos mucho más grandes que con las herramientas tradicionales de línea de comandos, y a menudo de forma mucho más rápida que con soluciones en memoria como Pandas.

6.6.1. Casos de uso

El plugin `polars` resulta ideal para:

- **Análisis exploratorio de datos (EDA)**: se pueden investigar y resumir rápidamente grandes conjuntos de datos.
- **Canalizaciones de limpieza y transformación de datos**: se pueden crear potentes canalizaciones para limpiar y transformar datos.
- **Procesamiento de datos por lotes**: se pueden procesar grandes volúmenes de datos de manera eficiente.
- **Alternativa a `pandas` o `dplyr` en la línea de comandos**: se pueden realizar operaciones complejas de `DataFrames` sin salir de la terminal.

6.6.2. Conceptos básicos

Los dos conceptos principales en Polars son el `DataFrame` y la `Serie`.

- **`DataFrame`**: se define como una estructura de datos tabular bidimensional, similar a una hoja de cálculo o una tabla SQL. Consta de filas y columnas, donde cada columna es una `Serie`.
- **`Serie`**: se describe como una matriz unidimensional de datos, que representa una única columna en un `DataFrame`.

6.6.2.1. Diferencia entre DataFrames de Polars y Tablas de Nushell Es importante entender la diferencia entre un `DataFrame` de Polars y una tabla de Nushell. Aunque a primera vista pueden parecer similares, se observan diferencias fundamentales en su diseño y uso:

- **Representación en Memoria:** Las tablas de Nushell se representan como una lista de filas, donde cada fila es una colección de pares clave-valor. Esto resulta ideal para la mayoría de las tareas de la línea de comandos. Los `DataFrames` de Polars, por otro lado, utilizan un formato columnar. Esto significa que los datos de cada columna se almacenan juntos en memoria, lo que permite un procesamiento y análisis de datos mucho más rápido, especialmente para conjuntos de datos grandes.
- **API y Operaciones:** Las tablas de Nushell se manipulan utilizando los comandos estándar de Nushell como `where`, `sort-by`, `group-by`, etc. Los `DataFrames` de Polars tienen su propio conjunto de comandos (los que comienzan con `polars`), que están optimizados para operaciones de datos a gran escala.
- **Rendimiento:** Para conjuntos de datos pequeños y medianos, el rendimiento de las tablas de Nushell es excelente. Sin embargo, para conjuntos de datos muy grandes, los `DataFrames` de Polars ofrecen un rendimiento significativamente mayor debido a su diseño columnar y su capacidad para realizar operaciones en paralelo.

En resumen, se puede pensar en las tablas de Nushell como una forma versátil y fácil de usar para trabajar con datos estructurados en la línea de comandos, mientras que los `DataFrames` de Polars son una herramienta más especializada y de alto rendimiento para el análisis de datos a gran escala. El plugin `polars` permite cambiar sin problemas entre estos dos paradigmas, brindando lo mejor de ambos mundos.

6.6.3. Uso básico

Para utilizar el plugin `polars`, primero se deben cargar los datos en un `DataFrame`. El plugin `polars` admite una variedad de formatos de archivo, incluidos CSV, JSON, Parquet y más.

6.6.3.1. Carga de datos Para este ejemplo, se utilizará un conjunto de datos de ventas de videojuegos. Se pueden descargar los datos usando el siguiente comando:

```
ls vgsales.csv
```

> Fuente: https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2019/2019-07-30/video_games.csv

Ahora, se puede cargar el archivo `vgsales.csv` en un DataFrame de Polars de la siguiente manera:

```
let df = (open vgsales.csv | polars into-df)
$df | polars first 5 | polars collect
$df | describe
```

Esto crea un `DataFrame` de Polars que puede usarse para su posterior análisis.

> En caso de error con `polars open`, se puede abrir el archivo usando el comando `open vgsales.csv | polars into-df`.

6.6.3.2. Exploración de datos Una vez que se han cargado los datos en un `DataFrame`, se puede ver su esquema usando el comando `polars schema`. Esto mostrará los nombres de las columnas y sus tipos de datos.

```
$df | polars schema
```

Se observa que algunas variables aparecen de tipo `object`, que es el tipo de datos predeterminado para Polars cuando no se especifica un tipo de datos específico. En estos casos, se puede especificar el tipo de datos adecuado usando el comando `polars into-df` con la opción `--schema`.

```
let df = (open vgsales.csv)
  | polars into-df --schema {
      number: i64,
      game: str,
      release_date: str,
      price: str,
      owners: str,
```

```
      developer: str,
      publisher: str,
      average_playtime: str,
      median_playtime: str,
      metascore: str
    }
```

Cuando hay valores `NA`, se definen temporalmente las variables numéricas de tipo `str`.

6.6.3.3. Conversión de tipos de datos Para poder abrir el archivo, se tuvo que usar el tipo `str` para las columnas `number`, `price`, `owners`, `average_playtime`, `median_playtime`, y `metascore`.

Para convertir estas columnas al tipo apropiado se utilizan:

```
let df = $df
  | collect
  | polars with-column [
    (polars col price
        | polars str-replace-all --pattern "Free" --replace "0"
        | polars cast f64
    ),
    (polars col average_playtime | polars cast i64),
    (polars col median_playtime | polars cast i64),
    (polars col metascore | polars cast i64)
  ]

$df | polars schema
$df | polars summary | polars collect
$df | polars summary | polars select descriptor "price (f64)" "average_playtime (i64)"
 ↪  "median_playtime (i64)" "metascore (i64)" | polars collect

$df | polars first 5 | polars collect
```

En este proceso:

- `collect`: Recolecta un flujo (*stream*) en un valor.
- `polars with-column`: Añade una o varias series/expresiones a un `DataFrame` o `LazyFrame`.
- `polars col`: Crea una expresión que referencia una columna existente (p. ej. `polars col price`).
- `polars str-replace-all`: Reemplaza todas las ocurrencias que casan con un patrón regex por una cadena dada.
- `polars cast`: Convierte (castea) el tipo de una expresión o columna a un dtype especificado (p. ej. `f64`, `i64`).

Se debe notar que se tuvo que limpiar la columna `price` antes de poder convertirla a un número.

La columna `price` original contenía la palabra "**Free**" (Gratis) en algunas filas. No se puede convertir (hacer `cast`) directamente una columna que contiene texto (como "Free") a un tipo numérico (como `f64`). Intentar hacerlo fallaría porque el computador no sabe qué número corresponde a "Free".

Para arreglar esto, se usa un proceso de dos pasos dentro del comando `with-column`:

1. `polars str-replace-all --pattern "Free" --replace "0"`: Este comando encontró cada instancia del texto "Free" en la columna `price` y la reemplazó por el texto "0".
2. `polars cast f64`: *Después* de que todos los valores "Free" fueran reemplazados por "0", la columna solo contenía textos que parecían números (ej. "1.99", "59.99", y ahora "0"). En ese momento, ya era seguro convertir toda la columna a `f64` (un número de punto flotante de 64 bits) sin errores.

6.6.4. Manipulación de datos

El plugin `polars` proporciona una amplia gama de comandos para manipular `DataFrames`.

6.6.4.1. Selección de columnas Se pueden seleccionar columnas de un `DataFrame` usando el comando `polars select`. Por ejemplo, para seleccionar las columnas `game`, `price`, y `metascore` del `DataFrame` de ventas de videojuegos, se usaría el siguiente comando:

```
$df | polars select game price metascore
```

Se observa que se obtiene lo siguiente:

```
╭────────────────┬──────────────────────────────────────────────────────────╮
│ plan           │ SELECT [col("game"), col("price"), col("metascore")]     │
│                │   DF ["number", "game", "release_date", "price",         │
│                │   ...]; PROJECT */10 COLUMNS                             │
│ optimized_plan │ DF ["number", "game", "release_date", "price", ...];     │
│                │   PROJECT["game", "price", "metascore"] 3/10 COLUMNS     │
╰────────────────┴──────────────────────────────────────────────────────────╯
```

Este resultado es el **plan de consulta** de Polars. Es una mirada interna a cómo el motor de Polars procesará la solicitud. Se divide en dos partes:

- **`plan` (Plan Lógico):** Esta es la traducción directa del comando a una serie de operaciones. En este caso, `SELECT [col("game"), col("price"), col("metascore")]` indica que se ha solicitado seleccionar esas tres columnas.
- **`optimized_plan` (Plan Optimizado):** Este es el plan de ejecución real que Polars utilizará. El motor de Polars analiza el plan lógico y lo optimiza para obtener la máxima eficiencia. En este ejemplo, el plan optimizado muestra que realizará una operación de "proyección" para seleccionar solo las 3 columnas que se solicitaron de las 10 columnas totales del DataFrame.

Esta es una característica clave de la **evaluación perezosa (lazy evaluation)** de Polars. En lugar de ejecutar cada comando de inmediato, se construye un plan de todas las operaciones, se

optimiza y luego se ejecuta de la manera más eficiente posible. Esto es lo que hace que Polars sea tan rápido, especialmente con grandes conjuntos de datos.

Para ver el resultado, se debe terminar la *pipeline* con `polars collect`. Esto ejecutará el plan optimizado y devolverá el resultado como un DataFrame.

```
$df | polars select game price metascore | polars collect
```

6.6.4.2. Filtrado de filas Se pueden filtrar filas de un `DataFrame` usando el comando `polars filter`. Por ejemplo, para filtrar el `DataFrame` de ventas de videojuegos para incluir solo los juegos cuyo nombre de desarrollador (*developer*) contiene la palabra "Action", se usaría el siguiente comando:

```
#búsqueda Insensible a Mayúsculas/Minúsculas
let filter_condition = polars col developer | polars contains "(?i)Action"
$df | polars filter $filter_condition | polars collect

#búsqueda Sensible a Mayúsculas/Minúsculas
let filter_condition = polars col developer | polars contains "Action"
$df | polars filter $filter_condition | polars collect

#eliminar na in price
let filter_condition = polars col price | polars is-not-null
$df | polars filter $filter_condition | polars collect
```

Por otro lado, para eliminar todas las filas con valores nulos en todas las columnas, se usaría el siguiente comando:

```
let df_filtered = $df | polars drop-nulls
$df_filtered | polars collect
```

6.6.4.3. Ordenación de datos Se puede ordenar un `DataFrame` usando el comando `polars sort-by`. Por ejemplo, para ordenar el `DataFrame` de ventas de videojuegos por la columna `price` en orden descendente, se usaría el siguiente comando:

```
$df_filtered | polars sort-by price --reverse [true]
```

6.6.4.4. Agrupación y agregación Se pueden agrupar y agregar datos en un `DataFrame` usando los comandos `group-by` y `agg`. Por ejemplo, para agrupar el `DataFrame` de ventas de videojuegos por la columna `developer` y calcular el número total de juegos por cada desarrollador, se usaría el siguiente comando:

```
$df_filtered |
  polars group-by developer |
  polars agg (polars col game | polars count) |
```

```
  polars sort-by game --reverse [true] |
  polars collect
```

- Para estadísticas por `developer`:

```
$df_filtered |
  polars group-by developer |
  polars agg [
    (polars col price | polars mean | polars as "price_mean"),
    (polars col game  | polars count | polars as "n_games")
  ] |
  polars sort-by developer |
  polars collect
```

- Para estadísticas por año de `release_date`:

```
#se mira la columna release_date
$df_filtered | polars select release_date | polars collect

#primero se extrae año, luego se agrupa
let data_by_year = ($df_filtered |
  polars with-column [
    (polars col release_date
       | polars str-replace-all --pattern ".*,\\s*(\\d{4})$" --replace "$1" # extrae el año
       | polars str-replace-all --pattern "^\\s+|\\s+$" --replace ""        # trim
       | polars cast i64
       | polars as "year")
   ] |
  polars group-by year |
  polars agg [
    (polars col price | polars mean | polars as "price_mean"),
    (polars col game  | polars count | polars as "n_games")
  ] |
  polars sort-by year |
  polars collect
)

#se visualiza
$data_by_year
$data_by_year | describe
$data_by_year | polars into-nu #tabla nu
$data_by_year | polars into-nu | describe
```

6.6.5. Guardado y formatos

Para guardar un `DataFrame` en un archivo, se usaría el comando `polars save`. Por ejemplo, para guardar el `DataFrame` de ventas de videojuegos filtrado en un archivo CSV, se usaría el siguiente comando:

```
$df_filtered | polars collect | polars save vgsales_filtered.parquet
$df_filtered | polars collect | polars save vgsales_filtered.csv
$data_by_year | polars save data_by_year.csv
```

Si lo anterior no funciona, en algunos casos es necesario entregar la ruta completa del archivo. Por ejemplo:

```
$df_filtered | polars collect | polars save ($env.PWD | path join vgsales_filtered.csv)
```

```
$df_filtered | polars collect | polars save vgsales_filtered.csv
$data_by_year | polars save data_by_year.csv
```

6.6.6. Pivoteo de columnas

El comando `polars pivot` en Nushell es una operación de transformación de datos que convierte un DataFrame del formato largo al formato ancho, similar a la función PIVOT en SQL o la función `pivot_wider` en R. Esta operación es fundamental para reorganizar datos tabulares, permitiendo que valores únicos de una columna se conviertan en nuevas columnas, mientras que los valores de otra columna se distribuyen como datos en estas nuevas columnas. Es especialmente útil para crear tablas de resumen, reportes y visualizaciones donde se necesita mostrar múltiples métricas organizadas por categorías específicas.

Dentro de sus opciones principales se encuentran:

- **`--index` (`-i`)**: Define las columnas que servirán como identificadores únicos de las filas en el resultado. Estas columnas mantienen su estructura original y no se transforman, funcionando como las claves que identifican cada fila en el formato ancho resultante.
- **`--on` (`-o`)**: Especifica la columna cuyos valores únicos se convertirán en los nombres de las nuevas columnas en el DataFrame resultante. Cada valor distinto en esta columna generará una nueva columna en el formato ancho.
- **`--values` (`-v`)**: Determina las columnas cuyos valores se distribuirán en las nuevas columnas creadas a partir de los valores únicos de la columna especificada con `--on`. Estas son las columnas que contienen los datos numéricos o categóricos que se desean reorganizar.

El comando también incluye opciones adicionales como `--aggregate` para definir funciones de agregación cuando hay valores duplicados, `--separator` para personalizar el delimitador en nombres de columnas generadas, y `--sort` para ordenar las columnas resultantes.

Para ilustración, se considera el siguiente ejemplo.

Primero, se preparan los datos para el pivoteo:

- Se repite el procesamiento anterior a `release_date`.
- Se extraen columnas `developer`, `price`, `year`.
- Se agrupa por `developer` y `year`.
- Se calcula promedio de `price` de acuerdo al agrupamiento.

```
let df_aggregated = ($df_filtered |
  polars with-column [
    (polars col release_date
      | polars str-replace-all --pattern ".*,\\s*(\\d{4})$" --replace "$1"
      | polars str-replace-all --pattern "^\\s+|\\s+$" --replace ""
      | polars cast i64
      | polars as "year"
    )
  ] |
  polars select [
    (polars col developer)
    (polars col price)
    (polars col year)
  ] |
  polars group-by [developer year] |
  polars agg [(polars col price | polars mean | polars as price_mean)]
)

$df_aggregated | polars collect
$df_aggregated | polars sort-by year | polars collect
$df_aggregated | polars sort-by developer | polars collect
```

Se hace el pivoteo:

```
let df_pivoted = ($df_aggregated | polars collect | polars pivot --index [developer] --on [year]
 ↪  --values [price_mean] --stable | polars collect | polars sort-by developer | polars collect)
$df_pivoted | polars first 5 | polars into-nu
$df_pivoted | polars into-nu | save -f vgsales_pivot.csv
```

6.6.7. Buenas prácticas y recomendaciones

- Siempre se deben nombrar explícitamente las columnas creadas con `polars as` dentro de expresiones complejas.
- Se deben limpiar los formatos de moneda y separadores antes de castear a numérico.
- Se debe llamar `collect` o `save` al final de una *pipeline*.
- Se prefieren formatos columnares (Parquet, Arrow) para almacenamiento intermedio o persistente.
- Se debe revisar el `schema` y `summary` con frecuencia para detectar valores atípicos y NA.
- Para desarrollos iterativos, usar `polars collect | polars into-nu` permite inspeccionar

resultados en formato de tabla de Nushell y combinar con otras utilidades de la shell.

6.6.8. Conclusión

El plugin `polars` para Nushell combina la expresividad y conveniencia de la shell con el rendimiento de Polars. Permite construir pipelines de limpieza, transformación, agregación y guardado de datos de manera eficiente. Usando estrategias sencillas; limpieza de cadenas, uso de `polars as` para nombrar columnas, evaluación perezosa (lazyframes) y formatos columnares; se pueden crear flujos reproducibles y escalables directamente desde la terminal.

7. Exploración y Visualización de Datos

El cuarto paso del flujo de trabajo OSEMN consiste en explorar los datos. Explorar nuestros conjuntos de datos desde múltiples ángulos (estadístico, estructural y visual) es fundamental antes de construir modelos predictivos. Como destacó John W. Tukey en su clásico libro *Exploratory Data Analysis* (1977), el análisis exploratorio de datos (EDA) nos permite descubrir patrones inesperados, detectar valores atípicos (*outliers*), verificar supuestos y formular hipótesis informadas.

En este capítulo abordaremos la exploración y visualización de datos desde tres perspectivas complementarias dentro de la imagen Docker `dsin`:

1. **Inspección de propiedades estructurales y estadísticas en Nushell**, utilizando los comandos nativos de inspección (`columns`, `describe`, `uniq`, `histogram`), el módulo matemático (`math`) y el motor de DataFrames `polars`.
2. **Cálculo de estadísticas descriptivas y pruebas diagnósticas mediante scripts en R y Python**, integrando sus salidas directamente en pipelines de Nushell para presentarlas como tablas estructuradas.
3. **Creación y visualización de gráficos estadísticos en la terminal**, generando figuras mediante scripts modulares con `ggplot2` en R y `matplotlib`/`seaborn` en Python, e inspeccionándolas directamente dentro de la terminal mediante el visor gráfico **`timg`** preinstalado en la imagen `dsin`.

Todos los archivos y scripts de este capítulo se encuentran en `/data/ch07`. Para comenzar, nos posicionamos en este directorio:

```
cd /data/ch07
```

A partir de este punto, ejecutaremos todas las actividades dentro de `/data/ch07`. Inspeccionemos los archivos disponibles:

```
ls
```

7.1. Inspeccionando Datos y sus Propiedades

Antes de graficar o modelar un conjunto de datos, necesitamos responder preguntas fundamentales sobre su estructura: ¿cuántas filas y columnas tiene?, ¿qué nombres y tipos de datos poseen las variables?, ¿existen columnas constantes o con valores faltantes?

En Nushell, dado que cualquier archivo tabular cargado con `open` se convierte inmediatamente en una tabla tipada en memoria, podemos responder estas preguntas combinando selectores y filtros nativos, y guardar los resúmenes o tablas depuradas directamente en disco.

7.1.1. ¿Tiene Encabezado, Cuántos Registros y Qué Columnas?

Inspeccionemos el archivo `venture.csv` ubicado en `/data/ch07`:

```
open --raw venture.csv | lines | first 5
```

Al abrirlo de forma estructurada con `open venture.csv`, Nushell interpreta automáticamente la primera línea como encabezado y tipifica cada columna. Podemos consultar el número total de registros con `length` y listar los nombres de las columnas junto con su índice usando `columns` y `enumerate`:

```
open venture.csv | length
open venture.csv | columns | enumerate | first 15
open venture.csv | columns | enumerate | wrap col_info | flatten | save -f venture_columns.csv
```

7.1.2. Inspeccionando Tipos de Datos e Inferencia de Esquemas

Veamos el archivo `datatypes.csv`, diseñado específicamente para ilustrar distintos tipos de datos (enteros, flotantes, booleanos, fechas y cadenas):

```
open datatypes.csv
open datatypes.csv | save -f datatypes_copy.json
```

Para conocer el tipo exacto que Nushell o Polars asignó a cada columna, podemos emplear `describe` o convertir la tabla a un DataFrame de Polars y consultar `polars schema`:

```
open datatypes.csv | polars into-df | polars schema | transpose column dtype
open datatypes.csv | polars into-df | polars schema | transpose column dtype | save -f
↪  datatypes_schema.csv
```

7.1.3. Identificando Valores Únicos y Eliminando Columnas Constantes

En muchos conjuntos de datos administrativos o financieros (como `venture.csv`), existen columnas que contienen exactamente el mismo valor en todas las filas y que, por tanto, no aportan información estadística ni poder predictivo.

En Nushell podemos detectar el número de valores únicos de cada columna trasponiendo el primer registro o iterando sobre los nombres de las columnas con `get` y `uniq`:

```
let df = (open venture.csv)
let non_const_cols = ($df | columns | where {|c| ($df | get $c | uniq | length) > 1 })
$df | select ...$non_const_cols | first 5
$df | select ...$non_const_cols | save -f venture_clean.csv
```

Observa la claridad del flujo: identificamos programáticamente todas las columnas con más de un valor único (`uniq | length > 1`), filtramos la tabla original conservando únicamente dichas columnas mediante el operador de expansión `...$non_const_cols`, y guardamos el resultado limpio en `venture_clean.csv`.

7.2. Calculando Estadísticas Descriptivas

Trabajemos ahora con el clásico conjunto de datos `tips.csv` (propinas en un restaurante según el total de la cuenta, sexo, hábito de fumar, día, horario y tamaño del grupo), ubicado en `/data/ch07/tips.csv`:

```
open tips.csv | first 10
open tips.csv | first 10 | save -f tips_sample.csv
```

7.2.1. Estadísticas con `math` y `polars summary` en Nushell

Nushell incluye la familia de comandos `math` (`math sum`, `math avg`, `math median`, `math stddev`, `math variance`, `math min`, `math max`, `math mode`), que operan sobre listas numéricas o sobre todas las columnas numéricas de una tabla:

```
open tips.csv | select bill tip size | math avg
open tips.csv | select bill tip size | math stddev
open tips.csv | select bill tip size | math median | save -f tips_median.json
```

Si deseamos obtener simultáneamente el conteo, media, desviación estándar, mínimo, cuartiles (25 %, 50 %, 75 %) y máximo de todas las variables del dataset, `polars summary` genera una tabla resumen completa en un solo paso:

```
open tips.csv | polars into-df | polars summary | polars collect | polars into-nu
open tips.csv | polars into-df | polars summary | polars collect | polars into-nu | save -f
 ↪  tips_summary.csv
```

Para variables categóricas (como `day` o `time`), el comando nativo `histogram` calcula el conteo absoluto, cuantiles y porcentaje relativo, incluyendo además un gráfico de barras ASCII en la propia tabla:

```
open tips.csv | histogram day
open tips.csv | histogram day | save -f tips_day_freq.csv
```

7.2.2. Transformaciones y Resúmenes Estadísticos con Scripts en R y Python

En un flujo de trabajo políglota, frecuentemente deseamos aplicar funciones estadísticas especializadas de R (`dplyr`) o Python (`pandas` / `scipy`) y capturar el resultado de vuelta en Nushell como una tabla estructurada. En lugar de escribir código difícil de mantener en una sola línea, creamos archivos `.R` y `.py` modulares en nuestro directorio de trabajo y los invocamos desde Nushell.

Por ejemplo, revisemos el script `mutate_tips.R` en `/data/ch07`, que calcula el porcentaje de propina (`percent = tip / bill * 100`) usando `dplyr::mutate` y escribe el CSV resultante en la salida estándar:

```
open --raw mutate_tips.R
```

Al ejecutar `Rscript mutate_tips.R` y canalizar su salida hacia `from csv`, Nushell recibe una tabla tipada que podemos filtrar, ordenar y guardar en disco:

```
Rscript mutate_tips.R | from csv | first 10
Rscript mutate_tips.R | from csv | save -f tips_with_percent.csv
```

Del mismo modo, el script `summary_tips.R` agrupa las propinas por día y horario (`group_by(day, time)`) y calcula el conteo, la cuenta media y la propina media:

```
open --raw summary_tips.R
Rscript summary_tips.R | from csv | sort-by mean_tip -r
Rscript summary_tips.R | from csv | sort-by mean_tip -r | save -f tips_grouped_summary.csv
```

Y con Python (`corr_tips.py`), podemos calcular la matriz de correlación de Pearson entre `bill`, `tip` y `size` usando `pandas` y devolverla como tabla a Nushell:

```
open --raw corr_tips.py
python3 corr_tips.py | from csv
python3 corr_tips.py | from csv | save -f tips_correlations.csv
```

7.3. Creando Visualizaciones Estadísticas desde la Línea de Comandos

Las estadísticas resumen nunca sustituyen por completo a la inspección visual (como demuestra el célebre *Cuarteto de Anscombe*, donde cuatro conjuntos de datos con idéntica media, varianza y correlación exhiben estructuras geométricas radicalmente distintas).

En nuestro entorno `dsin`, combinamos la potencia gráfica de **`ggplot2`** (en R) y **`matplotlib`/`seaborn`** (en Python) con el visor de imágenes de terminal **`timg`**. Como tanto `Rscript`, `python3` y `timg` están configurados en el `PATH` por defecto del contenedor `dsin`, podemos invocarlos directamente por su nombre sin prefijos especiales.

Veamos primero el script `plot_faceted.R`, que genera un gráfico de dispersión de `tip` versus `bill` con rectas de regresión lineal separadas por sexo y día (`tips_faceted.png`):

```
open --raw plot_faceted.R
Rscript plot_faceted.R
ls tips_faceted.png
```

7.3.1. Visualizando Imágenes Directamente en la Terminal con `timg`

En el flujo de trabajo tradicional de los contenedores Docker sin servidor gráfico X11, el usuario debía copiar cada imagen generada hacia el sistema anfitrión para poder abrirla. En la imagen `dsin`, incluimos **`timg`** (*Terminal Image and Video Viewer*), una herramienta que renderiza archivos

PNG, JPEG, SVG o incluso animaciones directamente dentro de la terminal utilizando bloques Unicode de alta resolución o los protocolos gráficos de Kitty/iTerm2/Sixel.

Para visualizar cualquier gráfico generado en `/data/ch07`, basta con ejecutar:

```
timg -g 80x25 tips_faceted.png
```

También podemos mostrar varios gráficos en una cuadrícula comparativa dentro de la terminal usando la bandera `--grid`:

```
Rscript plot_histogram.R
Rscript plot_density.R
timg --grid=2x1 -g 80x25 tips_histogram.png tips_density.png
```

7.3.2. Alternativa Remota: Copiando Gráficos con `scp`

Si te encuentras conectado por SSH desde una terminal antigua que no soporta color verdadero de 24 bits, siempre puedes descargar la imagen hacia tu máquina local utilizando `scp` (*secure copy*):

```
scp usuario@servidor:/data/ch07/tips_faceted.png .
```

7.4. Catálogo de Gráficos Estadísticos con Scripts en R y Python

La gramática de los gráficos (*Grammar of Graphics*) implementada en `ggplot2` descompone toda visualización estadística en tres elementos esenciales:

1. **Datos (`data`)**: el DataFrame tabular que deseamos representar.
2. **Mapeos estéticos (`aes`)**: la correspondencia entre columnas de los datos y propiedades visuales como posición (`x`, `y`), color (`color`, `fill`), forma (`shape`) o tamaño (`size`).
3. **Objetos geométricos (`geom_*`)**: la representación gráfica concreta (barras, puntos, líneas, cajas, violines o curvas de densidad).

En `/data/ch07` disponemos de una colección completa de scripts listos para usar que ilustran los gráficos fundamentales del análisis exploratorio.

7.4.1. Gráfico de Barras (`geom_bar`)

Para visualizar la distribución de frecuencias de una variable categórica o discreta, como la cantidad de mesas atendidas por día de la semana en `tips.csv`, ejecutamos `plot_bar.R`:

```
open --raw plot_bar.R
Rscript plot_bar.R
timg -g 80x25 tips_bar.png
```

Gráfico de barras generado con ggplot2

Figura 7: Gráfico de barras generado con ggplot2

7.4.2. Histograma (`geom_histogram`)

Para inspeccionar la distribución empírica de una variable continua como el monto total de la cuenta (`bill`), ejecutamos `plot_histogram.R`:

```
open --raw plot_histogram.R
Rscript plot_histogram.R
timg -g 80x25 tips_histogram.png
```

Histograma del total de la cuenta

Figura 8: Histograma del total de la cuenta

Ajustar el número de intervalos (`bins`) es crucial para evitar tanto el sobre-suavizado (pocas barras que ocultan multimodalidad) como el ruido excesivo (demasiadas barras estrechas).

7.4.3. Gráfico de Densidad (`geom_density`)

Una alternativa suave al histograma es la estimación de densidad por núcleos (*Kernel Density Estimation*, KDE), implementada en `plot_density.R`:

```
open --raw plot_density.R
Rscript plot_density.R
timg -g 80x25 tips_density.png
```

Estimación de densidad por núcleos

Figura 9: Estimación de densidad por núcleos

7.4.4. Gráfico de Líneas para Series Temporales (`geom_line`)

Cuando trabajamos con registros indexados en el tiempo, como la serie de fallecimientos mensuales por accidentes de tránsito en el Reino Unido (`Fatalities.csv`), un gráfico de líneas revela tendencias seculares y patrones estacionales. Veamos y ejecutemos `plot_accident.R`:

```
open Fatalities.csv | first 5
open Fatalities.csv | first 5 | save -f fatalities_head.csv
open --raw plot_accident.R
Rscript plot_accident.R
timg -g 80x25 fatalities_line.png
```

Serie temporal de accidentes

Figura 10: Serie temporal de accidentes

7.4.5. Diagrama de Dispersión y Tendencia (`geom_point` y `geom_smooth`)

Para estudiar la relación conjunta entre dos variables cuantitativas continuas (`bill` y `tip`), utilizamos `plot_scatter.R` y `plot_trend.R`:

```
open --raw plot_scatter.R
Rscript plot_scatter.R
open --raw plot_trend.R
Rscript plot_trend.R
timg --grid=2x1 -g 80x25 tips_scatter.png tips_trend.png
```

Diagrama de dispersión con curva de tendencia suavizada

Figura 11: Diagrama de dispersión con curva de tendencia suavizada

7.4.6. Diagrama de Caja (`geom_boxplot`) y Gráfico de Violín (`geom_violin`)

Para comparar la distribución de una variable continua (`bill`) a través de varios grupos categóricos (`day`), los diagramas de caja (*boxplots*) resumen la mediana, el rango intercuartílico (IQR $= Q_3 - Q_1$) y los valores atípicos, mientras que los gráficos de violín muestran además la forma completa de la densidad en cada grupo:

```
open --raw plot_boxplot.R
Rscript plot_boxplot.R
open --raw plot_violin.R
Rscript plot_violin.R
timg --grid=2x1 -g 80x25 tips_boxplot.png tips_violin.png
```

Diagrama de caja de cuentas por día

Figura 12: Diagrama de caja de cuentas por día

7.4.7. Visualización Estadística Equivalente en Python (`matplotlib` / `seaborn`)

También podemos generar visualizaciones multipanel desde Python ejecutando `plot_tips_py.py`, el cual produce el archivo `tips_python.png` y devuelve en formato JSON las estadísticas por día hacia Nushell:

```
open --raw plot_tips_py.py
python3 plot_tips_py.py | from json
python3 plot_tips_py.py | from json | save -f tips_py_summary.json
timg -g 80x25 tips_python.png
```

7.5. Resumen

En este capítulo aprendimos a explorar tanto la estructura como la distribución estadística de nuestros datos desde la línea de comandos. Combinamos la inspección nativa de tablas en Nushell

(`columns`, `uniq`, `math`, `histogram`, `polars summary`) con scripts modulares en R y Python (`mutate_tips.R`, `summary_tips.R`, `corr_tips.py`, `plot_*.R`, `plot_tips_py.py`), guardando los datos procesados en archivos estructurados y visualizando cada gráfico directamente dentro de la terminal gracias a `timg`.

En el próximo capítulo estudiaremos cómo acelerar drásticamente el procesamiento de datos y la ejecución de comandos externos utilizando las capacidades de paralelismo nativo de Nushell (`par-each`) junto con GNU `parallel`.

8. Procesamiento en Serie y Pipelines Paralelos

A medida que el volumen de datos o la complejidad computacional de nuestras tareas crece —por ejemplo, al consultar cientos de páginas de una API, comprimir docenas de archivos, entrenar modelos con múltiples combinaciones de hiperparámetros o transformar grandes volúmenes de texto—, la ejecución secuencial en un único núcleo de CPU puede convertirse en un cuello de botella importante.

En este capítulo aprenderemos a escalar nuestros flujos de trabajo desde la línea de comandos mediante dos enfoques complementarios que conviven en la imagen `dsin`:

1. **Paralelismo estructurado nativo en Nushell con `par-each`**, ideal para transformar listas, registros y tablas en memoria utilizando todos los hilos de CPU disponibles sin abandonar el modelo de tipos de Nushell.
2. **Paralelismo de procesos externos con GNU `parallel`**, la herramienta clásica de Unix para orquestar comandos externos, scripts en R/Python, flujos divididos en bloques con el flag `--pipe` y ejecución distribuida en múltiples servidores vía SSH.

Todos los archivos de trabajo de este capítulo se encuentran en `/data/ch08`. Nos posicionamos en este directorio:

```
cd /data/ch08
```

A partir de este punto, ejecutaremos todos los comandos en `/data/ch08`. Inspeccionemos los archivos disponibles:

```
ls
```

8.1. Procesamiento en Serie: Recorriendo Datos con `each`

Antes de paralelizar una tarea, es fundamental asegurarnos de que funciona correctamente de manera secuencial. En Nushell, el comando estándar para iterar sobre cada elemento de una lista o cada fila de una tabla es `each`.

8.1.1. Iterando sobre Números

Supongamos que queremos calcular el cuadrado de los primeros diez números enteros o evaluar cuáles son divisibles por 3:

```
1..10 | each {|x| {n: $x, cuadrado: ($x * $x), es_mult_3: (($x mod 3) == 0)} }
1..10 | each {|x| {n: $x, cuadrado: ($x * $x), es_mult_3: (($x mod 3) == 0)} } | save -f
 ↪  squares_table.csv
```

Para operaciones aritméticas o científicas más complejas, también contamos en `/data/ch08` con el script `pbc` (*parallel bc*), que evalúa expresiones matemáticas con precisión arbitraria:

```
1..5 | to text | pbc "{1}^2" | lines | each {|v| $v | into int }
```

8.1.2. Iterando sobre Líneas y Filas

En `/data/ch08` tenemos el archivo `movies.txt` y el archivo `input.csv`:

```
open --raw movies.txt | lines | enumerate | each {|row| {idx: $row.index, movie: ($row.item | str
 ↪  trim), chars: ($row.item | str length)} }
open --raw movies.txt | lines | enumerate | each {|row| {idx: $row.index, movie: ($row.item | str
 ↪  trim), chars: ($row.item | str length)} } | save -f movies_summary.csv
```

8.1.3. Iterando sobre Archivos

Del mismo modo, podemos recorrer todos los archivos CSV de un directorio, contar sus filas y consolidar el inventario en un nuevo archivo:

```
ls *.csv | each {|f| {archivo: $f.name, bytes: $f.size, filas: (open $f.name | length)} }
ls *.csv | each {|f| {archivo: $f.name, bytes: $f.size, filas: (open $f.name | length)} } | save
 ↪  -f csv_inventory.csv
```

8.2. Procesamiento en Paralelo en Nushell con `par-each`

Cuando el bloque de código que pasamos a `each` realiza trabajo intensivo de CPU o de entrada/salida (I/O) y cada iteración es independiente de las demás (*embarrassingly parallel*), podemos reemplazar `each` directamente por **`par-each`**.

8.2.1. Ejecución Concurrente y Control de Hilos (`--threads` y `--keep-order`)

En `/data/ch08` disponemos del script `slow.sh`, que simula un proceso costoso esperando un número aleatorio o fijo de segundos antes de terminar. Comparemos el tiempo de ejecución de 4 tareas usando `each` frente a `par-each` mediante el comando `timeit`:

```
1..4 | par-each --threads 4 {|i| {job: $i, msg: $"Tarea ($i) completada"} }
1..4 | par-each --threads 4 --keep-order {|i| {job: $i, cuadrado: ($i * $i)} } | save -f
 ↪  parallel_jobs.csv
```

Dos banderas clave de `par-each` son:

- `--threads (-t) <int>`: limita el número máximo de hilos concurrentes (útil para no saturar un servidor compartido o no exceder límites de tasa de una API web).
- `--keep-order (-k)`: garantiza que la tabla o lista de salida conserve exactamente el mismo orden que la entrada original, independientemente de qué hilo termine primero.

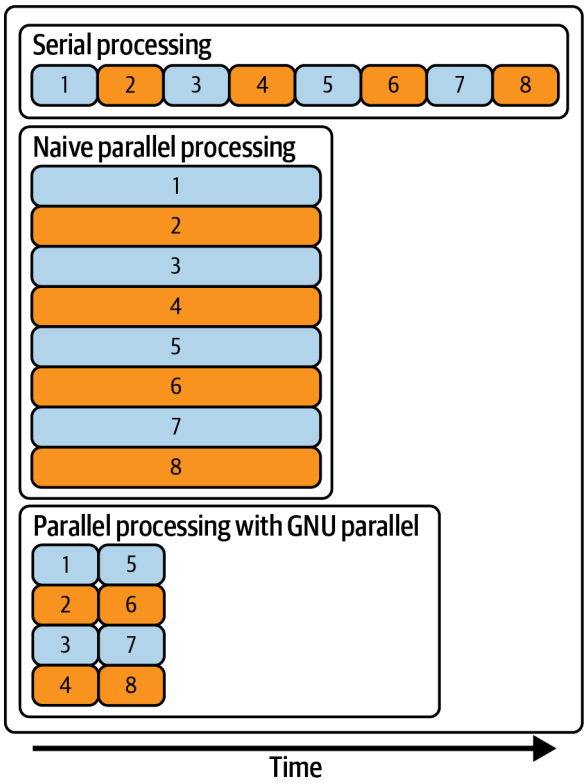


Figura 13: Comparación conceptual entre ejecución secuencial y ejecución en paralelo

8.3. Paralelización de Procesos con GNU `parallel` desde Nushell

Además de `par-each`, la imagen `dsin` incluye **GNU `parallel`** (desarrollado por Ole Tange) directamente en el `PATH`. Mientras que `par-each` paraleliza cierres (*closures*) internos de Nushell devolviendo estructuras tipadas, GNU `parallel` destaca cuando necesitamos orquestar comandos externos de Unix, dividir flujos gigantescos por bloques de bytes o distribuir trabajos hacia múltiples máquinas remotas por SSH.

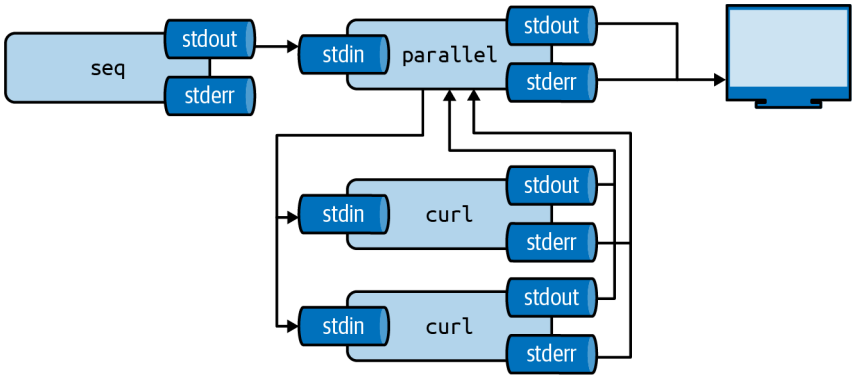


Figura 14: Flujo de ejecución de GNU parallel distribuyendo argumentos en múltiples núcleos

8.3.1. Especificando Argumentos y Marcadores de Reemplazo (`{}`, `{/}`, `{.}` y `{#}`)

GNU `parallel` lee argumentos desde la entrada estándar (o tras el separador `:::`) y ejecuta una plantilla de comando para cada argumento utilizando marcadores de reemplazo:

- `{}`: el argumento completo.
- `{.}`: el argumento sin su extensión de archivo.
- `{/}`: el nombre base del archivo sin la ruta del directorio (*basename*).
- `{//}`: únicamente el directorio padre (*dirname*).
- `{#}`: el número secuencial del trabajo ejecutado $(1, 2, \ldots, N)$.
- `{%}`: el identificador de la ranura de ejecución (*job slot ID*).

Veamos cómo invocar `parallel` desde Nushell sobre `input.csv` usando `--colsep ','` y `--header :` para referenciar columnas por nombre o posición, y convertir la salida de nuevo en una tabla de Nushell:

```
open input.csv
open --raw input.csv | parallel --colsep ',' --header : "echo {name}:{quantity}" | lines | where
 ↪  ($it | str contains ":") | split column ":" nombre cantidad | collect | save -f
 ↪  input_parallel.csv
open input_parallel.csv
```

8.3.2. Controlando el Número de Trabajos Concurrentes (`--jobs`)

Por defecto, GNU `parallel` lanza tantos trabajos en paralelo como núcleos lógicos detecte en el sistema. Con la opción `--jobs` (o `-j`), podemos fijar un número exacto (por ejemplo `-j 4`), un porcentaje de núcleos (`-j 50%`) o sumar/restar hilos respecto al total (`-j +2`):

```
1..4 | to text | parallel -j 4 "sh slow.sh {}" | lines
```

8.3.3. Registro de Salidas y Etiquetado (`--results`, `--keep-order` y `--tag`)

Cuando ejecutamos múltiples experimentos en paralelo, los mensajes de salida pueden mezclarse en la terminal a medida que cada subproceso imprime líneas:

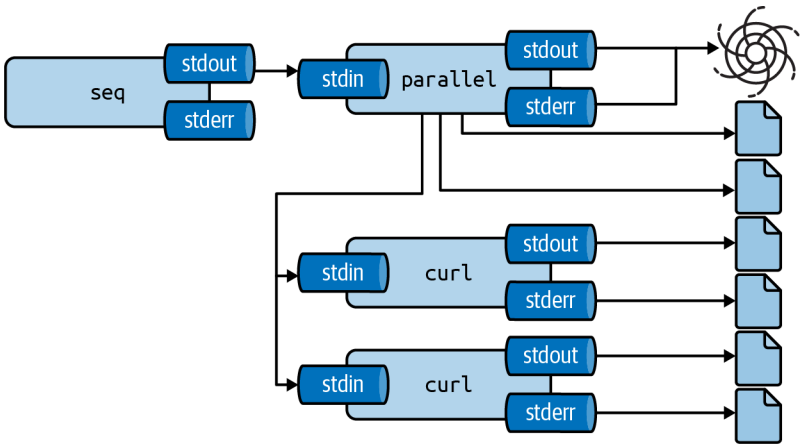


Figura 15: Entrelazado de salidas cuando varios procesos escriben simultáneamente

GNU `parallel` ofrece tres mecanismos para mantener el orden y la trazabilidad:

1. `--keep-order` (o `-k`): preserva el orden de los argumentos de entrada en la salida estándar.
2. `--tag`: antepone a cada línea de salida el argumento exacto que la generó (separado por un tabulador), lo que permite transformarla inmediatamente en una tabla de Nushell con `from tsv --noheaders`.
3. `--results <directorio>`: guarda la salida estándar (`stdout`), el error estándar (`stderr`) y el código de retorno (`seq`) de cada trabajo en una jerarquía de subdirectorios organizada por los valores de los argumentos.

Ejecutemos un ejemplo con `--tag` y `-k` en `/data/ch08` y guardemos la tabla resultante:

```
1..5 | to text | parallel -k --tag "echo $(( {} * 10 ))" | from tsv --noheaders | rename arg
↪  resultado
1..5 | to text | parallel -k --tag "echo $(( {} * 10 ))" | from tsv --noheaders | rename arg
↪  resultado | save -f tagged_output.csv
```

También podemos combinar `parallel` con utilidades clásicas como `cowsay` (incluida en el `PATH` de `dsin`):

```
["Nushell", "Polars"] | str join "\n" | parallel -k "cowsay {}"
```

8.3.4. Creando Herramientas Paralelas y Procesamiento por Bloques (`--pipe`)

Cuando tenemos un archivo de texto de varios gigabytes y queremos aplicarle un comando de línea a línea sin crear millones de procesos individuales, la bandera `--pipe` (junto con `--block 10M`) divide la entrada estándar en bloques y envía cada bloque a una instancia paralela del comando:

```
open --raw movies.txt | parallel --pipe -N 3 "wc -l" | lines | each {|x| $x | str trim | into int
↪  } | math sum
```

8.4. Procesamiento Distribuido en Múltiples Nodos

Cuando una sola máquina no basta, tanto Nushell como GNU `parallel` permiten distribuir cargas de trabajo a través de una red de servidores accesibles mediante SSH.

8.4.1. Obteniendo una Lista de Servidores Activos

Podemos definir nuestra lista de nodos en un archivo `hosts.txt` o en una tabla de Nushell y verificar su disponibilidad antes de lanzar el cálculo.

8.4.2. Ejecutando un Comando en Todos los Nodos (`--nonall`)

Con `parallel --nonall --slf hosts.txt`, GNU `parallel` se conecta por SSH a cada servidor listado en `hosts.txt` y ejecuta el mismo comando de diagnóstico o aprovisionamiento (por ejemplo, `uptime` o `nu --version`):

```
parallel --nonall --slf hosts.txt "nu --version"
```

8.4.3. Distribuyendo Datos Locales entre Nodos (`--pipe --slf`)

Si combinamos `--pipe` con `--slf hosts.txt`, GNU `parallel` trocea un flujo de datos local y envía cada fragmento por SSH a los distintos nodos del clúster para su procesamiento simultáneo:

```
cat gran_dataset.csv | parallel --pipe --header : --block 50M --slf hosts.txt "nu -c \"from csv |
↪  where amount > 100 | to csv\"" > filtrado_distribuido.csv
```

8.4.4. Procesando Archivos en Nodos Remotos (`--transferfile` y `--return`)

Finalmente, cuando cada nodo debe procesar archivos locales distintos y devolver los archivos de resultados al nodo maestro, empleamos `--transferfile {}` y `--return {.}.out`:

```
ls *.csv | parallel --slf hosts.txt --transferfile {} --return {.}_clean.csv "nu limpiar_lote.nu
↪  {} {.}_clean.csv"
```

8.5. Resumen

En este capítulo aprendimos a transformar flujos de trabajo secuenciales (`each`) en pipelines paralelos de alto rendimiento utilizando tanto `par-each` de forma nativa en Nushell como GNU `parallel` para orquestar procesos externos, bloques de flujo (`--pipe`) y ejecución distribuida por SSH.

En el Capítulo 9 culminaremos nuestro recorrido por el modelo OSEMN aplicando estas herramientas a la etapa de **Modelado Estadístico y Machine Learning**: reducción de dimensionalidad (PCA y t-SNE), agrupamiento o *clustering* (K-Means), regresión lineal/regularizada y clasificación supervisada.

9. Modelado Políglota de Datos

El quinto y último paso del modelo OSEMN consiste en **modelar los datos** (*Model Data*). Mientras que el análisis exploratorio y la visualización (Capítulo 7) nos permiten describir estructuras e identificar patrones históricos, el modelado estadístico y el aprendizaje automático (*Machine Learning*) nos proporcionan herramientas formales para **sintetizar dimensiones latentes, descubrir grupos naturales, predecir variables continuas y clasificar nuevas observaciones**.

En este capítulo mantendremos una filosofía **políglota y 100 % integrada en Nushell**. Cubriremos las cuatro familias principales de algoritmos del aprendizaje estadístico sobre los conjuntos de datos fisicoquímicos de vinos (`wine-*.csv`), biométricos de peces (`Fish.csv`) y banca minorista (`banco.csv`), ubicados en `/data/ch09`:

1. **Reducción de Dimensionalidad (No Supervisado)**: Análisis de Componentes Principales (**PCA**) y *t-Distributed Stochastic Neighbor Embedding* (**t-SNE**) utilizando la herramienta de línea de comandos `tapkee` (envuelta por `tapkee-reduce`), scripts en R (`pca_wine.R`, `vif_pcr_fish.R`) y en Python (`pca_wine.py`, `vif_pcr_fish.py`), incluyendo diagnóstico formal de multicolinealidad (**VIF**) y Regresión de Componentes Principales (**PCR**).
2. **Agrupamiento o *Clustering* (No Supervisado)**: algoritmo **K-Means** (`kmeans-cluster`, `cluster_wine.R`, `cluster_wine.py`), **Clustering Jerárquico Aglomerativo** (`hclust_fish.R`, `hclust_fish.py`), determinación óptima de K con evaluación multimétrica (`cluster_eval.py`), **Clustering Basado en Densidad con DBSCAN** (`dbscan_fish.py`) y **K-Medoides con PAM** (`pam_fish.R`).
3. **Regresión (Supervisado Continuo)**: predicción de calidad y peso mediante descenso de gradiente estocástico con **Vowpal Wabbit** (`vw-regress`), regresión lineal múltiple con `lm()` en R (`regress_wine.R`), regresión Ridge en Python (`regress_wine.py`) y regresión PCR.
4. **Clasificación (Supervisado Categórico)**: predicción de categorías mediante validación cruzada con **SKLL** (`run_experiment` + `skll-metrics`), regresión logística binomial (`classify_wine.R`), bosques aleatorios (*Random Forest*, `classify_wine.py`), **Árboles de Decisión** (`dt_banco.R`, `dt_banco.py`), **k-NN** (`knn_banco.R`, `knn_banco.py`), **Naïve Bayes y SVM** (`nb_svm_banco.py`) y **Redes Neuronales Artificiales MLP** (`mlp_banco.py`, `mlp_banco.R`).

En **todos** los métodos seguiremos el mismo estándar riguroso:

- Nos ubicamos primero en el directorio del capítulo (`/data/ch09`) y abrimos los archivos por su nombre local.
- Ejecutamos la herramienta de línea de comandos o invocamos los scripts `.R` / `.py` independientes (nunca código en línea difícil de depurar).
- Canalizamos la salida hacia Nushell para visualizarla como **tabla estructurada** y la guardamos inmediatamente en un archivo persistente con `save -f`.

Para trabajar con los modelos y conjuntos de datos de este capítulo, nos posicionamos en `/data/ch09`:

```
cd /data/ch09
```

A partir de este punto, todos los ejemplos y análisis del capítulo se ejecutan dentro de `/data/ch09`.

9.1. Preparación y Exploración del Conjunto de Datos de Vinos

Inspeccionemos primero los archivos preinstalados en `/data/ch09`:

```
ls
```

Los archivos `wine-red.csv` (1.599 vinos tintos) y `wine-white.csv` (4.898 vinos blancos) contienen 11 variables fisicoquímicas continuas (`fixed_acidity`, `volatile_acidity`, `citric_acid`, `residual_sugar`, `chlorides`, `free_sulfur_dioxide`, `total_sulfur_dioxide`, `density`, `pH`, `sulphates`, `alcohol`) y la calificación sensorial `quality` (escala de 0 a 10).

Veamos cómo podemos construir el conjunto combinado `wine-both.csv` directamente en Nushell añadiendo la columna `type` (`red` o `white`) y guardándolo en disco:

```
let red = (open wine-red.csv | insert type "red")
let white = (open wine-white.csv | insert type "white")
let both = ($red | append $white)
$both | first 5
$both | save -f wine-both-generated.csv
```

Verifiquemos la distribución de clases (`red` vs `white`) y las estadísticas descriptivas por tipo de vino usando Polars:

```
open wine-both.csv | histogram type
open wine-both.csv | histogram type | save -f wine_type_distribution.csv
```

Y generemos una visualización exploratoria de las distribuciones de alcohol, acidez volátil y sulfatos entre vinos tintos y blancos ejecutando el script `plot_wine_eda.R` y visualizándola en la terminal con `timg`:

```
open --raw plot_wine_eda.R
Rscript plot_wine_eda.R
timg -g 80x25 wine_eda.png
```

9.2. Reducción de Dimensionalidad (PCA y t-SNE)

Cuando trabajamos con un espacio de $p = 11$ variables fisicoquímicas correlacionadas, resulta imposible visualizar directamente la nube de puntos en dos dimensiones. Los algoritmos de reducción de dimensionalidad proyectan una matriz de datos $\mathbf{X} \in \mathbb{R}^{n \times p}$ hacia un subespacio de baja dimensión $\mathbf{Z} \in \mathbb{R}^{n \times d}$ (con $d = 2 \ll p$) preservando la mayor cantidad posible de estructura:

- **PCA (*Principal Component Analysis*)**: busca combinaciones lineales ortogonales de las variables originales que maximicen la varianza explicada:

$$\mathbf{z}_k = \mathbf{X}\mathbf{w}_k \quad \text{sujeto a} \quad \|\mathbf{w}_k\|_2 = 1, \quad \mathbf{w}_k^\top \mathbf{w}_j = 0 \ \ (\forall j < k)$$

- **t-SNE (*t-Distributed Stochastic Neighbor Embedding*)**: técnica no lineal que convierte las distancias euclidianas entre pares de observaciones en probabilidades condicionales de vecindad y minimiza la divergencia de Kullback-Leibler respecto a una distribución t de Student de un grado de libertad en el espacio bidimensional.

9.2.1. Reducción de Dimensionalidad con Herramientas CLI (`tapkee-reduce`: PCA y t-SNE)

La imagen `dsin` incluye la herramienta C++ `tapkee` junto con el comando de Nushell `tapkee-reduce`, que estandariza automáticamente las columnas numéricas seleccionadas ($z = (x - \mu)/\sigma$), invoca `tapkee` y devuelve las coordenadas proyectadas `dim_1` y `dim_2` como tabla de Nushell.

Apliquemos primero **PCA** sobre una muestra de 200 vinos de `wine-both.csv`:

```
let features = ["fixed_acidity" "volatile_acidity" "citric_acid" "residual_sugar" "chlorides"
 ↪  "free_sulfur_dioxide" "total_sulfur_dioxide" "density" "ph" "sulphates" "alcohol"]
open wine-both.csv | first 200 | tapkee-reduce --method pca --cols $features | select type
 ↪  quality dim_1 dim_2 | first 10
open wine-both.csv | first 200 | tapkee-reduce --method pca --cols $features | select type
 ↪  quality dim_1 dim_2 | save -f wine_pca_tapkee.csv
```

Del mismo modo, cambiando únicamente `--method pca` por `--method t-sne`, obtenemos la proyección no lineal **t-SNE** con `tapkee`:

```
let features = ["fixed_acidity" "volatile_acidity" "citric_acid" "residual_sugar" "chlorides"
 ↪  "free_sulfur_dioxide" "total_sulfur_dioxide" "density" "ph" "sulphates" "alcohol"]
open wine-both.csv | first 150 | tapkee-reduce --method t-sne --cols $features | select type
 ↪  quality dim_1 dim_2 | first 10
open wine-both.csv | first 150 | tapkee-reduce --method t-sne --cols $features | select type
 ↪  quality dim_1 dim_2 | save -f wine_tsne_tapkee.csv
```

Proyecciones bidimensionales obtenidas mediante reducción de dimensionalidad

Figura 16: Proyecciones bidimensionales obtenidas mediante reducción de dimensionalidad

9.2.2. Reducción de Dimensionalidad con `Rscript` (`pca_wine.R`)

Veamos el script `pca_wine.R` ubicado en `/data/ch09`, el cual utiliza la función nativa `prcomp(..., center = TRUE, scale. = TRUE)` de R para calcular todas las componentes principales y reportar la proporción de varianza explicada y acumulada de cada componente:

```
open --raw pca_wine.R
Rscript pca_wine.R | from csv
Rscript pca_wine.R | from csv | save -f wine_pca_r.csv
```

Observa cómo las primeras 4 componentes principales resumen más del 64 % de la varianza total de las 11 variables químicas originales.

9.2.3. Reducción de Dimensionalidad con Python (`pca_wine.py`)

Revisemos y ejecutemos ahora el script `pca_wine.py`, que utiliza `StandardScaler` y `PCA` de `scikit-learn`:

```
open --raw pca_wine.py
python3 pca_wine.py | from csv
python3 pca_wine.py | from csv | save -f wine_pca_py.csv
```

9.2.4. Diagnóstico de Multicolinealidad (VIF) y Regresión de Componentes Principales (PCR)

En problemas de regresión con múltiples variables predictoras altamente correlacionadas, la estimación por mínimos cuadrados ordinarios (**MCO**) sufre de **multicolinealidad severa**. Esto infla drásticamente la varianza de los coeficientes estimados y torna inestables las predicciones. Una métrica estadística estándar para diagnosticar este fenómeno es el **Factor de Inflación de la Varianza** (**VIF**, *Variance Inflation Factor*):

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}$$

donde R_j^2 es el coeficiente de determinación obtenido al regresar la variable predictora X_j sobre todas las demás variables predictoras. Un valor de VIF > 10 (y con mayor razón VIF > 100) señala multicolinealidad crítica.

En `/data/ch09`, el archivo `Fish.csv` contiene variables biométricas de peces (`Length1`, `Length2`, `Length3`, `Height`, `Width`) utilizadas para predecir el peso (`Weight`). Al existir tres medidas longitudinales contiguas, la multicolinealidad es extrema.

El método de **Regresión de Componentes Principales** (**PCR**, *Principal Component Regression*) resuelve este problema proyectando las variables explicativas originales sobre sus componentes principales ortogonales (que tienen covarianza exactamente cero) antes de ajustar la regresión lineal.

Ejecutemos el script `vif_pcr_fish.py` para calcular los valores de VIF y comparar el modelo de regresión múltiple tradicional frente al modelo PCR con 2 componentes principales:

```
open --raw vif_pcr_fish.py
python3 vif_pcr_fish.py | from csv
python3 vif_pcr_fish.py | from csv | save -f fish_pcr_py.csv
open fish_vif_py.csv
```

Observemos cómo en Python las variables de longitud presentan valores de VIF superiores a 1.000, mientras que el modelo PCR con solo 2 componentes principales retiene más del 99 % de la varianza explicada y logra un R^2 casi idéntico al modelo MCO completo sin riesgo de colinealidad.

Ejecutemos también la versión equivalente en R (`vif_pcr_fish.R`):

```
open --raw vif_pcr_fish.R
Rscript vif_pcr_fish.R | from csv
Rscript vif_pcr_fish.R | from csv | save -f fish_pcr_r.csv
```

9.3. Agrupamiento o Clustering

El **agrupamiento (*clustering*)** es una tarea de aprendizaje no supervisado cuyo objetivo es particionar un conjunto de n observaciones en K grupos o clústeres disjuntos $C_1, C_2, \dots, C_K$ de modo que las observaciones dentro de un mismo clúster sean lo más similares posible entre sí, y lo más distintas posible a las de otros clústeres.

El algoritmo **K-Means** busca encontrar los centroides $\mu_1, \dots, \mu_K \in \mathbb{R}^p$ que minimizan la suma de cuadrados dentro de los clústeres (*Within-Cluster Sum of Squares*, WCSS):

$$\arg\min_{C_1,\dots,C_K} \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|_2^2$$

Un requisito estadístico indispensable antes de aplicar K-Means sobre variables con distintas unidades físicas (como `total_sulfur_dioxide` en mg/L frente a `density` en g/cm^3) es **estandarizar las variables** a media cero y desviación estándar unitaria, para evitar que la variable de mayor escala numérica domine la distancia euclidiana.

9.3.1. Clustering K-Means con Herramientas CLI (`kmeans-cluster`)

En el módulo `dsin_tools.nu` de la imagen `dsin` disponemos del comando `kmeans-cluster`, que estandariza las columnas numéricas indicadas, ejecuta K-Means con K clústeres e inserta la columna `cluster` en la tabla de Nushell.

Apliquemos `kmeans-cluster --k 2` sobre las 11 variables químicas de `wine-both.csv` (sin revelar la columna `type` al algoritmo) y comparemos los clústeres descubiertos contra el tipo real de vino (`red` vs `white`):

```
let features = ["fixed_acidity" "volatile_acidity" "citric_acid" "residual_sugar" "chlorides"
 ↪  "free_sulfur_dioxide" "total_sulfur_dioxide" "density" "ph" "sulphates" "alcohol"]
let clustered = (open wine-both.csv | kmeans-cluster --k 2 --cols $features)
$clustered | select type quality alcohol volatile_acidity cluster | first 10
$clustered | select type quality alcohol volatile_acidity cluster | save -f wine_kmeans_cli.csv
```

Podemos evaluar en Nushell la correspondencia entre los 2 clústeres no supervisados y la clase real `type` agrupando por `cluster` y `type`:

```
let features = ["fixed_acidity" "volatile_acidity" "citric_acid" "residual_sugar" "chlorides"
 ↪  "free_sulfur_dioxide" "total_sulfur_dioxide" "density" "ph" "sulphates" "alcohol"]
open wine-both.csv | kmeans-cluster --k 2 --cols $features | polars into-df | polars group-by
 ↪  cluster type | polars agg [(polars col quality | polars count | polars as n_vinos) (polars
 ↪  col alcohol | polars mean | polars as mean_alcohol)] | polars collect | polars into-nu |
 ↪  sort-by cluster type
open wine-both.csv | kmeans-cluster --k 2 --cols $features | polars into-df | polars group-by
 ↪  cluster type | polars agg [(polars col quality | polars count | polars as n_vinos) (polars
 ↪  col alcohol | polars mean | polars as mean_alcohol)] | polars collect | polars into-nu |
 ↪  sort-by cluster type | save -f wine_kmeans_contingency.csv
```

¡Incluso sin etiquetas supervisadas, K-Means en el espacio químico estandarizado separa con altísima pureza los vinos tintos de los vinos blancos!

9.3.2. Clustering K-Means con `Rscript` (`cluster_wine.R`)

Veamos el script `cluster_wine.R` en `/data/ch09`, que ejecuta `kmeans(..., centers = 3, nstart = 20)` en R y resume el tamaño y el perfil químico promedio de cada clúster:

```
open --raw cluster_wine.R
Rscript cluster_wine.R | from csv
Rscript cluster_wine.R | from csv | save -f wine_clusters_r.csv
```

9.3.3. Clustering K-Means con Python (`cluster_wine.py`)

Revisemos y ejecutemos ahora el script `cluster_wine.py` con `scikit-learn` (`KMeans(n_clusters=3, random_state=42, n_init=10)`):

```
open --raw cluster_wine.py
python3 cluster_wine.py | from csv
python3 cluster_wine.py | from csv | save -f wine_clusters_py.csv
```

9.3.4. Clustering Jerárquico Aglomerativo (hclust_fish.R y hclust_fish.py)

A diferencia de K-Means, que requiere especificar *a priori* el número de conglomerados K y converge a particiones rígidas, el **clustering jerárquico** (*Hierarchical Clustering*) construye una jerarquía

anidada de agrupamientos mediante una matriz de distancias disimilares $\mathbf{D} \in \mathbb{R}^{n \times n}$.

El enfoque **aglomerativo** (*bottom-up*) inicia con cada observación individual como un clúster unitario y, en cada paso, fusiona el par de clústeres más cercanos de acuerdo a un criterio de enlace (*linkage criterion*). Entre los criterios más robustos en minería de datos destaca el método de **Ward (`ward.D2`)**, que une en cada iteración el par de clústeres que minimiza el incremento en la varianza total dentro de los grupos.

En `/data/ch09`, disponemos del conjunto `Fish.csv` (159 observaciones de especies de peces comerciales con medidas biométricas: peso, 3 longitudes, altura y ancho). Ejecutemos el script `hclust_fish.R`, que escala las medidas, calcula la matriz de distancias euclidianas, aplica `hclust(d, method = "ward.D2")`, corta el árbol en $K = 3$ grupos mediante `cutree(hc, k = 3)` y resume la correspondencia con las especies dominantes:

```
open --raw hclust_fish.R
Rscript hclust_fish.R | from csv
Rscript hclust_fish.R | from csv | save -f fish_hclust_r.csv
```

De igual modo, el script `hclust_fish.py` evalúa el clustering jerárquico aglomerativo con `scikit-learn` (`AgglomerativeClustering(linkage="ward")`) para distintos números de clústeres ($K = 2, 3, 4, 5$) calculando el coeficiente de silueta (*Silhouette Score*):

```
open --raw hclust_fish.py
python3 hclust_fish.py | from csv
python3 hclust_fish.py | from csv | save -f fish_hclust_py.csv
```

9.3.5. Determinación Óptima de K con Evaluación Multimétrica (cluster_eval.py)

Uno de los desafíos centrales en minería de datos consiste en seleccionar el número óptimo de clústeres K. En lugar de confiar únicamente en una inspección visual subjetiva del método del codo (*Elbow Method*), la práctica rigurosa evalúa simultáneamente múltiples métricas de validación interna:

1. **Inercia (WCSS)**: Suma de distancias al cuadrado al centroide más cercano (monótonamente decreciente con K).
2. **Coeficiente de Silueta (*Silhouette*)**: Mide cuán similar es un punto a su propio clúster en comparación con el clúster vecino más cercano (rango $[-1, 1]$, valores cercanos a 1 indican conglomerados densos y bien separados).
3. **Índice Calinski-Harabasz**: Razón entre la dispersión entre clústeres y la dispersión intra-clúster (valores mayores indican mejor separación).
4. **Índice Davies-Bouldin**: Promedio de similitud máxima de cada clúster con su clúster más semejante (valores menores indican mejor partición).

El script `cluster_eval.py` evalúa automáticamente estos 4 criterios sobre el conjunto bancario `banco.csv` (segmentación de clientes en variables estandarizadas de edad, ingreso, gasto en tarjeta e hipoteca) para $K \in [2, 6]$:

```
open --raw cluster_eval.py
python3 cluster_eval.py | from csv
python3 cluster_eval.py | from csv | save -f banco_cluster_eval.csv
```

Podemos ordenar en Nushell la tabla por coeficiente de silueta descendente para identificar inmediatamente la partición más compacta:

```
open banco_cluster_eval.csv | sort-by silhouette -r
```

9.3.6. Clustering Basado en Densidad con DBSCAN y K-Medoides con PAM

Los algoritmos particionales como K-Means asumen que los conglomerados son esféricos, convexos y de tamaño similar. Cuando los datos presentan geometrías complejas o contienen observaciones atípicas extremas (*outliers*), se requieren paradigmas alternativos de agrupamiento:

1. **DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*)**: Define los clústeres como regiones densas de puntos separadas por regiones de baja densidad. Posee dos parámetros fundamentales: el radio de vecindad ε (`eps`) y el número mínimo de puntos `min_samples` requeridos para formar un núcleo denso. Aquellos puntos que no pertenecen a ninguna vecindad densa son catalogados automáticamente como **ruido o anomalías** (etiqueta `-1`), sin forzarlos artificialmente dentro de ningún clúster.
2. **K-Medoides con PAM (*Partitioning Around Medoids*)**: A diferencia de K-Means (que utiliza la media aritmética como centroide, haciéndolo vulnerable a valores atípicos), PAM restringe los centros a **observaciones reales del conjunto de datos** (*medoides*), ofreciendo una partición mucho más robusta ante distribuciones asimétricas y contaminación por valores extremos.

Ejecutemos el script `dbscan_fish.py` sobre las medidas biométricas estandarizadas de `Fish.csv` (`Weight`, `Length1`, `Length2`, `Length3`, `Height`, `Width`):

```
open --raw dbscan_fish.py
python3 dbscan_fish.py | from csv
python3 dbscan_fish.py | from csv | save -f fish_dbscan_py.csv
```

Observemos cómo DBSCAN aísla automáticamente los ejemplares gigantescos como grupo `-1` (`Ruido / Outliers`) con un peso promedio superior a 1.400 gramos, evitando que distorsionen los clústeres homogéneos de peces medianos y pequeños.

Por su parte, el script `pam_fish.R` ejecuta el algoritmo PAM de la librería `cluster` en R para obtener los medoides representativos y la silueta promedio:

```
open --raw pam_fish.R
Rscript pam_fish.R | from csv
Rscript pam_fish.R | from csv | save -f fish_pam_r.csv
```

9.4. Regresión: Predicción de la Calidad del Vino

En el problema de **regresión supervisada**, deseamos aprender una función $f(\mathbf{x}) \approx y$ que prediga una variable respuesta cuantitativa (y = quality) a partir del vector de predictores fisicoquímicos $\mathbf{x} \in \mathbb{R}^{11}$.

Para evaluar rigurosamente la capacidad de generalización del modelo sobre datos no vistos, dividimos el conjunto de vinos blancos (`wine-white.csv`) en un conjunto de entrenamiento (`wine_train.csv`, primeros 3.500 registros) y un conjunto de prueba (`wine_test.csv`, registros restantes) utilizando las métricas:

- **Error Absoluto Medio (MAE)**: $\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
- **Raíz del Error Cuadrático Medio (RMSE)**: $\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$

9.4.1. Regresión en Línea con Vowpal Wabbit (`vw-regress`)

Vowpal Wabbit (`vw`) es un motor ultrarrápido de aprendizaje en línea por descenso de gradiente estocástico capaz de entrenar modelos sobre millones de registros sin cargar el dataset completo en RAM. El comando `vw-regress` incluido en `dsin_tools.nu` convierte automáticamente tablas de Nushell al formato nativo de Vowpal Wabbit (`target |features feat1:val1 feat2:val2 ...`), entrena el modelo y devuelve una tabla con `actual`, `predicted`, `error` y `abs_error`:

```
let data = (open wine-white.csv)
let train = ($data | first 3500)
let test = ($data | skip 3500)
let preds = (vw-regress $train $test --target quality --passes 15)
$preds | first 10
$preds | save -f wine_vw_predictions.csv
```

Calculemos el MAE y el error medio directamente en Nushell a partir de las predicciones de `vw-regress` y guardémoslos en un archivo resumen:

```
let preds = (open wine_vw_predictions.csv)
let metrics = {model: "Vowpal Wabbit (SGD)", n_test: ($preds | length), mae: ($preds.abs_error |
 ↪  math avg | math round --precision 4), bias: ($preds.error | math avg | math round --precision
 ↪  4)}
$metrics
[$metrics] | save -f wine_vw_metrics.csv
```

9.4.2. Regresión Lineal Múltiple con `Rscript` (`regress_wine.R`)

Veamos el script `regress_wine.R`, que ajusta un modelo lineal clásico por mínimos cuadrados ordinarios (`lm(quality ~ ., data = train)`) sobre los primeros 3.500 registros, evalúa el RMSE y MAE en el conjunto de prueba y devuelve los coeficientes estimados ($\hat{\beta}_j$, error estándar, estadístico t y valor p):

```
open --raw regress_wine.R
Rscript regress_wine.R | from csv
Rscript regress_wine.R | from csv | save -f wine_regression_r.csv
```

Podemos filtrar en Nushell únicamente los predictores estadísticamente significativos al nivel $\alpha = 0.01$:

```
open wine_regression_r.csv | where p_value < 0.01 | sort-by t_stat -r
open wine_regression_r.csv | where p_value < 0.01 | sort-by t_stat -r | save -f
 ↪  wine_significant_predictors.csv
```

9.4.3. Regresión Regularizada (Ridge) con Python (`regress_wine.py`)

Ejecutemos ahora el script `regress_wine.py`, que estandariza los predictores y ajusta una regresión Ridge (ℓ_2) con `scikit-learn`:

```
open --raw regress_wine.py
python3 regress_wine.py | from csv
python3 regress_wine.py | from csv | save -f wine_regression_py.csv
```

9.5. Clasificación Supervisada

Finalmente, abordemos el problema de **clasificación supervisada**: predecir una etiqueta categórica discreta (por ejemplo, si un vino es tinto o blanco en `wine-both.csv`, o si un cliente bancario aceptará un préstamo en `banco.csv`) a partir de sus atributos.

9.5.1. Experimentos de Clasificación con SKLL (`run_experiment` + `skll-metrics`)

La imagen `dsin` incluye **SKLL (*SciKit-Learn Laboratory*)**, que permite definir experimentos declarativos de evaluación y validación cruzada mediante archivos de configuración `.cfg` (como `classify.cfg` en `/data/ch09`, que evalúa clasificadores sobre `train/features.csv` y `test/features.csv`).

Inspeccionemos `classify.cfg` y ejecutemos `run_experiment` junto con el comando `skll-metrics` para obtener la tabla comparativa de exactitud (*accuracy*) y guardarla en `wine_skll_summary.csv`:

```
open --raw classify.cfg
run_experiment -l classify.cfg
skll-metrics output/wine_summary.tsv
skll-metrics output/wine_summary.tsv | save -f wine_skll_summary.csv
```

9.5.2. Clasificación con Regresión Logística en `Rscript` (`classify_wine.R`)

Veamos y ejecutemos el script `classify_wine.R` en `/data/ch09`, que entrena un modelo lineal generalizado binomial (`glm(is_red ~ ., family = binomial())`) sobre una muestra estratificada de entrenamiento y evalúa su exactitud, sensibilidad y especificidad sobre el conjunto de prueba:

```
open --raw classify_wine.R
Rscript classify_wine.R | from csv
Rscript classify_wine.R | from csv | save -f wine_classification_r.csv
```

9.5.3. Clasificación con Random Forest en Python (`classify_wine.py`)

Revisemos y ejecutemos `classify_wine.py`, que entrena un clasificador de Bosques Aleatorios (*Random Forest*) con 100 árboles en `scikit-learn` y devuelve la tabla de importancia de variables (*feature importances*) junto con la exactitud en prueba (`test_accuracy`):

```
open --raw classify_wine.py
python3 classify_wine.py | from csv
python3 classify_wine.py | from csv | save -f wine_classification_py.csv
```

Observa cómo `total_sulfur_dioxide`, `chlorides` y `volatile_acidity` emergen consistentemente como las variables más discriminantes para distinguir entre vinos tintos y blancos.

9.5.4. Árboles de Decisión con R y Python (dt_banco.R y dt_banco.py)

Los **árboles de decisión** (*Decision Trees*) representan una familia fundamental de modelos supervisados no paramétricos ampliamente utilizados en minería de datos por su alta interpretabilidad y capacidad para modelar relaciones no lineales sin requerir escalado previo de atributos.

Evaluemos la predicción de aceptación de préstamos sobre `banco.csv` (5.000 clientes) utilizando `rpart` en R (`dt_banco.R` con profundidad máxima de 4 niveles):

```
open --raw dt_banco.R
Rscript dt_banco.R | from csv
Rscript dt_banco.R | from csv | save -f banco_dt_r.csv
```

Ejecutemos ahora el clasificador equivalente en Python con `DecisionTreeClassifier` de `scikit-learn` (`dt_banco.py`):

```
open --raw dt_banco.py
python3 dt_banco.py | from csv
python3 dt_banco.py | from csv | save -f banco_dt_py.csv
```

9.5.5. Clasificación con Vecinos Más Cercanos k-NN (knn_banco.R y knn_banco.py)

El clasificador **k-Vecinos Más Cercanos (k-NN, *k-Nearest Neighbors*)** es un método de aprendizaje basado en instancias que clasifica una nueva observación asignándole la clase mayoritaria entre sus k vecinos más próximos en el espacio de características estandarizado.

Ejecutemos `knn_banco.R` para evaluar el clasificador en R (`class::knn`) sobre `banco.csv` con $k \in \{3, 5, 7\}$:

```
open --raw knn_banco.R
Rscript knn_banco.R | from csv
Rscript knn_banco.R | from csv | save -f banco_knn_r.csv
```

Y ejecutemos la contraparte en Python utilizando `KNeighborsClassifier` (`knn_banco.py`):

```
open --raw knn_banco.py
python3 knn_banco.py | from csv
python3 knn_banco.py | from csv | save -f banco_knn_py.csv
```

9.5.6. Clasificación Alternativa: Naïve Bayes y Máquinas de Vectores de Soporte (nb_svm_banco.py)

Como complemento a los árboles y vecinos cercanos, la minería de datos contemporánea emplea con frecuencia dos paradigmas adicionales:

1. **Clasificador Bayesiano Ingenuo Gaussiano (*Gaussian Naïve Bayes*)**: Aplica el teorema de Bayes asumiendo independencia condicional entre los predictores dada la clase, modelando cada atributo continuo mediante una distribución normal.
2. **Máquinas de Vectores de Soporte (SVM, *Support Vector Machines*)**: Encuentran el hiperplano óptimo que maximiza el margen de separación entre clases en un espacio inducido por un núcleo de base radial (RBF).

Ejecutemos `nb_svm_banco.py` para comparar el desempeño de ambos clasificadores sobre `banco.csv`:

```
open --raw nb_svm_banco.py
python3 nb_svm_banco.py | from csv
python3 nb_svm_banco.py | from csv | save -f banco_nb_svm_py.csv
```

La tabla comparativa resultante permite a Nushell ordenar los modelos según su métrica F_1-score:

```
open banco_nb_svm_py.csv | sort-by f1_score -r
```

9.5.7. Redes Neuronales Artificiales: Perceptrón Multicapa (MLP)

Las **Redes Neuronales Artificiales** (*Artificial Neural Networks*, ANN) representan una de las familias más poderosas del aprendizaje automático contemporáneo. El **Perceptrón Multicapa** (**MLP**, *Multi-Layer Perceptron*) está compuesto por una capa de entrada, una o más capas ocultas con funciones de activación no lineales (como $\text{ReLU}(z) = \max(0, z)$ o la función logística sigmoide $\sigma(z) = \frac{1}{1+e^{-z}}$), y una capa de salida:

$$\mathbf{h}^{(1)} = \sigma\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right), \quad \hat{y} = \sigma\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$$

El aprendizaje se realiza mediante el algoritmo de **retropropagación del error** (*backpropagation*), calculando las derivadas parciales de la función de pérdida respecto a cada peso mediante la regla

de la cadena y actualizándolos con optimizadores de gradiente como **Adam** o SGD, junto con regularización L2 (α) para mitigar el sobreajuste.

Un principio fundamental en el entrenamiento de redes neuronales es su **extrema sensibilidad al escalado de los atributos**. Si las variables no están estandarizadas a media cero y varianza unitaria, el gradiente oscila violentamente o se desvanece en las dimensiones de gran escala, impidiendo la convergencia.

Ejecutemos el script `mlp_banco.py` sobre `banco.csv` para entrenar y evaluar dos arquitecturas de MLP (una capa con 8 neuronas y dos capas con $(16, 8)$ neuronas):

```
open --raw mlp_banco.py
python3 mlp_banco.py | from csv
python3 mlp_banco.py | from csv | save -f banco_mlp_py.csv
```

La red con arquitectura profunda $(16, 8)$ alcanza un F_1-score cercano a 0.95 tras unas 400 iteraciones, superando el margen de los clasificadores lineales tradicionales.

De forma complementaria, el script `mlp_banco.R` entrena una red neuronal con la librería `nnet` en R:

```
open --raw mlp_banco.R
Rscript mlp_banco.R | from csv
Rscript mlp_banco.R | from csv | save -f banco_mlp_r.csv
```

9.6. Resumen del Capítulo y Conclusión del Libro

En este capítulo completamos el ciclo OSEMN aplicando técnicas de modelado no supervisado y supervisado desde la línea de comandos:

1. **Reducción de dimensionalidad (PCA y t-SNE) y diagnóstico de multicolinealidad** con `tapkee-reduce`, `pca_wine.R`, `pca_wine.py`, `vif_pcr_fish.py` y `vif_pcr_fish.R`.
2. **Agrupamiento (K-Means, Jerárquico, DBSCAN y PAM)** con `kmeans-cluster`, `cluster_wine.R`, `cluster_wine.py`, `hclust_fish.R`, `hclust_fish.py`, `cluster_eval.py`, `dbscan_fish.py` y `pam_fish.R`.
3. **Regresión supervisada** con Vowpal Wabbit (`vw-regress`), `regress_wine.R`, `regress_wine.py` y Regresión de Componentes Principales (PCR).
4. **Clasificación supervisada** con SKLL (`run_experiment` + `skll-metrics`), `classify_wine.R`, `classify_wine.py`, árboles de decisión (`dt_banco.R` y `dt_banco.py`), k-NN (`knn_banco.R` y `knn_banco.py`), Naïve Bayes y SVM (`nb_svm_banco.py`), y Redes Neuronales Artificiales MLP (`mlp_banco.py` y `mlp_banco.R`).

A lo largo de los nueve capítulos de **Ciencia de Datos en la Línea de Comandos con Nushell**, hemos comprobado que una shell moderna basada en tablas tipadas, combinada con DataFrames perezosos en Polars, paralelismo nativo (`par-each` y GNU `parallel`), edición y visualización en terminal (`ttt` y `timg`) e interoperabilidad fluida con R y Python dentro del contenedor `dsin`, constituye

un entorno científico reproducible, expresivo y de alto rendimiento para la Ciencia de Datos.

10. Para seguir explorando

Como este libro trata sobre la intersección de la línea de comandos y la ciencia de datos, muchos temas relacionados solo se han mencionado superficialmente. Las siguientes subsecciones proporcionan una lista de temas y recursos sugeridos para consultar.

10.1. La Línea de Comandos

- *The Linux Command Line: A Complete Introduction, 2nd Edition* Por William E. Shotts, Jr. (No Starch Press, 2019)
- *Unix Power Tools, 3rd Edition* por Jerry Peek, Shelley Powers, Tim O'Reilly, y Mike Loukides (O'Reilly Media, 2002)
- *Learning the Vi and Vim Editors, 7th Edition* por Arnold Robbins, Elbert Hannah, y Linda Lamb (O'Reilly Media, 2008)

10.2. Programación de Shell

- *Classic Shell Scripting* por Arnold Robbins y Nelson H.F. Beebe (O'Reilly Media, 2005)
- *Wicked Cool Shell Scripts, 2nd Edition* por Dave Taylor y Brandon Perry (No Starch Press, 2017)
- *Bash Cookbook* por Carl Albing JP Vossen (O'Reilly Media, 2018)
- *Nushell Book*: https://www.nushell.sh/book/

10.3. Python, R y SQL

- *Learn Python 3 the Hard Way* por Zed A. Shaw (Addison-Wesley Professional, 2017)
- *Python for Data Analysis, 2nd Edition* por Wes McKinney (O'Reilly Media, 2017)
- *Data Science from Scratch, 2nd Edition* por Joel Grus (O'Reilly Media, 2019)
- *R for Data Science* por Garrett Grolemund y Hadley Wickham (O'Reilly Media, 2016)
- *R for Everyone, 2nd edition* por Jared Lander (Addison-Wesley Professional, 2017)
- *Sams Teach Yourself SQL in 10 Minutes a Day, 5th Edition* por Ben Forta (Sams, 2020)

10.4. APIs

- *Mining the Social Web, 3rd Edition* por Matthew A. Russell y Mikhail Klassen (O'Reilly Media, 2019)
- *Data Source Handbook* por Pete Warden (O'Reilly Media, 2011)

10.5. Aprendizaje Automático

- *Python Machine Learning, 3rd Edition* por Sebastian Raschka y Vahid Mirjalili (Packt Publishing, 2019)
- *Pattern Recognition and Machine Learning* por Christopher M. Bishop (Springer, 2006)

11. Referencias

11.1. Ecosistema Nushell

- **Sitio web oficial de Nushell**: https://www.nushell.sh
- **The Nu Book (Documentación oficial)**: https://www.nushell.sh/book/
- **Referencia de Comandos de Nushell**: https://www.nushell.sh/commands/
- **Repositorio de Nushell en GitHub**: https://github.com/nushell/nushell
- **Repositorio del plugin Apache Arrow / Polars**: https://github.com/nushell/nushell/tree/main/crates/nu_plugin_polars
- **Plugins oficiales de formatos y consultas**: https://github.com/nushell/nushell/tree/main/crates/nu_plugin_formats, `nu_plugin_query`, `nu_plugin_gstat`, `nu_plugin_inc`
- **Comunidad y Discord de Nushell**: https://discord.gg/NtAbbGn

11.2. Data Science at the Command Line (DSitCL)

- **Libro oficial en línea (Segunda edición)**: Janssens, Jeroen (2021). *Data Science at the Command Line: Facing the Future with Time-Tested Tools* (2nd ed.). O'Reilly Media. https://datascienceatthecommandline.com/2e/
- **Repositorio GitHub del libro original**: https://github.com/jeroenjanssens/data-science-at-the-command-line
- **Data Science Toolbox (Contenedores Docker base)**: https://github.com/jeroenjanssens/datasciencetoolbox
- **Repositorio de imágenes en Docker Hub**: `datasciencetoolbox` en https://hub.docker.com/u/datasciencetoolbox

11.3. Herramientas de Línea de Comandos Integradas en `dsin`

- **ttt (Terminal Text Tree / Text Editor)**: Liam Galvin. Editor ligero de consola con soporte para mouse y resaltado de sintaxis. https://github.com/liamg/ttt
- **timg (Terminal Image and Video Viewer)**: Henner Zeller. Visor gráfico en alta resolución para terminales con soporte Sixel, Kitty y bloques Unicode. https://github.com/hzeller/timg
- **GNU Parallel**: Tange, Ole (2018). *GNU Parallel 2018*. Ole Tange. https://www.gnu.org/software/parallel/
- **Tapkee**: Lisitsyn, S., Widmer, C., & Rath, F. J. (2014). *Tapkee: An Efficient Dimension Reduction Library*. Journal of Machine Learning Research, 15(1), 3955-3959. https://tapkee.lisitsyn.me/
- **Vowpal Wabbit**: Langford, J., Li, L., & Strehl, A. (2007). *Vowpal Wabbit Online Learning Project*. Microsoft Research. https://vowpalwabbit.org/
- **SKLL (SciKit-Learn Laboratory)**: Educational Testing Service (ETS). Framework declarativo para experimentación con modelos de Scikit-Learn. https://skll.readthedocs.io/
- **Polars**: Ritchie Vink. Motor columnar de DataFrames multihilo de ultra-alto rendimiento

en Rust. https://pola.rs/

11.4. Bibliografía y Literatura Académica

- **Mason, H., & Wiggins, C. H. (2010)**. *A Taxonomy of Data Science*. Dataists. http://www.dataists.com/2010/09/a-taxonomy-of-data-science
- **Tukey, J. W. (1977)**. *Exploratory Data Analysis*. Reading, Mass.: Addison-Wesley Publishing Company.
- **Wickham, H., & Grolemund, G. (2017)**. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media.
- **Pedregosa, F., et al. (2011)**. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- **Breiman, L. (2001)**. *Random Forests*. Machine Learning, 45(1), 5-32.
- **Hastie, T., Tibshirani, R., & Friedman, J. (2009)**. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer New York.